

(continued from previous page)

```

ASSERT(cudaDeviceSynchronize() == cudaSuccess,
       "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
ASSERT(munmap(mapped, file_stat.st_size) == 0, "Cannot unmap file");
ASSERT(close(fd) == 0, "Cannot close file");
}

```

Note that on systems without the `hostNativeAtomicSupported` property (see *Host Native Atomics*) including systems with Linux HMM enabled, atomic accesses to file-backed memory are not supported.

4.1.1.1.2 Inter-Process Communication (IPC) with Unified Memory

Note

As of now, using IPC with unified memory can have significant performance implications.

Many applications prefer to manage one GPU per process, but still need to use unified memory, for example for over-subscription, and access it from multiple GPUs.

CUDA IPC (see *Interprocess Communication*) does not support managed memory: handles to this type of memory may not be shared through any of the mechanisms discussed in this section. On systems with full CUDA unified memory support, system-allocated memory is IPC capable. Once access to system-allocated memory has been shared with other processes, the same programming model applies, similar to *File-backed Unified Memory*.

See the following references for more information on various ways of creating IPC-capable system-allocated memory under Linux:

- ▶ `mmap` with `MAP_SHARED`
- ▶ POSIX IPC APIs
- ▶ Linux `memfd_create` .

Note that it is not possible to share memory between different hosts and their devices using this technique.

4.1.1.1.2 Performance Tuning

In order to achieve good performance with unified memory, it is important to:

- ▶ understand how paging works on your system, and how to avoid unnecessary page faults
- ▶ understand the various mechanisms allowing you to keep data local to the accessing processor
- ▶ consider tuning your application for the granularity of memory transfers of your system.

As general advice, performance hints (see *Performance Hints*) might provide improved performance, but using them incorrectly might degrade performance compared to the default behavior. Also note that any hint has a performance cost associated with it on the host, thus useful hints must at the very least improve performance enough to overcome this cost.

4.1.1.2.1 Memory Paging and Page Sizes

To better understand the performance implication of unified memory, it is important to understand virtual addressing, memory pages and page sizes. This sub-section attempts to define all necessary terms and explain why paging matters for performance.

All currently supported systems for unified memory use a virtual address space: this means that memory addresses used by an application represent a *virtual* location which might be *mapped* to a physical location where the memory actually resides.

All currently supported processors, including both CPUs and GPUs, additionally use memory *paging*. Because all systems use a virtual address space, there are two types of memory pages:

- ▶ **Virtual pages:** This represents a fixed-size contiguous chunk of virtual memory per process tracked by the operating system, which can be *mapped* into physical memory. Note that the virtual page is linked to the *mapping*: for example, a single virtual address might be mapped into physical memory using different page sizes.
- ▶ **Physical pages:** This represents a fixed-size contiguous chunk of memory the processor's main Memory Management Unit (MMU) supports and into which a virtual page can be mapped.

Currently, all x86_64 CPUs use a default physical page size of 4KiB. Arm CPUs support multiple physical page sizes - 4KiB, 16KiB, 32KiB and 64KiB - depending on the exact CPU. Finally, NVIDIA GPUs support multiple physical page sizes, but prefer 2MiB physical pages or larger. Note that these sizes are subject to change in future hardware.

The default page size of virtual pages usually corresponds to the physical page size, but an application may use different page sizes as long as they are supported by the operating system and the hardware. Typically, supported virtual page sizes must be powers of 2 and multiples of the physical page size.

The logical entity tracking the mapping of virtual pages into physical pages will be referred to as a *page table*, and each mapping of a given virtual page with a given virtual size to physical pages is called a *Page Table Entry (PTE)*. All supported processors provide specific caches for the page table to speed up the translation of virtual addresses to physical addresses. These caches are called *Translation Lookaside Buffers (TLBs)*.

There are two important aspects for performance tuning of applications:

- ▶ the choice of virtual page size,
- ▶ whether the system offers a combined page table used by both CPUs and GPUs, or separate page tables for each CPU and GPU individually.

4.1.1.2.1.1 Choosing the Right Page Size

In general, small page sizes lead to less (virtual) memory fragmentation but more TLB misses, whereas larger page sizes lead to more memory fragmentation but less TLB misses. Additionally, memory migration is generally more expensive with larger page sizes compared to smaller page sizes, because we typically migrate full memory pages. This can cause larger latency spikes in an application using large page sizes. See also the next section for more details on page faults.

One important aspect for performance tuning is that TLB misses are generally significantly more expensive on the GPU compared to the CPU. This means that if a GPU thread frequently accesses random locations of unified memory mapped using a small enough page size, it might be significantly slower compared to the same accesses to unified memory mapped using a large enough page size. While a similar effect might occur for a CPU thread randomly accessing a large area of memory mapped using a small page size, the slowdown is less pronounced, meaning that the application might want to trade-off this slowdown with having less memory fragmentation.

Note that in general, applications should not tune their performance to the physical page size of a given processor, since physical page sizes are subject to change depending on the hardware. The advice above only applies to virtual page sizes.

4.1.1.2.1.2 CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency

Hardware-coherent systems such as NVIDIA Grace Hopper offer a logically combined page table for both CPUs and GPUs. This is important because in order to access system-allocated memory from the GPU, the GPU uses whichever page table entry was created by the CPU for the requested memory. If that page table entry uses the default CPU page size of 4KiB or 64KiB, accesses to large virtual memory areas will cause significant TLB misses, thus significant slowdowns.

On the other hand, on software-coherent systems where the CPUs and GPUs each have their own logical page table, different performance tuning aspects should be considered: in order to guarantee coherency, these systems usually use *page faults* in case a processor accesses a memory address mapped into the physical memory of a different processor. Such a page fault means that:

- ▶ It needs to be ensured that the currently owning processor (where the physical page currently resides) cannot access this page anymore, either by deleting the page table entry or updating it.
- ▶ It needs to be ensured that the processor requesting access can access this page, either by creating a new page table entry or updating an existing entry, such that it becomes valid/active.
- ▶ The physical page backing this virtual page must be moved/migrated to the processor requesting access: this can be an expensive operation, and the amount of work is proportional to the page size.

Overall, hardware-coherent systems provide significant performance benefits compared to software-coherent systems in cases where frequent concurrent accesses to the same memory page are made by both CPU and GPU threads:

- ▶ less page-faults: these systems do not need to use page-faults for emulating coherency or migrating memory,
- ▶ less contention: these systems are coherent at cache-line granularity instead of page-size granularity, that is, when there is contention from multiple processors within a cache line, only the cache line is exchanged which is much smaller than the smallest page-size, and when the different processors access different cache-lines within a page, then there is no contention.

This impacts the performance of the following scenarios:

- ▶ atomic updates to the same address concurrently from both CPUs and GPUs
- ▶ signaling a GPU thread from a CPU thread or vice-versa.

4.1.1.2.1.3 Mixing Hardware and Software Coherency

Some systems with hardware coherence such as NVIDIA DGX Station also support installing discrete, non-coherent GPU hardware. Accesses from a hardware coherent GPU will continue to use the hardware based coherency as described in [CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency](#) while accesses from the discrete GPU will use software based coherency.

Sharing of the unified address space is permitted between both classes of GPU. This results in different performance and migration behaviour based on accessing GPU. In particular accesses from a software coherent GPU will have more page faults and memory migration, while accesses from hardware coherent GPU will fault less and map remotely where possible.

For best performance, sharing of data between both GPUs should be limited or done via explicit copies. Alternatively steps such as calling `cudaMemAdviseSetPreferredLocation` should be taken to en-

sure frequently shared data remains physically resident on CPU or coherent GPU memory as by default accessing software coherent memory requires a fault and migration of data.

On mixed coherency systems, the behavior of `cudaHostRegister` and other host memory access APIs also changes for the software coherent GPU. Instead of using pinned mappings, software coherent GPUs on a mixed coherence system use software mirroring of CPU page tables. This means some GPU accesses may page fault where they would not on a non-mixed coherency system, although such faults are rare and should only occur under memory pressure.

4.1.1.2.2 Direct Unified Memory Access from the Host

Some devices have hardware support for coherent reads, stores and atomic accesses from the host on GPU-resident unified memory. These devices have the attribute `cudaDevAttrDirectManagedMemAccessFromHost` set to 1. Note that all hardware-coherent systems have this attribute set for NVLink-connected devices. On these systems, the host has direct access to GPU-resident memory without page faults and data migration. Note that with CUDA managed memory, the `cudaMemAdviseSetAccessedBy` hint with location type `cudaMemLocationTypeHost` is necessary to enable this direct access without page faults, see example below.

System Allocator

```
__global__ void write(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

__global__ void append(int *ret, int a, int b) {
    ret[threadIdx.x] += a + b + threadIdx.x;
}

void test_malloc() {
    int *ret = (int*)malloc(1000 * sizeof(int));
    // for shared page table systems, the following hint is not necessary
    cudaMemLocation location = {.type = cudaMemLocationTypeHost};
    cudaMemAdvise(ret, 1000 * sizeof(int), cudaMemAdviseSetAccessedBy,
        ↪location);

    write<<< 1, 1000 >>>(ret, 10, 100);           // pages populated in GPU
    ↪memory
    cudaDeviceSynchronize();
    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, ret[i]);      //
    ↪directManagedMemAccessFromHost=1: CPU accesses GPU memory directly without
    ↪migrations
    //
    ↪directManagedMemAccessFromHost=0: CPU faults and triggers device-to-host
    ↪migrations
    append<<< 1, 1000 >>>(ret, 10, 100);          //
    ↪directManagedMemAccessFromHost=1: GPU accesses GPU memory without migrations
    cudaDeviceSynchronize();                      //
    ↪directManagedMemAccessFromHost=0: GPU faults and triggers host-to-device
    ↪migrations
    free(ret);
}
```

Managed

```

__global__ void write(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

__global__ void append(int *ret, int a, int b) {
    ret[threadIdx.x] += a + b + threadIdx.x;
}

void test_managed() {
    int *ret;
    cudaMallocManaged(&ret, 1000 * sizeof(int));
    cudaMemLocation location = {.type = cudaMemLocationTypeHost};
    cudaMemAdvise(ret, 1000 * sizeof(int), cudaMemAdviseSetAccessedBy,
    ↪location); // set direct access hint

    write<<< 1, 1000 >>>(ret, 10, 100); // pages populated in GPU
    ↪memory
    cudaDeviceSynchronize();
    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, ret[i]); //
    ↪directManagedMemAccessFromHost=1: CPU accesses GPU memory directly without
    ↪migrations
    //
    ↪directManagedMemAccessFromHost=0: CPU faults and triggers device-to-host
    ↪migrations
    append<<< 1, 1000 >>>(ret, 10, 100); //
    ↪directManagedMemAccessFromHost=1: GPU accesses GPU memory without migrations
    cudaDeviceSynchronize(); //
    ↪directManagedMemAccessFromHost=0: GPU faults and triggers host-to-device
    ↪migrations
    cudaFree(ret);
}

```

After write kernel is completed, ret will be created and initialized in GPU memory. Next, the CPU will access ret followed by append kernel using the same ret memory again. This code will show different behavior depending on the system architecture and support of hardware coherency:

- ▶ on devices with `directManagedMemAccessFromHost=1`: CPU accesses to the managed buffer will not trigger any migrations; the data will remain resident in GPU memory and any subsequent GPU kernels can continue to access it directly without inflicting faults or migrations
- ▶ on devices with `directManagedMemAccessFromHost=0`: CPU accesses to the managed buffer will page fault and initiate data migration; any GPU kernel trying to access the same data first time will page fault and migrate pages back to GPU memory.

4.1.1.2.3 Host Native Atomics

Some devices, including NVLink-connected devices of hardware-coherent systems, support hardware-accelerated atomic accesses to CPU-resident memory. This implies that atomic accesses to host memory do not have to be emulated with a page fault. For these devices, the attribute `cudaDevAttrHostNativeAtomicSupported` is set to 1.

4.1.1.2.4 Atomic Accesses and Synchronization Primitives

CUDA unified memory supports all atomic operations available to host and device threads, enabling all threads to cooperate by concurrently accessing the same shared memory location. The `libc++` library provides many heterogeneous synchronization primitives tuned for concurrent use between host and device threads, including `cuda::atomic`, `cuda::atomic_ref`, `cuda::barrier`, `cuda::semaphore`, among many others.

On software-coherent systems, atomic accesses from the device to file-backed host memory are not supported. The following example code is valid on hardware-coherent systems but exhibits undefined behavior on other systems:

```
#include <cuda/atomic>

#include <cstdio>
#include <fcntl.h>
#include <sys/mman.h>

#define ERR(msg, ...) { fprintf(stderr, msg, ##__VA_ARGS__); return EXIT_
    ↪FAILURE; }

__global__ void kernel(int* ptr) {
    cuda::atomic_ref{*ptr}.store(2);
}

int main() {
    // this will be closed/deleted by default on exit
    FILE* tmp_file = tmpfile64();
    // need to allocate space in the file, we do this with posix_fallocate here
    int status = posix_fallocate(fileno(tmp_file), 0, 4096);
    if (status != 0) ERR("Failed to allocate space in temp file\n");
    int* ptr = (int*)mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE,
    ↪fileno(tmp_file), 0);
    if (ptr == MAP_FAILED) ERR("Failed to map temp file\n");

    // initialize the value in our file-backed memory
    *ptr = 1;
    printf("Atom value: %d\n", *ptr);

    // device and host thread access ptr concurrently, using cuda::atomic_ref
    kernel<<<1, 1>>>(ptr);
    while (cuda::atomic_ref{*ptr}.load() != 2);
    // this will always be 2
    printf("Atom value: %d\n", *ptr);

    return EXIT_SUCCESS;
}
```

On software-coherent systems, atomic accesses to unified memory may incur page faults which can lead to significant latencies. Note that this is not the case for all GPU atomics to CPU memory on these systems: operations listed by `nvidia-smi -q | grep "Atomic Caps Outbound"` may avoid page faults.

On hardware-coherent systems, atomics between host and device do not require page faults, but may still fault for other reasons that can cause any memory access to fault.

4.1.1.2.5 Malloc()/Memset() Behavior With Unified Memory

`cudaMemcpy*()` and `cudaMemset*()` accept any unified memory pointer as arguments.

For `cudaMemcpy*()`, the direction specified as `cudaMemcpyKind` is a performance hint, which can have a higher performance impact if any of the arguments is a unified memory pointer.

Thus, it is recommended to follow the following performance advice:

- ▶ When the physical location of unified memory is known, use an accurate `cudaMemcpyKind` hint.
- ▶ Prefer `cudaMemcpyDefault` over an inaccurate `cudaMemcpyKind` hint.
- ▶ Always use populated (initialized) buffers: avoid using these APIs to initialize memory.
- ▶ Avoid using `cudaMemcpy*()` if both pointers point to system-allocated memory: launch a kernel or use a CPU memory copy algorithm such as `std::memcpy` instead.

4.1.1.2.6 Overview of Memory Allocators for Unified Memory

For systems with full CUDA unified memory support various different allocators may be used to allocate unified memory. The following table shows an overview of a selection of allocators with their respective features. Note that all information in this section is subject to change in future CUDA versions.

Table 7: Overview of unified memory support of different allocators

API	Place- ment Policy	Acces- sible From	Migrate Based On Access ²	Page Sizes ⁴⁵
<code>malloc</code> , <code>new</code> , <code>mmap</code>	First touch/hi	CPU, GPU	Yes ³	System or huge page size ⁶
<code>cudaMallocManaged</code>	First touch/hi	CPU, GPU	Yes	CPU resident: system page size GPU resident: 2MB
<code>cudaMalloc</code>	GPU	GPU	No	GPU page size: 2MB
<code>cudaMallocHost</code> , <code>cudaHostAlloc</code> , <code>cudaHostRegister</code>	CPU	CPU, GPU	No	Mapped by CPU: system page size Mapped by GPU: 2MB
Memory pools, location type host: <code>cuMemCreate</code> , <code>cudaMemPoolCreate</code>	CPU	CPU, GPU	No	Mapped by CPU: system page size Mapped by GPU: 2MB
Memory pools, location type device: <code>cuMemCreate</code> , <code>cudaMemPoolCreate</code> , <code>cudaMallocAsync</code>	GPU	GPU	No	2MB

² This feature can be overridden with `cudaMemAdvise`. Even if access-based migrations are disabled, if the backing memory space is full, memory might migrate.

⁴ The default system page size is 4KiB or 64KiB on most systems, unless huge page size was explicitly specified (for example, with `mmap MAP_HUGETLB / MAP_HUGE_SHIFT`). In this case, any huge page size configured on the system is supported.

⁵ Page-sizes for GPU-resident memory may evolve in future CUDA versions.

¹ For `mmap`, file-backed memory is placed on the CPU by default, unless specified otherwise through `cudaMemAdviseSet-`

The table *Overview of unified memory support of different allocators* shows the difference in semantics of several allocators that may be considered to allocate data accessible from multiple processors at a time, including host and device. For additional details about `cudaMemPoolCreate`, see the *Memory Pools* section, for additional details about `cuMemCreate`, see the *Virtual Memory Management* section.

On hardware-coherent systems where device memory is exposed as a NUMA domain to the system, special allocators such as `numa_alloc_on_node` may be used to pin memory to the given NUMA node, either host or device. This memory is accessible from both host and device and does not migrate. Similarly, `mbind` can be used to pin memory to the given NUMA node(s), and can cause file-backed memory to be placed on the given NUMA node(s) before it is first accessed.

The following applies to allocators of memory that is shared:

- ▶ System allocators such as `mmap` allow sharing the memory between processes using the `MAP_SHARED` flag. This is supported in CUDA and can be used to share memory between different devices connected to the same host. However, this is currently not supported for sharing memory between multiple hosts as well as multiple devices. See *Inter-Process Communication (IPC) with Unified Memory* for details.
- ▶ For access to unified memory or other CUDA memory through a network on multiple hosts, consult the documentation of the communication library used, for example `NCCL`, `NVSHMEM`, `OpenMPI`, `UCX`, etc.

4.1.1.2.7 Access Counter Migration

On hardware-coherent systems, the access counters feature keeps track of the frequency of access that a GPU makes to memory located on other processors. This is needed to ensure memory pages are moved to the physical memory of the processor that is accessing the pages most frequently. It can guide migrations between CPU and GPU, as well as between peer GPUs, a process called access counter migration.

Starting with CUDA 12.4, access counters are supported system-allocated memory. Note that file-backed memory does not migrate based on access. For system-allocated memory, access counters migration can be switched on by using the `cudaMemAdviseSetAccessedBy` hint to a device with the corresponding device id. If access counters are on, one can use `cudaMemAdviseSetPreferredLocation` set to host to prevent migrations. Per default `cudaMallocManaged` migrates based on a fault-and-migrate mechanism.⁷

The driver may also use access counters for more efficient thrashing mitigation or memory oversubscription scenarios.

4.1.1.2.8 Avoid Frequent Writes to GPU-Resident Memory from the CPU

If the host accesses unified memory, cache misses may introduce more traffic than expected between host and device. Many CPU architectures require all memory operations to go through the cache hierarchy, including writes. If system memory is resident on the GPU, this means that frequent writes by the CPU to this memory can cause cache misses, thus transferring the data first from the GPU to CPU before writing the actual value into the requested memory range. On software-coherent systems, this may introduce additional page faults, while on hardware-coherent systems, it may cause higher latencies between CPU operations. Thus, in order to share data produced by the host with the device,

PreferredLocation (or `mbind`, see bullet points below).

³ File-backed memory will not migrate based on access.

⁶ Currently huge page sizes may not be kept when migrating memory to the GPU or placing it through first-touch on the GPU.

⁷ Current systems allow the use of access-counter migration with managed memory when the accessed-by device hint is set. This is an implementation detail and should not be relied on for future compatibility.

consider writing to CPU-resident memory and reading the values directly from the device. The code below shows how to achieve this with unified memory.

System Allocator

```
size_t data_size = sizeof(int);
int* data = (int*)malloc(data_size);
// ensure that data stays local to the host and avoid faults
cudaMemLocation location = {.type = cudaMemLocationTypeHost};
cudaMemAdvise(data, data_size, cudaMemAdviseSetPreferredLocation, location);
cudaMemAdvise(data, data_size, cudaMemAdviseSetAccessedBy, location);

// frequent exchanges of small data: if the CPU writes to CPU-resident
↪memory,
// and GPU directly accesses that data, we can avoid the CPU caches re-
↪loading
// data if it was evicted in between writes
for (int i = 0; i < 10; ++i) {
    *data = 42 + i;
    kernel<<<1, 1>>>(data);
    cudaDeviceSynchronize();
    // CPU cache potentially evicted data here
}
free(data);
```

Managed

```
int* data;
size_t data_size = sizeof(int);
cudaMallocManaged(&data, data_size);
// ensure that data stays local to the host and avoid faults
cudaMemLocation location = {.type = cudaMemLocationTypeHost};
cudaMemAdvise(data, data_size, cudaMemAdviseSetPreferredLocation, location);
cudaMemAdvise(data, data_size, cudaMemAdviseSetAccessedBy, location);

// frequent exchanges of small data: if the CPU writes to CPU-resident
↪memory,
// and GPU directly accesses that data, we can avoid the CPU caches re-
↪loading
// data if it was evicted in between writes
for (int i = 0; i < 10; ++i) {
    *data = 42 + i;
    kernel<<<1, 1>>>(data);
    cudaDeviceSynchronize();
    // CPU cache potentially evicted data here
}
cudaFree(data);
```

4.1.1.2.9 Exploiting Asynchronous Access to System Memory

If an application needs to share results from work on the device with the host, there are several possible options:

1. The device writes its result to GPU-resident memory, the result is transferred using `cudaMemcpy*`, and the host reads the transferred data.
2. The device directly writes its result to CPU-resident memory, and the host reads that data.
3. The device writes to GPU-resident memory, and the host directly accesses that data.

If independent work can be scheduled on the device while the result is transferred/accessed by the host, options 1 or 3 are preferred. If the device is starved until the host has accessed the result, option 2 might be preferred. This is because the device can generally write at a higher bandwidth than the host can read, unless many host threads are used to read the data.

1. Explicit Copy

```
void exchange_explicit_copy(cudaStream_t stream) {
    int* data, *host_data;
    size_t n_bytes = sizeof(int) * 16;
    // allocate receiving buffer
    host_data = (int*)malloc(n_bytes);
    // allocate, since we touch on the device first, will be GPU-resident
    cudaMallocManaged(&data, n_bytes);
    kernel<<<1, 16, 0, stream>>>(data);
    // launch independent work on the device
    // other_kernel<<<1024, 256, 0, stream>>>(other_data, ...);
    // transfer to host
    cudaMemcpyAsync(host_data, data, n_bytes, cudaMemcpyDeviceToHost, stream);
    // sync stream to ensure data has been transferred
    cudaStreamSynchronize(stream);
    // read transferred data
    printf("Got values %d - %d from GPU\n", host_data[0], host_data[15]);
    cudaFree(data);
    free(host_data);
}
```

2. Device Direct Write

```
void exchange_device_direct_write(cudaStream_t stream) {
    int* data;
    size_t n_bytes = sizeof(int) * 16;
    // allocate receiving buffer
    cudaMallocManaged(&data, n_bytes);
    // ensure that data is mapped and resident on the host
    cudaMemLocation location = {.type = cudaMemLocationTypeHost};
    cudaMemAdvise(data, n_bytes, cudaMemAdviseSetPreferredLocation, location);
    cudaMemAdvise(data, n_bytes, cudaMemAdviseSetAccessedBy, location);
    kernel<<<1, 16, 0, stream>>>(data);
    // sync stream to ensure data has been transferred
    cudaStreamSynchronize(stream);
    // read transferred data
```

(continues on next page)

(continued from previous page)

```
printf("Got values %d - %d from GPU\n", data[0], data[15]);
cudaFree(data);
}
```

3. Host Direct Read

```
void exchange_host_direct_read(cudaStream_t stream) {
    int* data;
    size_t n_bytes = sizeof(int) * 16;
    // allocate receiving buffer
    cudaMallocManaged(&data, n_bytes);
    // ensure that data is mapped and resident on the device
    cudaMemLocation device_loc = {};
    cudaGetDevice(&device_loc.id);
    device_loc.type = cudaMemLocationTypeDevice;
    cudaMemAdvise(data, n_bytes, cudaMemAdviseSetPreferredLocation, device_loc);
    cudaMemAdvise(data, n_bytes, cudaMemAdviseSetAccessedBy, device_loc);
    kernel<<<1, 16, 0, stream>>>(data);
    // launch independent work on the GPU
    // other_kernel<<<1024, 256, 0, stream>>>(other_data, ...);
    // sync stream to ensure data may be accessed (has been written by device)
    cudaStreamSynchronize(stream);
    // read data directly from host
    printf("Got values %d - %d from GPU\n", data[0], data[15]);
    cudaFree(data);
}
```

Finally, in the Explicit Copy example above, instead of using `cudaMemcpy*` to transfer data, one could use a host or device kernel to perform this transfer explicitly. For contiguous data, using the CUDA copy-engines is preferred because operations performed by copy-engines can be overlapped with work on both the host and device. Copy-engines might be used in `cudaMemcpy*` and `cudaMemPrefetchAsync` APIs, but there is no guarantee that copy-engines are used with `cudaMemcpy*` API calls. For the same reason, explicitly copy is preferred over direct host read for large enough data: if both host and device perform work that does not saturate their respective memory systems, the transfer can be performed by the copy-engines concurrently with the work performed by both host and device.

Copy-engines are generally used for both transfers between host and device as well as between peer devices within an NVLink-connected system. Due to the limited total number of copy-engines, some systems may have a lower bandwidth of `cudaMemcpy*` compared to using the device to explicitly perform the transfer. In such a case, if the transfer is in the critical path of the application, it may be preferred to use an explicit device-based transfer.

4.1.2. Unified Memory on Devices with only CUDA Managed Memory Support

For devices with compute capability 6.x or higher but without pageable memory access, see table [Overview of Unified Memory Paradigms](#), CUDA managed memory is fully supported and coherent but the GPU cannot access system-allocated memory. The programming model and performance tuning of unified memory is largely similar to the model as described in the section, [Unified Memory on Devices with Full CUDA Unified Memory Support](#), with the notable exception that system allocators cannot be used to allocate memory. Thus, the following list of sub-sections do not apply:

- ▶ *Unified Memory: In-Depth Examples*
- ▶ *CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency*
- ▶ *Atomic Accesses and Synchronization Primitives*
- ▶ *Access Counter Migration*
- ▶ *Avoid Frequent Writes to GPU-Resident Memory from the CPU*
- ▶ *Exploiting Asynchronous Access to System Memory*

4.1.3. Unified Memory on Windows, WSL, and Tegra

Note

This section is only looking at devices with compute capability lower than 6.0 or Windows platforms, devices with `concurrentManagedAccess` property set to 0.

Devices with compute capability lower than 6.0 or Windows platforms, devices with `concurrentManagedAccess` property set to 0, see *Overview of Unified Memory Paradigms*, support CUDA managed memory with the following limitations:

- ▶ **Data Migration and Coherency:** Fine-grained movement of the managed data to GPU on-demand is not supported. Whenever a GPU kernel is launched all managed memory generally has to be transferred to GPU memory to avoid faulting on memory access. Page faulting is only supported from the CPU side.
- ▶ **GPU Memory Oversubscription:** They cannot allocate more managed memory than the physical size of GPU memory.
- ▶ **Coherency and Concurrency:** Simultaneous access to managed memory is not possible, because coherence could not be guaranteed if the CPU accessed a unified memory allocation while a GPU kernel is active because of the missing GPU page faulting mechanism.

4.1.3.1 Multi-GPU

On systems with devices of compute capabilities lower than 6.0 or Windows platforms managed allocations are automatically visible to all GPUs in a system via the peer-to-peer capabilities of the GPUs. Managed memory allocations behave similar to unmanaged memory allocated using `cudaMalloc()`: the current active device is the home for the physical allocation but other GPUs in the system will access the memory at reduced bandwidth over the PCIe bus.

On Linux the managed memory is allocated in GPU memory as long as all GPUs that are actively being used by a program have the peer-to-peer support. If at any time the application starts using a GPU that doesn't have peer-to-peer support with any of the other GPUs that have managed allocations on them, then the driver will migrate all managed allocations to system memory. In this case, all GPUs experience PCIe bandwidth restrictions.

On Windows, if peer mappings are not available (for example, between GPUs of different architectures), then the system will automatically fall back to using mapped memory, regardless of whether both GPUs are actually used by a program. If only one GPU is actually going to be used, it is necessary to set the `CUDA_VISIBLE_DEVICES` environment variable before launching the program. This constrains which GPUs are visible and allows managed memory to be allocated in GPU memory.

Alternatively, on Windows users can also set `CUDA_MANAGED_FORCE_DEVICE_ALLOC` to a non-zero value to force the driver to always use device memory for physical storage. When this environment

variable is set to a non-zero value, all devices used in that process that support managed memory have to be peer-to-peer compatible with each other. The error `cudaErrorInvalidDevice` will be returned if a device that supports managed memory is used and it is not peer-to-peer compatible with any of the other managed memory supporting devices that were previously used in that process, even if `cudaDeviceReset` has been called on those devices. These environment variables are described in [CUDA Environment Variables](#).

4.1.3.2 Coherency and Concurrency

To ensure coherency the unified memory programming model puts constraints on data accesses while both the CPU and GPU are executing concurrently. In effect, the GPU has exclusive access to all managed data and the CPU is not permitted to access it, while any kernel operation is executing, regardless of whether the specific kernel is actively using the data. Concurrent CPU/GPU accesses, even to different managed memory allocations, will cause a segmentation fault because the page is considered inaccessible to the CPU.

For example the following code runs successfully on devices of compute capability 6.x due to the GPU page faulting capability which lifts all restrictions on simultaneous access but fails on pre-6.x architectures and Windows platforms because the GPU program kernel is still active when the CPU touches `y`:

```
__device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    kernel<<< 1, 1 >>>();
    y = 20;           // Error on GPUs not supporting concurrent access

    cudaDeviceSynchronize();
    return 0;
}
```

The program must explicitly synchronize with the GPU before accessing `y` (regardless of whether the GPU kernel actually touches `y` (or any managed data at all):

```
__device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    kernel<<< 1, 1 >>>();
    cudaDeviceSynchronize();
    y = 20;           // Success on GPUs not supporting concurrent access
    return 0;
}
```

Note that any function call that logically guarantees the GPU completes its work is valid to ensure logically that the GPU work is completed, see [Explicit Synchronization](#).

Note that if memory is dynamically allocated with `cudaMallocManaged()` or `cuMemAllocManaged()` while the GPU is active, the behavior of the memory is unspecified until additional work is launched or the GPU is synchronized. Attempting to access the memory on the CPU during this time may or may not cause a segmentation fault. This does not apply to memory allocated using the flag `cudaMemAttachHost` or `CU_MEM_ATTACH_HOST`.

4.1.3.3 Stream Associated Unified Memory

The CUDA programming model provides streams as a mechanism for programs to indicate dependence and independence among kernel launches. Kernels launched into the same stream are guaranteed to execute consecutively, while kernels launched into different streams are permitted to execute concurrently. See section [CUDA Streams](#).

4.1.3.3.1 Stream Callbacks

It is legal for the CPU to access managed data from within a stream callback, provided no other stream that could potentially be accessing managed data is active on the GPU. In addition, a callback that is not followed by any device work can be used for synchronization: for example, by signaling a condition variable from inside the callback; otherwise, CPU access is valid only for the duration of the callback(s). There are several important points of note:

1. It is always permitted for the CPU to access non-managed mapped memory data while the GPU is active.
2. The GPU is considered active when it is running any kernel, even if that kernel does not make use of managed data. If a kernel might use data, then access is forbidden
3. There are no constraints on concurrent inter-GPU access of managed memory, other than those that apply to multi-GPU access of non-managed memory.
4. There are no constraints on concurrent GPU kernels accessing managed data.

Note how the last point allows for races between GPU kernels, as is currently the case for non-managed GPU memory. In the perspective of the GPU, managed memory functions are identical to non-managed memory. The following code example illustrates these points:

```
int main() {
    cudaStream_t stream1, stream2;
    cudaStreamCreate(&stream1);
    cudaStreamCreate(&stream2);
    int *non_managed, *managed, *also_managed;
    cudaMallocHost(&non_managed, 4);    // Non-managed, CPU-accessible memory
    cudaMallocManaged(&managed, 4);
    cudaMallocManaged(&also_managed, 4);
    // Point 1: CPU can access non-managed data.
    kernel<<< 1, 1, 0, stream1 >>>(managed);
    *non_managed = 1;
    // Point 2: CPU cannot access any managed data while GPU is busy,
    //           unless concurrentManagedAccess = 1
    // Note we have not yet synchronized, so "kernel" is still active.
    *also_managed = 2;    // Will issue segmentation fault
    // Point 3: Concurrent GPU kernels can access the same data.
    kernel<<< 1, 1, 0, stream2 >>>(managed);
    // Point 4: Multi-GPU concurrent access is also permitted.
    cudaSetDevice(1);
    kernel<<< 1, 1 >>>(managed);
    return 0;
}
```

4.1.3.3.2 Managed memory associated to streams allows for finer-grained control

Unified memory builds upon the stream-independence model by allowing a CUDA program to explicitly associate managed allocations with a CUDA stream. In this way, the programmer indicates the use of data by kernels based on whether they are launched into a specified stream or not. This enables opportunities for concurrency based on program-specific data access patterns. The function to control this behavior is:

```
cudaError_t cudaStreamAttachMemAsync(cudaStream_t stream,
                                     void *ptr,
                                     size_t length=0,
                                     unsigned int flags=0);
```

The `cudaStreamAttachMemAsync()` function associates `length` bytes of memory starting from `ptr` with the specified stream. This allows CPU access to that memory region so long as all operations in stream have completed, regardless of whether other streams are active. In effect, this constrains exclusive ownership of the managed memory region by an active GPU to per-stream activity instead of whole-GPU activity. Most importantly, if an allocation is not associated with a specific stream, it is visible to all running kernels regardless of their stream. This is the default visibility for a `cudaMallocManaged()` allocation or a `__managed__` variable; hence, the simple-case rule that the CPU may not touch the data while any kernel is running.

Note

By associating an allocation with a specific stream, the program makes a guarantee that only kernels launched into that stream will touch that data. No error checking is performed by the unified memory system.

Note

In addition to allowing greater concurrency, the use of `cudaStreamAttachMemAsync()` can enable data transfer optimizations within the unified memory system that may affect latencies and other overhead.

The following example shows how to explicitly associate `y` with host accessibility, thus enabling access at all times from the CPU. (Note the absence of `cudaDeviceSynchronize()` after the kernel call.) Accesses to `y` by the GPU running kernel will now produce undefined results.

```
--device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    cudaStream_t stream1;
    cudaStreamCreate(&stream1);
    cudaStreamAttachMemAsync(stream1, &y, 0, cudaMemAttachHost);
    cudaDeviceSynchronize(); // Wait for Host attachment to occur.
    kernel<<< 1, 1, 0, stream1 >>>(); // Note: Launches into stream1.
    y = 20; // Success - a kernel is running but "y"
           // has been associated with no stream.

    return 0;
}
```

4.1.3.3.3 A more elaborate example on multithreaded host programs

The primary use for `cudaStreamAttachMemAsync()` is to enable independent task parallelism using CPU threads. Typically in such a program, a CPU thread creates its own stream for all work that it generates because using CUDA's NULL stream would cause dependencies between threads. The default global visibility of managed data to any GPU stream can make it difficult to avoid interactions between CPU threads in a multi-threaded program. Function `cudaStreamAttachMemAsync()` is therefore used to associate a thread's managed allocations with that thread's own stream, and the association is typically not changed for the life of the thread. Such a program would simply add a single call to `cudaStreamAttachMemAsync()` to use unified memory for its data accesses:

```
// This function performs some task, in its own , in its own private stream and
↳ can be run in parallel
void run_task(int *in, int *out, int length) {
    // Create a stream for us to use.
    cudaStream_t stream;
    cudaStreamCreate(&stream);
    // Allocate some managed data and associate with our stream.
    // Note the use of the host-attach flag to cudaMallocManaged();
    // we then associate the allocation with our stream so that
    // our GPU kernel launches can access it.
    int *data;
    cudaMallocManaged((void **)&data, length, cudaMemAttachHost);
    cudaStreamAttachMemAsync(stream, data);
    cudaStreamSynchronize(stream);
    // Iterate on the data in some way, using both Host & Device.
    for(int i=0; i<N; i++) {
        transform<<< 100, 256, 0, stream >>>(in, data, length);
        cudaStreamSynchronize(stream);
        host_process(data, length);    // CPU uses managed data.
        convert<<< 100, 256, 0, stream >>>(out, data, length);
    }
    cudaStreamSynchronize(stream);
    cudaStreamDestroy(stream);
    cudaFree(data);
}
```

In this example, the allocation-stream association is established just once, and then data is used repeatedly by both the host and device. The result is much simpler code than occurs with explicitly copying data between host and device, although the result is the same.

The function `cudaMallocManaged()` specifies the `cudaMemAttachHost` flag, which creates an allocation that is initially invisible to device-side execution. (The default allocation would be visible to all GPU kernels on all streams.) This ensures that there is no accidental interaction with another thread's execution in the interval between the data allocation and when the data is acquired for a specific stream.

Without this flag, a new allocation would be considered in-use on the GPU if a kernel launched by another thread happens to be running. This might impact the thread's ability to access the newly allocated data from the CPU before it is able to explicitly attach it to a private stream. To enable safe independence between threads, therefore, allocations should be made specifying this flag.

An alternative would be to place a process-wide barrier across all threads after the allocation has been attached to the stream. This would ensure that all threads complete their data/stream associations before any kernels are launched, avoiding the hazard. A second barrier would be needed before the stream is destroyed because stream destruction causes allocations to revert to their default visibil-

ity. The `cudaMemAttachHost` flag exists both to simplify this process, and because it is not always possible to insert global barriers where required.

4.1.3.3.4 Data Movement of Stream Associated Unified Memory

`Memcpy()/Memset()` with stream associated unified memory behaves different on devices where `concurrentManagedAccess` is not set, the following rules apply:

If `cudaMemcpyHostTo*` is specified and the source data is unified memory, then it will be accessed from the host if it is coherently accessible from the host in the copy stream (1); otherwise it will be accessed from the device. Similar rules apply to the destination when `cudaMemcpy*ToHost` is specified and the destination is unified memory.

If `cudaMemcpyDeviceTo*` is specified and the source data is unified memory, then it will be accessed from the device. The source must be coherently accessible from the device in the copy stream (2); otherwise, an error is returned. Similar rules apply to the destination when `cudaMemcpy*ToDevice` is specified and the destination is unified memory.

If `cudaMemcpyDefault` is specified, then unified memory will be accessed from the host either if it cannot be coherently accessed from the device in the copy stream (2) or if the preferred location for the data is `cudaCpuDeviceId` and it can be coherently accessed from the host in the copy stream (1); otherwise, it will be accessed from the device.

When using `cudaMemset*()` with unified memory, the data must be coherently accessible from the device in the stream being used for the `cudaMemset()` operation (2); otherwise, an error is returned.

When data is accessed from the device either by `cudaMemcpy*` or `cudaMemset*`, the stream of operation is considered to be active on the GPU. During this time, any CPU access of data that is associated with that stream or data that has global visibility, will result in a segmentation fault if the GPU has a zero value for the device attribute `concurrentManagedAccess`. The program must synchronize appropriately to ensure the operation has completed before accessing any associated data from the CPU.

1. Coherently accessible from the host in a given stream means that the memory neither has global visibility nor is it associated with the given stream.
2. Coherently accessible from the device in a given stream means that the memory either has global visibility or is associated with the given stream.

4.1.4. Performance Hints

Performance hints allow programmers to provide CUDA with more information about unified memory usage. CUDA uses performance hints to managed memory more efficiently and improve application performance. Performance hints never impact the correctness of an application. Performance hints only affect performance.

Note

Applications should only use unified memory performance hints if they improve performance.

Performance hints may be used on any unified memory allocation, including CUDA managed memory. On systems with full CUDA unified memory support, performance hints can be applied to all system-allocated memory.

4.1.4.1 Data Prefetching

The `cudaMemPrefetchAsync` API is an asynchronous stream-ordered API that may migrate data to reside closer to the specified processor. The data may be accessed while it is being prefetched. The migration does not begin until all prior operations in the stream have completed, and completes before any subsequent operation in the stream.

```
cudaError_t cudaMemPrefetchAsync(const void *devPtr,
                                size_t count,
                                struct cudaMemLocation location,
                                unsigned int flags,
                                cudaStream_t stream=0);
```

A memory region containing `[devPtr, devPtr + count)` may be migrated to the destination device `location.id` if `location.type` is `cudaMemLocationTypeDevice`, or CPU if `location.type` is `cudaMemLocationTypeHost`, when the prefetch task is executed in the given stream. For details on flags, see the current [CUDA Runtime API documentation](#).

Consider the simple code example below:

System Allocator

```
void test_prefetch_sam(const cudaStream_t& s) {
    // initialize data on CPU
    char *data = (char*)malloc(dataSizeBytes);
    init_data(data, dataSizeBytes);
    cudaMemLocation location = {.type = cudaMemLocationTypeDevice, .id =
    ↪ myGpuId};

    // encourage data to move to GPU before use
    const unsigned int flags = 0;
    cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

    // use data on GPU
    const unsigned num_blocks = (dataSizeBytes + threadsPerBlock - 1) /
    ↪ threadsPerBlock;
    mykernel<<<num_blocks, threadsPerBlock, 0, s>>>(data, dataSizeBytes);

    // encourage data to move back to CPU
    location = {.type = cudaMemLocationTypeHost};
    cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

    cudaStreamSynchronize(s);

    // use data on CPU
    use_data(data, dataSizeBytes);
    free(data);
}
```

Managed

```
void test_prefetch_managed(const cudaStream_t& s) {
    // initialize data on CPU
```

(continues on next page)

(continued from previous page)

```

char *data;
cudaMallocManaged(&data, dataSizeBytes);
init_data(data, dataSizeBytes);
cudaMemLocation location = {.type = cudaMemLocationTypeDevice, .id =
↪myGpuId};

// encourage data to move to GPU before use
const unsigned int flags = 0;
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

// use data on GPU
const unsigned num_blocks = (dataSizeBytes + threadsPerBlock - 1) /
↪threadsPerBlock;
mykernel<<<num_blocks, threadsPerBlock, 0, s>>>(data, dataSizeBytes);

// encourage data to move back to CPU
location = {.type = cudaMemLocationTypeHost};
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

cudaStreamSynchronize(s);

// use data on CPU
use_data(data, dataSizeBytes);
cudaFree(data);
}

```

4.1.4.2 Data Usage Hints

When multiple processors simultaneously access the same data, `cudaMemAdvise` may be used to hint how the data at `[devPtr, devPtr + count)` will be accessed:

```

cudaError_t cudaMemAdvise(const void *devPtr,
                        size_t count,
                        enum cudaMemoryAdvise advice,
                        struct cudaMemLocation location);

```

The example shows how to use `cudaMemAdvise`:

```

init_data(data, dataSizeBytes);
cudaMemLocation location = {.type = cudaMemLocationTypeDevice, .id =
↪myGpuId};

// encourage data to move to GPU before use
const unsigned int flags = 0;
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

// use data on GPU
const unsigned num_blocks = (dataSizeBytes + threadsPerBlock - 1) /
↪threadsPerBlock;
mykernel<<<num_blocks, threadsPerBlock, 0, s>>>(data, dataSizeBytes);

// encourage data to move back to CPU

```

(continues on next page)

(continued from previous page)

```

location = {.type = cudaMemLocationTypeHost};
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

cudaStreamSynchronize(s);

// use data on CPU
use_data(data, dataSizeBytes);
cudaFree(data);
}
// test-prefetch-managed-end

static const int maxDevices = 1;
static const int maxOuterLoopIter = 3;
static const int maxInnerLoopIter = 4;

// test-advise-managed-begin
void test_advise_managed(cudaStream_t stream) {
    char *dataPtr;
    size_t dataSize = 64 * threadsPerBlock; // 16 KiB

```

Where advice may take the following values:

► **cudaMemAdviseSetReadMostly:**

This implies that the data is mostly going to be read from and only occasionally written to. In general, it allows trading off read bandwidth for write bandwidth on this region.

► **cudaMemAdviseSetPreferredLocation:**

This hint sets the preferred location for the data to be the specified device's physical memory. This hint encourages the system to keep the data at the preferred location, but does not guarantee it. Passing in a value of `cudaMemLocationTypeHost` for `location.type` sets the preferred location as CPU memory. Other hints, like `cudaMemPrefetchAsync`, may override this hint and allow the memory to migrate away from its preferred location.

► **cudaMemAdviseSetAccessedBy:**

In some systems, it may be beneficial for performance to establish a mapping into memory before accessing the data from a given processor. This hint tells the system that the data will be frequently accessed by `location.id` when `location.type` is `cudaMemLocationTypeDevice`, enabling the system to assume that creating these mappings pays off. This hint does not imply where the data should reside, but it can be combined with `cudaMemAdviseSetPreferredLocation` to specify that. On hardware-coherent systems, this hint switches on access counter migration, see [Access Counter Migration](#).

Each advice can be also unset by using one of the following values: `cudaMemAdviseUnsetReadMostly`, `cudaMemAdviseUnsetPreferredLocation` and `cudaMemAdviseUnsetAccessedBy`.

The example shows how to use `cudaMemAdvise`:

System Allocator

```

void test_advise_sam(cudaStream_t stream) {
    char *dataPtr;
    size_t dataSize = 64 * threadsPerBlock; // 16 KiB

    // Allocate memory using malloc or cudaMallocManaged

```

(continues on next page)

(continued from previous page)

```

dataPtr = (char*)malloc(dataSize);

// Set the advice on the memory region
cudaMemLocation loc = {.type = cudaMemLocationTypeDevice, .id = myGpuId};
cudaMemAdvise(dataPtr, dataSize, cudaMemAdviseSetReadMostly, loc);

int outerLoopIter = 0;
while (outerLoopIter < maxOuterLoopIter) {
    // The data is written by the CPU each outer loop iteration
    init_data(dataPtr, dataSize);

    // The data is made available to all GPUs by prefetching.
    // Prefetching here causes read duplication of data instead
    // of data migration
    cudaMemLocation location;
    location.type = cudaMemLocationTypeDevice;
    for (int device = 0; device < maxDevices; device++) {
        location.id = device;
        const unsigned int flags = 0;
        cudaMemPrefetchAsync(dataPtr, dataSize, location, flags, stream);
    }

    // The kernel only reads this data in the inner loop
    int innerLoopIter = 0;
    while (innerLoopIter < maxInnerLoopIter) {
        mykernel<<<32, threadsPerBlock, 0, stream>>>((const char *)dataPtr,
→dataSize);
        innerLoopIter++;
    }
    outerLoopIter++;
}

free(dataPtr);
}

```

Managed

```

void test_advise_managed(cudaStream_t stream) {
    char *dataPtr;
    size_t dataSize = 64 * threadsPerBlock; // 16 KiB

    // Allocate memory using cudaMallocManaged
    // (malloc may be used on systems with full CUDA Unified memory support)
    cudaMallocManaged(&dataPtr, dataSize);

    // Set the advice on the memory region
    cudaMemLocation loc = {.type = cudaMemLocationTypeDevice, .id = myGpuId};
    cudaMemAdvise(dataPtr, dataSize, cudaMemAdviseSetReadMostly, loc);

    int outerLoopIter = 0;
    while (outerLoopIter < maxOuterLoopIter) {

```

(continues on next page)

(continued from previous page)

```

// The data is written by the CPU each outer loop iteration
init_data(dataPtr, dataSize);

// The data is made available to all GPUs by prefetching.
// Prefetching here causes read duplication of data instead
// of data migration
cudaMemLocation location;
location.type = cudaMemLocationTypeDevice;
for (int device = 0; device < maxDevices; device++) {
    location.id = device;
    const unsigned int flags = 0;
    cudaMemPrefetchAsync(dataPtr, dataSize, location, flags, stream);
}

// The kernel only reads this data in the inner loop
int innerLoopIter = 0;
while (innerLoopIter < maxInnerLoopIter) {
    mykernel<<<32, threadsPerBlock, 0, stream>>>((const char *)dataPtr,
→dataSize);
    innerLoopIter++;
}
outerLoopIter++;
}

cudaFree(dataPtr);
}

```

4.1.4.3 Memory Discarding

The `cudaMemDiscardBatchAsync` API allows applications to inform the CUDA runtime that the contents of specified memory ranges are no longer useful. The Unified Memory driver performs automatic memory transfers due to fault-based migration or memory evictions to support device memory oversubscription. These automatic memory transfers can sometimes be redundant, which severely decreases performance. Marking an address range as ‘discard’ will inform the Unified Memory driver that the application has consumed the contents in the range and there is no need to migrate this data on prefetches or page evictions in order to make room for other allocations. Reading a discarded page without a subsequent write access or prefetch will yield an indeterminate value. Whereas any new writes after the discard operation is guaranteed to be seen by a subsequent read access. Concurrent accesses or prefetches to address ranges being discarded will result in undefined behavior.

```

cudaError_t cudaMemDiscardBatchAsync(void **dptrs,
                                     size_t *sizes,
                                     size_t count,
                                     unsigned long long flags,
                                     cudaStream_t stream);

```

The function performs a batch of memory discards on address ranges specified in `dptrs` and `sizes` arrays. Both arrays must be of the same length as specified by `count`. Each memory range must refer to managed memory allocated via `cudaMallocManaged` or declared via `__managed__` variables.

The `cudaMemDiscardAndPrefetchBatchAsync` API combines both discard and prefetch operations. Calling `cudaMemDiscardAndPrefetchBatchAsync` is semantically equivalent to calling `cudaMemDiscardBatchAsync` followed by `cudaMemPrefetchBatchAsync`, but is more optimal. This is

useful when the application needs the memory to be on the target location but does not need the contents of the memory.

```
cudaError_t cudaMemDiscardAndPrefetchBatchAsync(void **dptrs,
                                                size_t *sizes,
                                                size_t count,
                                                struct cudaMemLocation
→ *prefetchLocs,
                                                size_t *prefetchLocIdxs,
                                                size_t numPrefetchLocs,
                                                unsigned long long flags,
                                                cudaStream_t stream);
```

The `prefetchLocs` array specifies the destinations for prefetching, while `prefetchLocIdxs` indicates which operations each prefetch location applies to. For example, if a batch has 10 operations and the first 6 should be prefetched to one location while the remaining 4 to another, then `numPrefetchLocs` would be 2, `prefetchLocIdxs` would be {0, 6}, and `prefetchLocs` would contain the two destination locations.

Important considerations:

- ▶ Reading from a discarded range without a subsequent write or prefetch will return an indeterminate value
- ▶ The discard operation can be undone by writing to the range or prefetching it via `cudaMemPrefetchAsync`
- ▶ Any reads, writes, or prefetches that occur simultaneously with the discard operation result in undefined behavior
- ▶ All devices must have a non-zero value for `cudaDevAttrConcurrentManagedAccess`

4.1.4.4 Querying Data Usage Attributes on Managed Memory

A program can query memory range attributes assigned through `cudaMemAdvise` or `cudaMemPrefetchAsync` on CUDA managed memory by using the following API:

```
cudaMemRangeGetAttribute(void *data,
                        size_t dataSize,
                        enum cudaMemRangeAttribute attribute,
                        const void *devPtr,
                        size_t count);
```

This function queries an attribute of the memory range starting at `devPtr` with a size of `count` bytes. The memory range must refer to managed memory allocated via `cudaMallocManaged` or declared via `__managed__` variables. It is possible to query the following attributes:

- ▶ `cudaMemRangeAttributeReadMostly`: returns 1 if the entire memory range has the `cudaMemAdviseSetReadMostly` attribute set, or 0 otherwise.
- ▶ `cudaMemRangeAttributePreferredLocation`: the result returned will be a GPU device id or `cudaCpuDeviceId` if the entire memory range has the corresponding processor as preferred location, otherwise `cudaInvalidDeviceId` will be returned. An application can use this query API to make decision about staging data through CPU or GPU depending on the preferred location attribute of the managed pointer. Note that the actual location of the memory range at the time of the query may be different from the preferred location.
- ▶ `cudaMemRangeAttributeAccessedBy`: will return the list of devices that have that advise set for that memory range.

- ▶ `cudaMemRangeAttributeLastPrefetchLocation`: will return the last location to which the memory range was prefetched explicitly using `cudaMemPrefetchAsync`. Note that this simply returns the last location that the application requested to prefetch the memory range to. It gives no indication as to whether the prefetch operation to that location has completed or even begun.
- ▶ `cudaMemRangeAttributePreferredLocationType`: it returns the location type of the preferred location with the following values:
 - ▶ `cudaMemLocationTypeDevice`: if all pages in the memory range have the same GPU as their preferred location,
 - ▶ `cudaMemLocationTypeHost`: if all pages in the memory range have the CPU as their preferred location,
 - ▶ `cudaMemLocationTypeHostNuma`: if all the pages in the memory range have the same host NUMA node ID as their preferred location,
 - ▶ `cudaMemLocationTypeInvalid`: if either all the pages don't have the same preferred location or some of the pages don't have a preferred location at all.
- ▶ `cudaMemRangeAttributePreferredLocationId`: returns the device ordinal if the `cudaMemRangeAttributePreferredLocationType` query for the same address range returns `cudaMemLocationTypeDevice`. If the preferred location type is a host NUMA node, it returns the host NUMA node ID. Otherwise, the id should be ignored.
- ▶ `cudaMemRangeAttributeLastPrefetchLocationType`: returns the last location type to which all pages in the memory range were prefetched explicitly via `cudaMemPrefetchAsync`. The following values are returned:
 - ▶ `cudaMemLocationTypeDevice`: if all pages in the memory range were prefetched to the same GPU,
 - ▶ `cudaMemLocationTypeHost`: if all pages in the memory range were prefetched to the CPU,
 - ▶ `cudaMemLocationTypeHostNuma`: if all the pages in the memory range were prefetched to the same host NUMA node ID,
 - ▶ `cudaMemLocationTypeInvalid`: if either all the pages were not prefetched to the same location or some of the pages were never prefetched at all.
- ▶ `cudaMemRangeAttributeLastPrefetchLocationId`: if the `cudaMemRangeAttributeLastPrefetchLocationType` query for the same address range returns `cudaMemLocationTypeDevice`, it will be a valid device ordinal or if it returns `cudaMemLocationTypeHostNuma`, it will be a valid host NUMA node ID. Otherwise, the id should be ignored.

Additionally, multiple attributes can be queried by using corresponding `cudaMemRangeGetAttributes` function.

4.1.4.5 GPU Memory Oversubscription

Unified memory enables applications to *oversubscribe* the memory of any individual processor: in other words they can allocate and share arrays larger than the memory capacity of any individual processor in the system, enabling among others out-of-core processing of datasets that do not fit within a single GPU, without adding significant complexity to the programming model.

Additionally, multiple attributes can be queried by using corresponding `cudaMemRangeGetAttributes` function.

4.2. CUDA Graphs

CUDA Graphs present another model for work submission in CUDA. A graph is a series of operations such as kernel launches, data movement, etc., connected by dependencies, which is defined separately from its execution. This allows a graph to be defined once and then launched repeatedly. Separating out the definition of a graph from its execution enables a number of optimizations: first, CPU launch costs are reduced compared to streams, because much of the setup is done in advance; second, presenting the whole workflow to CUDA enables optimizations which might not be possible with the piecewise work submission mechanism of streams.

To see the optimizations possible with graphs, consider what happens in a stream: when you place a kernel into a stream, the host driver performs a sequence of operations in preparation for the execution of the kernel on the GPU. These operations, necessary for setting up and launching the kernel, are an overhead cost which must be paid for each kernel that is issued. For a GPU kernel with a short execution time, this overhead cost can be a significant fraction of the overall end-to-end execution time. By creating a CUDA graph that encompasses a workflow that will be launched many times, these overhead costs can be paid once for the entire graph during instantiation, and the graph itself can then be launched repeatedly with very little overhead.

4.2.1. Graph Structure

An operation forms a node in a graph. The dependencies between the operations are the edges. These dependencies constrain the execution sequence of the operations.

An operation may be scheduled at any time once the nodes on which it depends are complete. Scheduling is left up to the CUDA system.

4.2.1.1 Node Types

A graph node can be one of:

- ▶ kernel
- ▶ CPU function call
- ▶ memory copy
- ▶ memset
- ▶ empty node
- ▶ waiting on a *CUDA Event*
- ▶ recording a *CUDA Event*
- ▶ signalling an *external semaphore*
- ▶ waiting on an *external semaphore*
- ▶ *conditional node*
- ▶ *memory node*
- ▶ child graph: To execute a separate nested graph, as shown in the following figure.

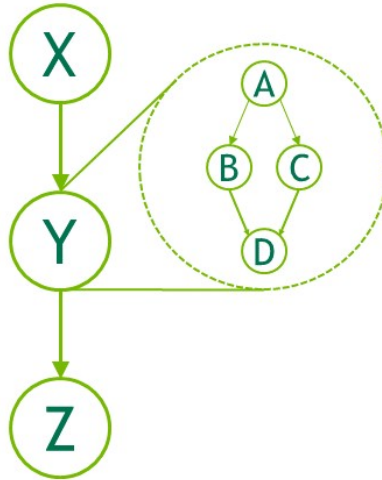


Figure 24: Child Graph Example

4.2.1.2 Edge Data

CUDA 12.3 introduced edge data on CUDA Graphs. At this time, the only use for non-default edge data is enabling *Programmatic Dependent Launch*.

Generally speaking, edge data modifies a dependency specified by an edge and consists of three parts: an outgoing port, an incoming port, and a type. An outgoing port specifies when an associated edge is triggered. An incoming port specifies what portion of a node is dependent on an associated edge. A type modifies the relation between the endpoints.

Port values are specific to node type and direction, and edge types may be restricted to specific node types. In all cases, zero-initialized edge data represents default behavior. Outgoing port 0 waits on an entire task, incoming port 0 blocks an entire task, and edge type 0 is associated with a full dependency with memory synchronizing behavior.

Edge data is optionally specified in various graph APIs via a parallel array to the associated nodes. If it is omitted as an input parameter, zero-initialized data is used. If it is omitted as an output (query) parameter, the API accepts this if the edge data being ignored is all zero-initialized, and returns `cudaErrorLossyQuery` if the call would discard information.

Edge data is also available in some stream capture APIs: `cudaStreamBeginCaptureToGraph()`, `cudaStreamGetCaptureInfo()`, and `cudaStreamUpdateCaptureDependencies()`. In these cases, there is not yet a downstream node. The data is associated with a dangling edge (half edge) which will either be connected to a future captured node or discarded at termination of stream capture. Note that some edge types do not wait on full completion of the upstream node. These edges are ignored when considering if a stream capture has been fully rejoined to the origin stream, and cannot be discarded at the end of capture. See *Stream Capture*.

No node types define additional incoming ports, and only kernel nodes define additional outgoing ports. There is one non-default dependency type, `cudaGraphDependencyTypeProgrammatic`, which is used to enable *Programmatic Dependent Launch* between two kernel nodes.

4.2.2. Building and Running Graphs

Work submission using graphs is separated into three distinct stages: definition, instantiation, and execution.

- ▶ During the **definition** or **creation** phase, a program creates a description of the operations in the graph along with the dependencies between them.
- ▶ **Instantiation** takes a snapshot of the graph template, validates it, and performs much of the setup and initialization of work with the aim of minimizing what needs to be done at launch. The resulting instance is known as an *executable graph*.
- ▶ An **executable** graph may be launched into a stream, similar to any other CUDA work. It may be launched any number of times without repeating the instantiation.

4.2.2.1 Graph Creation

Graphs can be created via two mechanisms: using the explicit Graph API and via stream capture.

4.2.2.1.1 Graph APIs

The following is an example (omitting declarations and other boilerplate code) of creating the below graph. Note the use of `cudaGraphCreate()` to create the graph and `cudaGraphAddNode()` to add the kernel nodes and their dependencies. [The CUDA Runtime API documentation](#) lists all the functions available for adding nodes and dependencies.

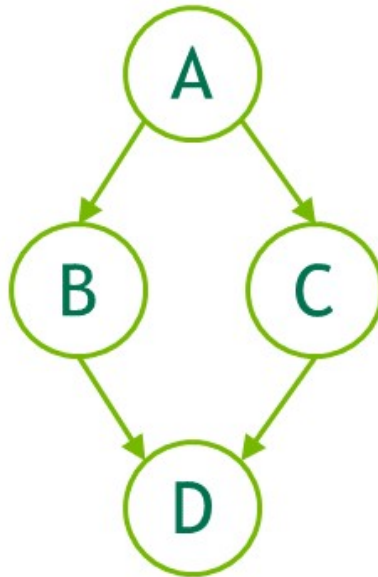


Figure 25: Creating a Graph Using Graph APIs Example

```
// Create the graph - it starts out empty
cudaGraphCreate(&graph, 0);

// Create the nodes and their dependencies
cudaGraphNode_t nodes[4];
cudaGraphNodeParams kParams = { cudaGraphNodeTypeKernel };
```

(continues on next page)

(continued from previous page)

```

kParams.kernel.func      = (void *)kernelName;
kParams.kernel.gridDim.x = kParams.kernel.gridDim.y = kParams.kernel.
↪gridDim.z = 1;
kParams.kernel.blockDim.x = kParams.kernel.blockDim.y = kParams.kernel.
↪blockDim.z = 1;

cudaGraphAddNode(&nodes[0], graph, NULL, NULL, 0, &kParams);
cudaGraphAddNode(&nodes[1], graph, &nodes[0], NULL, 1, &kParams);
cudaGraphAddNode(&nodes[2], graph, &nodes[0], NULL, 1, &kParams);
cudaGraphAddNode(&nodes[3], graph, &nodes[1], NULL, 2, &kParams);

```

The example above shows four kernel nodes with dependencies between them to illustrate the creation of a very simple graph. In a typical user application there would also need to be nodes added for memory operations, such as `cudaGraphAddMemcpyNode()` and the like. For full reference of all graph API functions to add nodes, see [The CUDA Runtime API documentation](#).

4.2.2.1.2 Stream Capture

Stream capture provides a mechanism to create a graph from existing stream-based APIs. A section of code which launches work into streams, including existing code, can be bracketed with calls to `cudaStreamBeginCapture()` and `cudaStreamEndCapture()`. See below.

```

cudaGraph_t graph;

cudaStreamBeginCapture(stream);

kernel_A<<< ..., stream >>>(...);
kernel_B<<< ..., stream >>>(...);
libraryCall(stream);
kernel_C<<< ..., stream >>>(...);

cudaStreamEndCapture(stream, &graph);

```

A call to `cudaStreamBeginCapture()` places a stream in capture mode. When a stream is being captured, work launched into the stream is not enqueued for execution. It is instead appended to an internal graph that is progressively being built up. This graph is then returned by calling `cudaStreamEndCapture()`, which also ends capture mode for the stream. A graph which is actively being constructed by stream capture is referred to as a *capture graph*.

Stream capture can be used on any CUDA stream except `cudaStreamLegacy` (the “NULL stream”). Note that it *can* be used on `cudaStreamPerThread`. If a program is using the legacy stream, it may be possible to redefine stream 0 to be the per-thread stream with no functional change. See [Blocking and non-blocking streams and the default stream](#).

Whether a stream is being captured can be queried with `cudaStreamIsCapturing()`.

Work can be captured to an existing graph using `cudaStreamBeginCaptureToGraph()`. Instead of capturing to an internal graph, work is captured to a graph provided by the user.

4.2.2.1.2.1 Cross-stream Dependencies and Events

Stream capture can handle cross-stream dependencies expressed with `cudaEventRecord()` and `cudaStreamWaitEvent()`, provided the event being waited upon was recorded into the same capture graph.

When an event is recorded in a stream that is in capture mode, it results in a *captured event*. A captured event represents a set of nodes in a capture graph.

When a captured event is waited on by a stream, it places the stream in capture mode if it is not already, and the next item in the stream will have additional dependencies on the nodes in the captured event. The two streams are then being captured to the same capture graph.

When cross-stream dependencies are present in stream capture, `cudaStreamEndCapture()` must still be called in the same stream where `cudaStreamBeginCapture()` was called; this is the *origin stream*. Any other streams which are being captured to the same capture graph, due to event-based dependencies, must also be joined back to the origin stream. This is illustrated below. All streams being captured to the same capture graph are taken out of capture mode upon `cudaStreamEndCapture()`. Failure to rejoin to the origin stream will result in failure of the overall capture operation.

```
// stream1 is the origin stream
cudaStreamBeginCapture(stream1);

kernel_A<<< ..., stream1 >>>(...);

// Fork into stream2
cudaEventRecord(event1, stream1);
cudaStreamWaitEvent(stream2, event1);

kernel_B<<< ..., stream1 >>>(...);
kernel_C<<< ..., stream2 >>>(...);

// Join stream2 back to origin stream (stream1)
cudaEventRecord(event2, stream2);
cudaStreamWaitEvent(stream1, event2);

kernel_D<<< ..., stream1 >>>(...);

// End capture in the origin stream
cudaStreamEndCapture(stream1, &graph);

// stream1 and stream2 no longer in capture mode
```

The graph returned by the above code is shown in [Figure 25](#).

Note

When a stream is taken out of capture mode, the next non-captured item in the stream (if any) will still have a dependency on the most recent prior non-captured item, despite intermediate items having been removed.

4.2.2.1.2.2 Prohibited and Unhandled Operations

It is invalid to synchronize or query the execution status of a stream which is being captured or a captured event, because they do not represent items scheduled for execution. It is also invalid to query the execution status of or synchronize a broader handle which encompasses an active stream capture, such as a device or context handle when any associated stream is in capture mode.

When any stream in the same context is being captured, and it was not created with `cudaStreamNonBlocking`, any attempted use of the legacy stream is invalid. This is because the legacy stream handle at all times encompasses these other streams; enqueueing to the legacy stream would create a dependency on the streams being captured, and querying it or synchronizing it would query or synchronize the streams being captured.

It is therefore also invalid to call synchronous APIs in this case. One example of a synchronous APIs is `cudaMemcpy()` which enqueues work to the legacy stream and synchronizes on it before returning.

Note

As a general rule, when a dependency relation would connect something that is captured with something that was not captured and instead enqueued for execution, CUDA prefers to return an error rather than ignore the dependency. An exception is made for placing a stream into or out of capture mode; this severs a dependency relation between items added to the stream immediately before and after the mode transition.

It is invalid to merge two separate capture graphs by waiting on a captured event from a stream which is being captured and is associated with a different capture graph than the event. It is invalid to wait on a non-captured event from a stream which is being captured without specifying the `cudaEventWaitExternal` flag.

A small number of APIs that enqueue asynchronous operations into streams are not currently supported in graphs and will return an error if called with a stream which is being captured, such as `cudaStreamAttachMemAsync()`.

4.2.2.1.2.3 Invalidation

When an invalid operation is attempted during stream capture, any associated capture graphs are *invalidated*. When a capture graph is invalidated, further use of any streams which are being captured or captured events associated with the graph is invalid and will return an error, until stream capture is ended with `cudaStreamEndCapture()`. This call will take the associated streams out of capture mode, but will also return an error value and a NULL graph.

4.2.2.1.2.4 Capture Introspection

Active stream capture operations can be inspected using `cudaStreamGetCaptureInfo()`. This allows the user to obtain the status of the capture, a unique (per-process) ID for the capture, the underlying graph object, and dependencies/edge data for the next node to be captured in the stream. This dependency information can be used to obtain a handle to the node(s) which were last captured in the stream.

4.2.2.1.3 Putting It All Together

The example in [Figure 25](#) is a simplistic example intended to show a small graph conceptually. In an application that utilizes CUDA graphs, there is more complexity to using either the graph API or stream

capture. The following code snippet shows a side by side comparison of the Graph API and Stream Capture to create a CUDA graph to execute a simple two stage reduction algorithm.

Figure 26 is an illustration of this CUDA graph and was generated using the `cudaGraphDebugDot-Print` function applied to the code below, with small adjustments for readability, and then rendered using `Graphviz`.

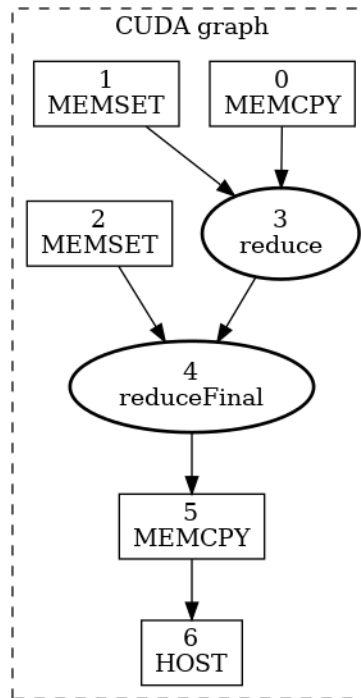


Figure 26: CUDA graph example using two stage reduction kernel

Graph API

```

void cudaGraphsManual(float *inputVec_h,
                     float *inputVec_d,
                     double *outputVec_d,
                     double *result_d,
                     size_t inputSize,
                     size_t numOfBlocks)
{
    cudaStream_t      streamForGraph;
    cudaGraph_t       graph;
    std::vector<cudaGraphNode_t> nodeDependencies;
    cudaGraphNode_t    memcpyNode, kernelNode, memsetNode;
    double             result_h = 0.0;

    cudaStreamCreate(&streamForGraph);

    cudaKernelNodeParams kernelNodeParams = {0};
    cudaMemcpy3DParms    memcpyParams     = {0};
    cudaMemsetParams      memsetParams     = {0};

```

(continues on next page)

(continued from previous page)

```

    memcpyParams.srcArray = NULL;
    memcpyParams.srcPos   = make_cudaPos(0, 0, 0);
    memcpyParams.srcPtr   = make_cudaPitchedPtr(inputVec_h, sizeof(float) *
→inputSize, inputSize, 1);
    memcpyParams.dstArray = NULL;
    memcpyParams.dstPos   = make_cudaPos(0, 0, 0);
    memcpyParams.dstPtr   = make_cudaPitchedPtr(inputVec_d, sizeof(float) *
→inputSize, inputSize, 1);
    memcpyParams.extent   = make_cudaExtent(sizeof(float) * inputSize, 1, 1);
    memcpyParams.kind      = cudaMemcpyHostToDevice;

    memsetParams.dst      = (void *)outputVec_d;
    memsetParams.value    = 0;
    memsetParams.pitch    = 0;
    memsetParams.elementSize = sizeof(float); // elementSize can be max 4 bytes
    memsetParams.width    = numOfBlocks * 2;
    memsetParams.height   = 1;

    cudaGraphCreate(&graph, 0);
    cudaGraphAddMemcpyNode(&memcpyNode, graph, NULL, 0, &memcpyParams);
    cudaGraphAddMemsetNode(&memsetNode, graph, NULL, 0, &memsetParams);

    nodeDependencies.push_back(memsetNode);
    nodeDependencies.push_back(memcpyNode);

    void *kernelArgs[4] = {(void *)&inputVec_d, (void *)&outputVec_d, &
→inputSize, &numOfBlocks};

    kernelNodeParams.func      = (void *)reduce;
    kernelNodeParams.gridDim   = dim3(numOfBlocks, 1, 1);
    kernelNodeParams.blockDim  = dim3(THREADS_PER_BLOCK, 1, 1);
    kernelNodeParams.sharedMemBytes = 0;
    kernelNodeParams.kernelParams = (void **)kernelArgs;
    kernelNodeParams.extra     = NULL;

    cudaGraphAddKernelNode(
        &kernelNode, graph, nodeDependencies.data(), nodeDependencies.size(), &
→kernelNodeParams);

    nodeDependencies.clear();
    nodeDependencies.push_back(kernelNode);

    memset(&memsetParams, 0, sizeof(memsetParams));
    memsetParams.dst      = result_d;
    memsetParams.value    = 0;
    memsetParams.elementSize = sizeof(float);
    memsetParams.width    = 2;
    memsetParams.height   = 1;
    cudaGraphAddMemsetNode(&memsetNode, graph, NULL, 0, &memsetParams);

    nodeDependencies.push_back(memsetNode);

```

(continues on next page)

(continued from previous page)

```

memset(&kernelNodeParams, 0, sizeof(kernelNodeParams));
kernelNodeParams.func          = (void *)reduceFinal;
kernelNodeParams.gridDim      = dim3(1, 1, 1);
kernelNodeParams.blockDim     = dim3(THREADS_PER_BLOCK, 1, 1);
kernelNodeParams.sharedMemBytes = 0;
void *kernelArgs2[3]          = {(void *)&outputVec_d, (void *)&result_d,
→ &numOfBlocks};
kernelNodeParams.kernelParams  = kernelArgs2;
kernelNodeParams.extra         = NULL;

cudaGraphAddKernelNode(
    &kernelNode, graph, nodeDependencies.data(), nodeDependencies.size(), &
→ kernelNodeParams);

nodeDependencies.clear();
nodeDependencies.push_back(kernelNode);

memset(&memcpyParams, 0, sizeof(memcpyParams));

memcpyParams.srcArray = NULL;
memcpyParams.srcPos   = make_cudaPos(0, 0, 0);
memcpyParams.srcPtr    = make_cudaPitchedPtr(result_d, sizeof(double), 1,
→ 1);
memcpyParams.dstArray = NULL;
memcpyParams.dstPos   = make_cudaPos(0, 0, 0);
memcpyParams.dstPtr    = make_cudaPitchedPtr(&result_h, sizeof(double), 1,
→ 1);
memcpyParams.extent    = make_cudaExtent(sizeof(double), 1, 1);
memcpyParams.kind      = cudaMemcpyDeviceToHost;

cudaGraphAddMemcpyNode(&memcpyNode, graph, nodeDependencies.data(),
→ nodeDependencies.size(), &memcpyParams);
nodeDependencies.clear();
nodeDependencies.push_back(memcpyNode);

cudaGraphNode_t    hostNode;
cudaHostNodeParams hostParams = {0};
hostParams.fn       = myHostNodeCallback;
callBackData_t hostFnData;
hostFnData.data     = &result_h;
hostFnData.fn_name  = "cudaGraphsManual";
hostParams.userData = &hostFnData;

cudaGraphAddHostNode(&hostNode, graph, nodeDependencies.data(),
→ nodeDependencies.size(), &hostParams);
}

```

Stream Capture

```

void cudaGraphsUsingStreamCapture(float *inputVec_h,
                                float *inputVec_d,
                                double *outputVec_d,
                                double *result_d,
                                size_t inputSize,
                                size_t numOfBlocks)
{
    cudaStream_t stream1, stream2, stream3, streamForGraph;
    cudaEvent_t forkStreamEvent, memsetEvent1, memsetEvent2;
    cudaGraph_t graph;
    double result_h = 0.0;

    cudaStreamCreate(&stream1);
    cudaStreamCreate(&stream2);
    cudaStreamCreate(&stream3);
    cudaStreamCreate(&streamForGraph);

    cudaEventCreate(&forkStreamEvent);
    cudaEventCreate(&memsetEvent1);
    cudaEventCreate(&memsetEvent2);

    cudaStreamBeginCapture(stream1, cudaStreamCaptureModeGlobal);

    cudaEventRecord(forkStreamEvent, stream1);
    cudaStreamWaitEvent(stream2, forkStreamEvent, 0);
    cudaStreamWaitEvent(stream3, forkStreamEvent, 0);

    cudaMemcpyAsync(inputVec_d, inputVec_h, sizeof(float) * inputSize,
    ↪ cudaMemcpyDefault, stream1);

    cudaMemsetAsync(outputVec_d, 0, sizeof(double) * numOfBlocks, stream2);

    cudaEventRecord(memsetEvent1, stream2);

    cudaMemsetAsync(result_d, 0, sizeof(double), stream3);
    cudaEventRecord(memsetEvent2, stream3);

    cudaStreamWaitEvent(stream1, memsetEvent1, 0);

    reduce<<<numOfBlocks, THREADS_PER_BLOCK, 0, stream1>>>(inputVec_d,
    ↪ outputVec_d, inputSize, numOfBlocks);

    cudaStreamWaitEvent(stream1, memsetEvent2, 0);

    reduceFinal<<<1, THREADS_PER_BLOCK, 0, stream1>>>(outputVec_d, result_d,
    ↪ numOfBlocks);
    cudaMemcpyAsync(&result_h, result_d, sizeof(double), cudaMemcpyDefault,
    ↪ stream1);

    callbackData_t hostFnData = {0};
    hostFnData.data = &result_h;

```

(continues on next page)

(continued from previous page)

```

hostFnData.fn_name      = "cudaGraphsUsingStreamCapture";
cudaHostFn_t fn         = myHostNodeCallback;
cudaLaunchHostFunc(stream1, fn, &hostFnData);
cudaStreamEndCapture(stream1, &graph);
}

```

4.2.2.2 Graph Instantiation

Once a graph has been created, either by the use of the graph API or stream capture, the graph must be instantiated to create an executable graph, which can then be launched. Assuming the `cudaGraph_t` graph has been created successfully, the following code will instantiate the graph and create the executable graph `cudaGraphExec_t graphExec`:

```

cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);

```

4.2.2.3 Graph Execution

After a graph has been created and instantiated to create an executable graph, it can be launched. Assuming the `cudaGraphExec_t graphExec` has been created successfully, the following code snippet will launch the graph into the specified stream:

```

cudaGraphLaunch(graphExec, stream);

```

Pulling it all together and using the stream capture example from [Section 4.2.2.1.2](#), the following code snippet will create a graph, instantiate it, and launch it:

```

cudaGraph_t graph;

cudaStreamBeginCapture(stream);

kernel_A<<< ..., stream >>>(...);
kernel_B<<< ..., stream >>>(...);
libraryCall(stream);
kernel_C<<< ..., stream >>>(...);

cudaStreamEndCapture(stream, &graph);

cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
cudaGraphLaunch(graphExec, stream);

```

4.2.3. Updating Instantiated Graphs

When a workflow changes, the graph becomes out of date and must be modified. Major changes to graph structure (such as topology or node types) require re-instantiation because topology-related optimizations must be re-applied. However, it is common for only node parameters (such as kernel parameters and memory addresses) to change while the graph topology remains the same. For this case, CUDA provides a lightweight “Graph Update” mechanism that allows certain node parameters to be modified in-place without rebuilding the entire graph, which is much more efficient than re-instantiation.

Updates take effect the next time the graph is launched, so they do not impact previous graph launches, even if they are running at the time of the update. A graph may be updated and relaunched repeatedly, so multiple updates/launches can be queued on a stream.

CUDA provides two mechanisms for updating instantiated graph parameters, whole graph update and individual node update. Whole graph update allows the user to supply a topologically identical `cudaGraph_t` object whose nodes contain updated parameters. Individual node update allows the user to explicitly update the parameters of individual nodes. Using an updated `cudaGraph_t` is more convenient when a large number of nodes are being updated, or when the graph topology is unknown to the caller (i.e., The graph resulted from stream capture of a library call). Using individual node update is preferred when the number of changes is small and the user has the handles to the nodes requiring updates. Individual node update skips the topology checks and comparisons for unchanged nodes, so it can be more efficient in many cases.

CUDA also provides a mechanism for enabling and disabling individual nodes without affecting their current parameters.

The following sections explain each approach in more detail.

4.2.3.1 Whole Graph Update

`cudaGraphExecUpdate()` allows an instantiated graph (the “original graph”) to be updated with the parameters from a topologically identical graph (the “updating” graph). The topology of the updating graph must be identical to the original graph used to instantiate the `cudaGraphExec_t`. In addition, the order in which the dependencies are specified must match. Finally, CUDA needs to consistently order the sink nodes (nodes with no dependencies). CUDA relies on the order of specific API calls to achieve consistent sink node ordering.

More explicitly, following the following rules will cause `cudaGraphExecUpdate()` to pair the nodes in the original graph and the updating graph deterministically:

1. For any capturing stream, the API calls operating on that stream must be made in the same order, including event wait and other API calls not directly corresponding to node creation.
2. The API calls which directly manipulate a given graph node’s incoming edges (including captured stream APIs, node add APIs, and edge addition / removal APIs) must be made in the same order. Moreover, when dependencies are specified in arrays to these APIs, the order in which the dependencies are specified inside those arrays must match.
3. Sink nodes must be consistently ordered. Sink nodes are nodes without dependent nodes / outgoing edges in the final graph at the time of the `cudaGraphExecUpdate()` invocation. The following operations affect sink node ordering (if present) and must (as a combined set) be made in the same order:
 - ▶ Node add APIs resulting in a sink node.
 - ▶ Edge removal resulting in a node becoming a sink node.
 - ▶ `cudaStreamUpdateCaptureDependencies()`, if it removes a sink node from a capturing stream’s dependency set.
 - ▶ `cudaStreamEndCapture()`.

The following example shows how the API could be used to update an instantiated graph:

```
cudaGraphExec_t graphExec = NULL;

for (int i = 0; i < 10; i++) {
    cudaGraph_t graph;
```

(continues on next page)

(continued from previous page)

```

cudaGraphExecUpdateResult updateResult;
cudaGraphNode_t errorNode;

// In this example we use stream capture to create the graph.
// You can also use the Graph API to produce a graph.
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);

// Call a user-defined, stream based workload, for example
do_cuda_work(stream);

cudaStreamEndCapture(stream, &graph);

// If we've already instantiated the graph, try to update it directly
// and avoid the instantiation overhead
if (graphExec != NULL) {
    // If the graph fails to update, errorNode will be set to the
    // node causing the failure and updateResult will be set to a
    // reason code.
    cudaGraphExecUpdate(graphExec, graph, &errorNode, &updateResult);
}

// Instantiate during the first iteration or whenever the update
// fails for any reason
if (graphExec == NULL || updateResult != cudaGraphExecUpdateSuccess) {
    // If a previous update failed, destroy the cudaGraphExec_t
    // before re-instantiating it
    if (graphExec != NULL) {
        cudaGraphExecDestroy(graphExec);
    }
    // Instantiate graphExec from graph. The error node and
    // error message parameters are unused here.
    cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
}

cudaGraphDestroy(graph);
cudaGraphLaunch(graphExec, stream);
cudaStreamSynchronize(stream);
}

```

A typical workflow is to create the initial `cudaGraph_t` using either the stream capture or graph API. The `cudaGraph_t` is then instantiated and launched as normal. After the initial launch, a new `cudaGraph_t` is created using the same method as the initial graph and `cudaGraphExecUpdate()` is called. If the graph update is successful, indicated by the `updateResult` parameter in the above example, the updated `cudaGraphExec_t` is launched. If the update fails for any reason, the `cudaGraphExecDestroy()` and `cudaGraphInstantiate()` are called to destroy the original `cudaGraphExec_t` and instantiate a new one.

It is also possible to update the `cudaGraph_t` nodes directly (i.e., using `cudaGraphKernelNodeSetParams()`) and subsequently update the `cudaGraphExec_t`, however it is more efficient to use the explicit node update APIs covered in the next section.

Conditional handle flags and default values are updated as part of the graph update.

Please see the [Graph API](#) for more information on usage and current limitations.

4.2.3.2 Individual Node Update

Instantiated graph node parameters can be updated directly. This eliminates the overhead of instantiation as well as the overhead of creating a new `cudaGraph_t`. If the number of nodes requiring update is small relative to the total number of nodes in the graph, it is better to update the nodes individually. The following methods are available for updating `cudaGraphExec_t` nodes:

Table 8: Individual Node Update APIs

API	Node Type
<code>cudaGraphExecKernelNodeSetParams()</code>	Kernel node
<code>cudaGraphExecMemcpyNodeSetParams()</code>	Memory copy node
<code>cudaGraphExecMemsetNodeSetParams()</code>	Memory set node
<code>cudaGraphExecHostNodeSetParams()</code>	Host node
<code>cudaGraphExecChildGraphNodeSetParams()</code>	Child graph node
<code>cudaGraphExecEventRecordNodeSetEvent()</code>	Event record node
<code>cudaGraphExecEventWaitNodeSetEvent()</code>	Event wait node
<code>cudaGraphExecExternalSemaphoresSignalNodeSetParams()</code>	External semaphore signal node
<code>cudaGraphExecExternalSemaphoresWaitNodeSetParams()</code>	External semaphore wait node

Please see the [Graph API](#) for more information on usage and current limitations.

4.2.3.3 Individual Node Enable

Kernel, memset and memcpy nodes in an instantiated graph can be enabled or disabled using the `cudaGraphNodeSetEnabled()` API. This allows the creation of a graph which contains a superset of the desired functionality which can be customized for each launch. The enable state of a node can be queried using the `cudaGraphNodeGetEnabled()` API.

A disabled node is functionally equivalent to an empty node until it is re-enabled. Node parameters are not affected by enabling/disabling a node. Enable state is unaffected by individual node update or whole graph update with `cudaGraphExecUpdate()`. Parameter updates while the node is disabled will take effect when the node is re-enabled.

Refer to the [Graph API](#) for more information on usage and current limitations.

4.2.3.4 Graph Update Limitations

Kernel nodes:

- The owning context of the function cannot change.
- A node whose function originally did not use CUDA dynamic parallelism cannot be updated to a function which uses CUDA dynamic parallelism.

`cudaMemset` and `cudaMemcpy` nodes:

- The CUDA device(s) to which the operand(s) was allocated/mapped cannot change.

- ▶ The source/destination memory must be allocated from the same context as the original source/destination memory.
- ▶ Only 1D `cudaMemset/cudaMemcpy` nodes can be changed.

Additional memcpy node restrictions:

- ▶ Changing either the source or destination memory type (i.e., `cudaPitchedPtr`, `cudaArray_t`, etc.), or the type of transfer (i.e., `cudaMemcpyKind`) is not supported.

External semaphore wait nodes and record nodes:

- ▶ Changing the number of semaphores is not supported.

Conditional nodes:

- ▶ The order of handle creation and assignment must match between the graphs.
- ▶ Changing node parameters is not supported (i.e. number of graphs in the conditional, node context, etc).
- ▶ Changing parameters of nodes within the conditional body graph is subject to the rules above.

Memory nodes:

- ▶ It is not possible to update a `cudaGraphExec_t` with a `cudaGraph_t` if the `cudaGraph_t` is currently instantiated as a different `cudaGraphExec_t`.

There are no restrictions on updates to host nodes, event record nodes, or event wait nodes.

4.2.4. Conditional Graph Nodes

Conditional nodes allow conditional execution and looping of a graph contained within the conditional node. This allows dynamic and iterative workflows to be represented completely within a graph and frees up the host CPU to perform other work in parallel.

Evaluation of the condition value is performed on the device when the dependencies of the conditional node have been met. Conditional nodes can be one of the following types:

- ▶ Conditional *IF nodes* execute their body graph once if the condition value is non-zero when the node is executed. An optional second body graph can be provided and this will be executed once if the condition value is zero when the node is executed.
- ▶ Conditional *WHILE nodes* execute their body graph if the condition value is non-zero when the node is executed and will continue to execute their body graph until the condition value is zero.
- ▶ Conditional *SWITCH nodes* execute the zero-indexed nth body graph once if the condition value is equal to n. If the condition value does not correspond to a body graph, no body graph is launched.

A condition value is accessed by a *conditional handle*, which must be created before the node. The condition value can be set by device code using `cudaGraphSetConditional()`. A default value, applied on each graph launch, can also be specified when the handle is created.

When the conditional node is created, an empty graph is created and the handle is returned to the user so that the graph can be populated. This conditional body graph can be populated using either the *graph APIs* or `cudaStreamBeginCaptureToGraph()`.

Conditional nodes can be nested.

4.2.4.1 Conditional Handles

A condition value is represented by `cudaGraphConditionalHandle` and is created by `cudaGraphConditionalHandleCreate()`.

The handle must be associated with a single conditional node. Handles cannot be destroyed and as such there is no need to keep track of them.

If `cudaGraphCondAssignDefault` is specified when the handle is created, the condition value will be initialized to the specified default at the beginning of each graph execution. If this flag is not provided, the condition value is undefined at the start of each graph execution and code should not assume that the condition value persists across executions.

The default value and flags associated with a handle will be updated during *whole graph update*.

4.2.4.2 Conditional Node Body Graph Requirements

General requirements:

- ▶ The graph's nodes must all reside on a single device.
- ▶ The graph can only contain kernel nodes, empty nodes, memcpy nodes, memset nodes, child graph nodes, and conditional nodes.

Kernel nodes:

- ▶ Use of CUDA Dynamic Parallelism or Device Graph Launch by kernels in the graph is not permitted.
- ▶ Cooperative launches are permitted so long as MPS is not in use.

Memcpy/Memset nodes:

- ▶ Only copies/memsets involving device memory and/or pinned device-mapped host memory are permitted.
- ▶ Copies/memsets involving CUDA arrays are not permitted.
- ▶ Both operands must be accessible from the current device at time of instantiation. Note that the copy operation will be performed from the device on which the graph resides, even if it is targeting memory on another device.

4.2.4.3 Conditional IF Nodes

The body graph of an IF node will be executed once if the condition is non-zero when the node is executed. The following diagram depicts a 3 node graph where the middle node, B, is a conditional node:

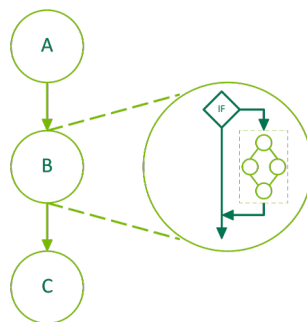


Figure 27: Conditional IF Node

The following code illustrates the creation of a graph containing an IF conditional node. The default value of the condition is set using an upstream kernel. The body of the conditional is populated using the *graph API*.

```
__global__ void setHandle(cudaGraphConditionalHandle handle, int value)
{
    ...
    // Set the condition value to the value passed to the kernel
    cudaGraphSetConditional(handle, value);
    ...
}

void graphSetup() {
    cudaGraph_t graph;
    cudaGraphExec_t graphExec;
    cudaGraphNode_t node;
    void *kernelArgs[2];
    int value = 1;

    // Create the graph
    cudaGraphCreate(&graph, 0);

    // Create the conditional handle; because no default value is provided,
    // the condition value is undefined at the start of each graph execution
    cudaGraphConditionalHandle handle;
    cudaGraphConditionalHandleCreate(&handle, graph);

    // Use a kernel upstream of the conditional to set the handle value
    cudaGraphNodeParams params = { cudaGraphNodeTypeKernel };
    params.kernel.func = (void *)setHandle;
    params.kernel.gridDim.x = params.kernel.gridDim.y = params.kernel.gridDim.
    z = 1;
    params.kernel.blockDim.x = params.kernel.blockDim.y = params.kernel.
    blockDim.z = 1;
    params.kernel.kernelParams = kernelArgs;
    kernelArgs[0] = &handle;
    kernelArgs[1] = &value;
    cudaGraphAddNode(&node, graph, NULL, 0, &params);

    // Create and add the conditional node
    cudaGraphNodeParams cParams = { cudaGraphNodeTypeConditional };
    cParams.conditional.handle = handle;
    cParams.conditional.type = cudaGraphCondTypeIf;
    cParams.conditional.size = 1; // There is only an "if" body graph
    cudaGraphAddNode(&node, graph, &node, 1, &cParams);

    // Get the body graph of the conditional node
    cudaGraph_t bodyGraph = cParams.conditional.phGraph_out[0];

    // Populate the body graph of the IF conditional node
    ...
    cudaGraphAddNode(&node, bodyGraph, NULL, 0, &params);
}
```

(continues on next page)

(continued from previous page)

```

// Instantiate and launch the graph
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
cudaGraphLaunch(graphExec, 0);
cudaDeviceSynchronize();

// Clean up
cudaGraphExecDestroy(graphExec);
cudaGraphDestroy(graph);
}

```

IF nodes can also have an optional second body graph which is executed once when the node is executed if the condition value is zero.

```

void graphSetup() {
    cudaGraph_t graph;
    cudaGraphExec_t graphExec;
    cudaGraphNode_t node;
    void *kernelArgs[2];
    int value = 1;

    // Create the graph
    cudaGraphCreate(&graph, 0);

    // Create the conditional handle; because no default value is provided,
    ↪ the condition value is undefined at the start of each graph execution
    cudaGraphConditionalHandle handle;
    cudaGraphConditionalHandleCreate(&handle, graph);

    // Use a kernel upstream of the conditional to set the handle value
    cudaGraphNodeParams params = { cudaGraphNodeTypeKernel };
    params.kernel.func = (void *)setHandle;
    params.kernel.gridDim.x = params.kernel.gridDim.y = params.kernel.gridDim.
    ↪ z = 1;
    params.kernel.blockDim.x = params.kernel.blockDim.y = params.kernel.
    ↪ blockDim.z = 1;
    params.kernel.kernelParams = kernelArgs;
    kernelArgs[0] = &handle;
    kernelArgs[1] = &value;
    cudaGraphAddNode(&node, graph, NULL, 0, &params);

    // Create and add the IF conditional node
    cudaGraphNodeParams cParams = { cudaGraphNodeTypeConditional };
    cParams.conditional.handle = handle;
    cParams.conditional.type = cudaGraphCondTypeIf;
    cParams.conditional.size = 2; // There is both an "if" and an "else"
    ↪ body graph
    cudaGraphAddNode(&node, graph, &node, 1, &cParams);

    // Get the body graphs of the conditional node
    cudaGraph_t ifBodyGraph = cParams.conditional.phGraph_out[0];
    cudaGraph_t elseBodyGraph = cParams.conditional.phGraph_out[1];
}

```

(continues on next page)

(continued from previous page)

```
// Populate the body graphs of the IF conditional node
...
cudaGraphAddNode(&node, ifBodyGraph, NULL, 0, &params);
...
cudaGraphAddNode(&node, elseBodyGraph, NULL, 0, &params);

// Instantiate and launch the graph
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
cudaGraphLaunch(graphExec, 0);
cudaDeviceSynchronize();

// Clean up
cudaGraphExecDestroy(graphExec);
cudaGraphDestroy(graph);
}
```

4.2.4.4 Conditional WHILE Nodes

The body graph of a WHILE node will be executed as long as the condition is non-zero. The condition will be evaluated when the node is executed and after completion of the body graph. The following diagram depicts a 3 node graph where the middle node, B, is a conditional node:

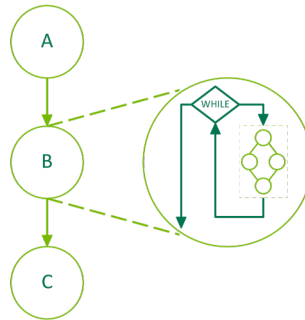


Figure 28: Conditional WHILE Node

The following code illustrates the creation of a graph containing a WHILE conditional node. The handle is created using `cudaGraphCondAssignDefault` to avoid the need for an upstream kernel. The body of the conditional is populated using the [graph API](#).

```
__global__ void loopKernel(cudaGraphConditionalHandle handle, char *dPtr)
{
    // Decrement the value of dPtr and set the condition value to 0 once dPtr
    is 0
    if (--(*dPtr) == 0) {
        cudaGraphSetConditional(handle, 0);
    }
}

void graphSetup() {
    cudaGraph_t graph;
    cudaGraphExec_t graphExec;
```

(continues on next page)

(continued from previous page)

```

cudaGraphNode_t node;
void *kernelArgs[2];

// Allocate a byte of device memory to use as input
char *dPtr;
cudaMalloc((void **)&dPtr, 1);

// Create the graph
cudaGraphCreate(&graph, 0);

// Create the conditional handle with a default value of 1
cudaGraphConditionalHandle handle;
cudaGraphConditionalHandleCreate(&handle, graph, 1,
→ cudaGraphCondAssignDefault);

// Create and add the WHILE conditional node
cudaGraphNodeParams cParams = { cudaGraphNodeTypeConditional };
cParams.conditional.handle = handle;
cParams.conditional.type = cudaGraphCondTypeWhile;
cParams.conditional.size = 1;
cudaGraphAddNode(&node, graph, NULL, 0, &cParams);

// Get the body graph of the conditional node
cudaGraph_t bodyGraph = cParams.conditional.phGraph_out[0];

// Populate the body graph of the conditional node
cudaGraphNodeParams params = { cudaGraphNodeTypeKernel };
params.kernel.func = (void *)loopKernel;
params.kernel.gridDim.x = params.kernel.gridDim.y = params.kernel.gridDim.
→ z = 1;
params.kernel.blockDim.x = params.kernel.blockDim.y = params.kernel.
→ blockDim.z = 1;
params.kernel.kernelParams = kernelArgs;
kernelArgs[0] = &handle;
kernelArgs[1] = &dPtr;
cudaGraphAddNode(&node, bodyGraph, NULL, 0, &params);

// Initialize device memory, instantiate, and launch the graph
cudaMemset(dPtr, 10, 1); // Set dPtr to 10; the loop will run until dPtr
→ is 0
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
cudaGraphLaunch(graphExec, 0);
cudaDeviceSynchronize();

// Clean up
cudaGraphExecDestroy(graphExec);
cudaGraphDestroy(graph);
cudaFree(dPtr);
}

```

4.2.4.5 Conditional SWITCH Nodes

The zero-indexed *n*th body graph of a SWITCH node will be executed once if the condition is equal to *n* when the node is executed. The following diagram depicts a 3 node graph where the middle node, B, is a conditional node:

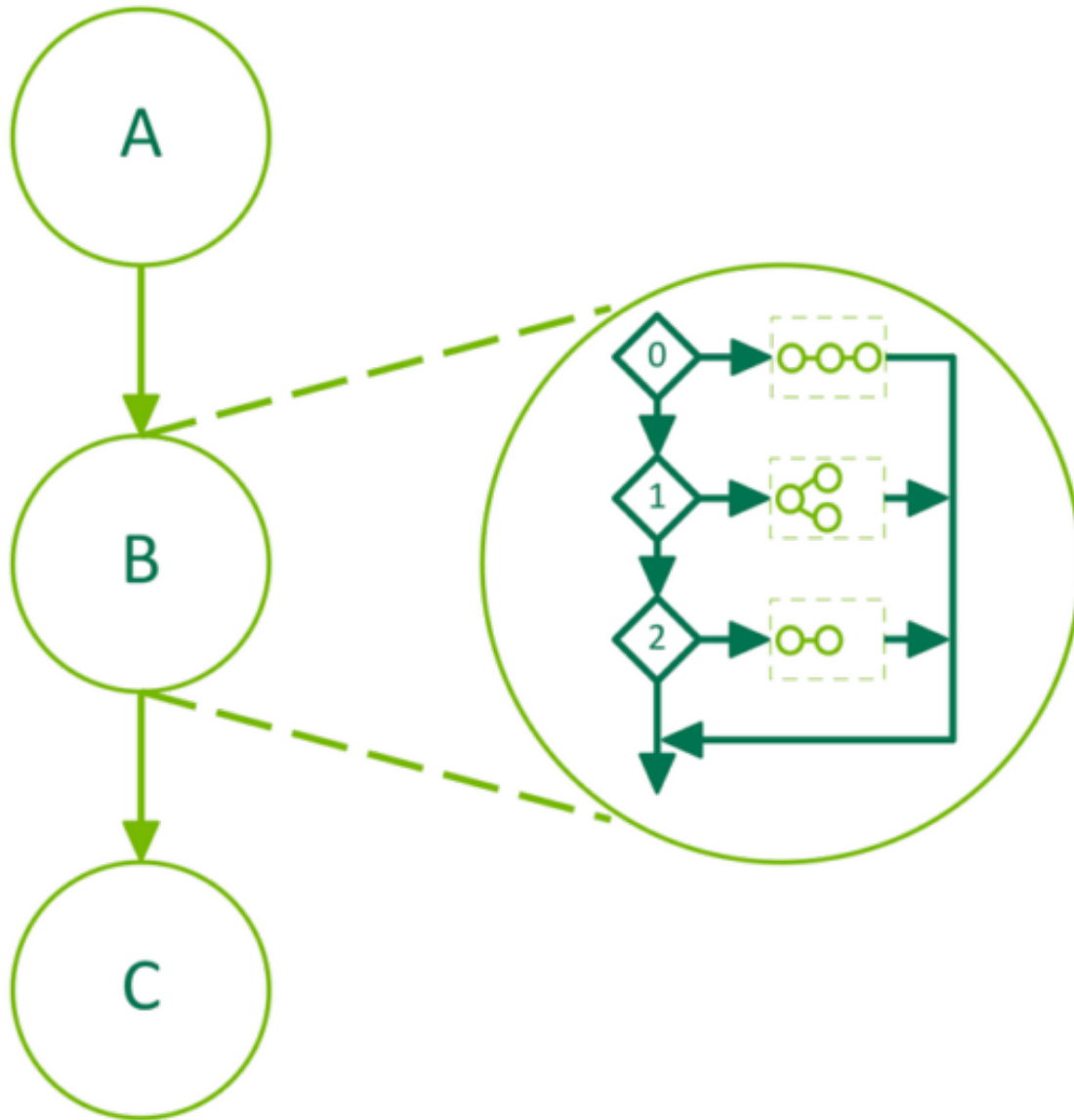


Figure 29: Conditional SWITCH Node

The following code illustrates the creation of a graph containing a SWITCH conditional node. The value of the condition is set using an upstream kernel. The bodies of the conditional are populated using the *graph API*.

```
__global__ void setHandle(cudaGraphConditionalHandle handle, int value)
{
    ...
    // Set the condition value to the value passed to the kernel
```

(continues on next page)

(continued from previous page)

```

    cudaGraphSetConditional(handle, value);
    ...
}

void graphSetup() {
    cudaGraph_t graph;
    cudaGraphExec_t graphExec;
    cudaGraphNode_t node;
    void *kernelArgs[2];
    int value = 1;

    // Create the graph
    cudaGraphCreate(&graph, 0);

    // Create the conditional handle; because no default value is provided,
    ↪ the condition value is undefined at the start of each graph execution
    cudaGraphConditionalHandle handle;
    cudaGraphConditionalHandleCreate(&handle, graph);

    // Use a kernel upstream of the conditional to set the handle value
    cudaGraphNodeParams params = { cudaGraphNodeTypeKernel };
    params.kernel.func = (void *)setHandle;
    params.kernel.gridDim.x = params.kernel.gridDim.y = params.kernel.gridDim.
    ↪ z = 1;
    params.kernel.blockDim.x = params.kernel.blockDim.y = params.kernel.
    ↪ blockDim.z = 1;
    params.kernel.kernelParams = kernelArgs;
    kernelArgs[0] = &handle;
    kernelArgs[1] = &value;
    cudaGraphAddNode(&node, graph, NULL, 0, &params);

    // Create and add the conditional SWITCH node
    cudaGraphNodeParams cParams = { cudaGraphNodeTypeConditional };
    cParams.conditional.handle = handle;
    cParams.conditional.type = cudaGraphCondTypeSwitch;
    cParams.conditional.size = 5;
    cudaGraphAddNode(&node, graph, &node, 1, &cParams);

    // Get the body graphs of the conditional node
    cudaGraph_t *bodyGraphs = cParams.conditional.phGraph_out;

    // Populate the body graphs of the SWITCH conditional node
    ...
    cudaGraphAddNode(&node, bodyGraphs[0], NULL, 0, &params);
    ...
    cudaGraphAddNode(&node, bodyGraphs[4], NULL, 0, &params);

    // Instantiate and launch the graph
    cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
    cudaGraphLaunch(graphExec, 0);
    cudaDeviceSynchronize();
}

```

(continues on next page)

(continued from previous page)

```
// Clean up
cudaGraphExecDestroy(graphExec);
cudaGraphDestroy(graph);
}
```

4.2.5. Graph Memory Nodes

4.2.5.1 Introduction

Graph memory nodes allow graphs to create and own memory allocations. Graph memory nodes have GPU ordered lifetime semantics, which dictate when memory is allowed to be accessed on the device. These GPU ordered lifetime semantics enable driver-managed memory reuse, and match those of the stream ordered allocation APIs `cudaMallocAsync` and `cudaFreeAsync`, which may be captured when creating a graph.

Graph allocations have fixed addresses over the life of a graph including repeated instantiations and launches. This allows the memory to be directly referenced by other operations within the graph without the need of a graph update, even when CUDA changes the backing physical memory. Within a graph, allocations whose graph ordered lifetimes do not overlap may use the same underlying physical memory.

CUDA may reuse the same physical memory for allocations across multiple graphs, aliasing virtual address mappings according to the GPU ordered lifetime semantics. For example when different graphs are launched into the same stream, CUDA may virtually alias the same physical memory to satisfy the needs of allocations which have single-graph lifetimes.

4.2.5.2 API Fundamentals

Graph memory nodes are graph nodes representing either memory allocation or free actions. As a shorthand, nodes that allocate memory are called allocation nodes. Likewise, nodes that free memory are called free nodes. Allocations created by allocation nodes are called graph allocations. CUDA assigns virtual addresses for the graph allocation at node creation time. While these virtual addresses are fixed for the lifetime of the allocation node, the allocation contents are not persistent past the freeing operation and may be overwritten by accesses referring to a different allocation.

Graph allocations are considered recreated every time a graph runs. A graph allocation's lifetime, which differs from the node's lifetime, begins when GPU execution reaches the allocating graph node and ends when one of the following occurs:

- ▶ GPU execution reaches the freeing graph node
- ▶ GPU execution reaches the freeing `cudaFreeAsync()` stream call
- ▶ immediately upon the freeing call to `cudaFree()`

Note

Graph destruction does not automatically free any live graph-allocated memory, even though it ends the lifetime of the allocation node. The allocation must subsequently be freed in another graph, or using `cudaFreeAsync()` / `cudaFree()`.

Just like other *graph-structure*, graph memory nodes are ordered within a graph by dependency edges. A program must guarantee that operations accessing graph memory:

- ▶ are ordered after the allocation node
- ▶ are ordered before the operation freeing the memory

Graph allocation lifetimes begin and usually end according to GPU execution (as opposed to API invocation). GPU ordering is the order that work runs on the GPU as opposed to the order that the work is enqueued or described. Thus, graph allocations are considered ‘GPU ordered.’

4.2.5.2.1 Graph Node APIs

Graph memory nodes may be explicitly created with the node creation API, `cudaGraphAddNode`. The address allocated when adding a `cudaGraphNodeTypeMemAlloc` node is returned to the user in the `alloc::dptr` field of the passed `cudaGraphNodeParams` structure. All operations using graph allocations inside the allocating graph must be ordered after the allocating node. Similarly, any free nodes must be ordered after all uses of the allocation within the graph. Free nodes are created using `cudaGraphAddNode` and a node type of `cudaGraphNodeTypeMemFree`.

In the following figure, there is an example graph with an alloc and a free node. Kernel nodes **a**, **b**, and **c** are ordered after the allocation node and before the free node such that the kernels can access the allocation. Kernel node **e** is not ordered after the alloc node and therefore cannot safely access the memory. Kernel node **d** is not ordered before the free node, therefore it cannot safely access the memory.

The following code snippet establishes the graph in this figure:

```
// Create the graph - it starts out empty
cudaGraphCreate(&graph, 0);

// parameters for a basic allocation
cudaGraphNodeParams params = { cudaGraphNodeTypeMemAlloc };
params.alloc.poolProps.allocType = cudaMemAllocationTypePinned;
params.alloc.poolProps.location.type = cudaMemLocationTypeDevice;
// specify device 0 as the resident device
params.alloc.poolProps.location.id = 0;
params.alloc.bytesize = size;

cudaGraphAddNode(&allocNode, graph, NULL, NULL, 0, &params);

// create a kernel node that uses the graph allocation
cudaGraphNodeParams nodeParams = { cudaGraphNodeTypeKernel };
nodeParams.kernel.kernelParams[0] = params.alloc.dptr;
// ...set other kernel node parameters...

// add the kernel node to the graph
cudaGraphAddNode(&a, graph, &allocNode, NULL, 1, &nodeParams);
cudaGraphAddNode(&b, graph, &a, NULL, 1, &nodeParams);
cudaGraphAddNode(&c, graph, &a, NULL, 1, &nodeParams);
cudaGraphNode_t dependencies[2];
// kernel nodes b and c are using the graph allocation, so the freeing node
↪ must depend on them. Since the dependency of node b on node a establishes
↪ an indirect dependency, the free node does not need to explicitly depend on
↪ node a.
dependencies[0] = b;
dependencies[1] = c;
cudaGraphNodeParams freeNodeParams = { cudaGraphNodeTypeMemFree };
```

(continues on next page)

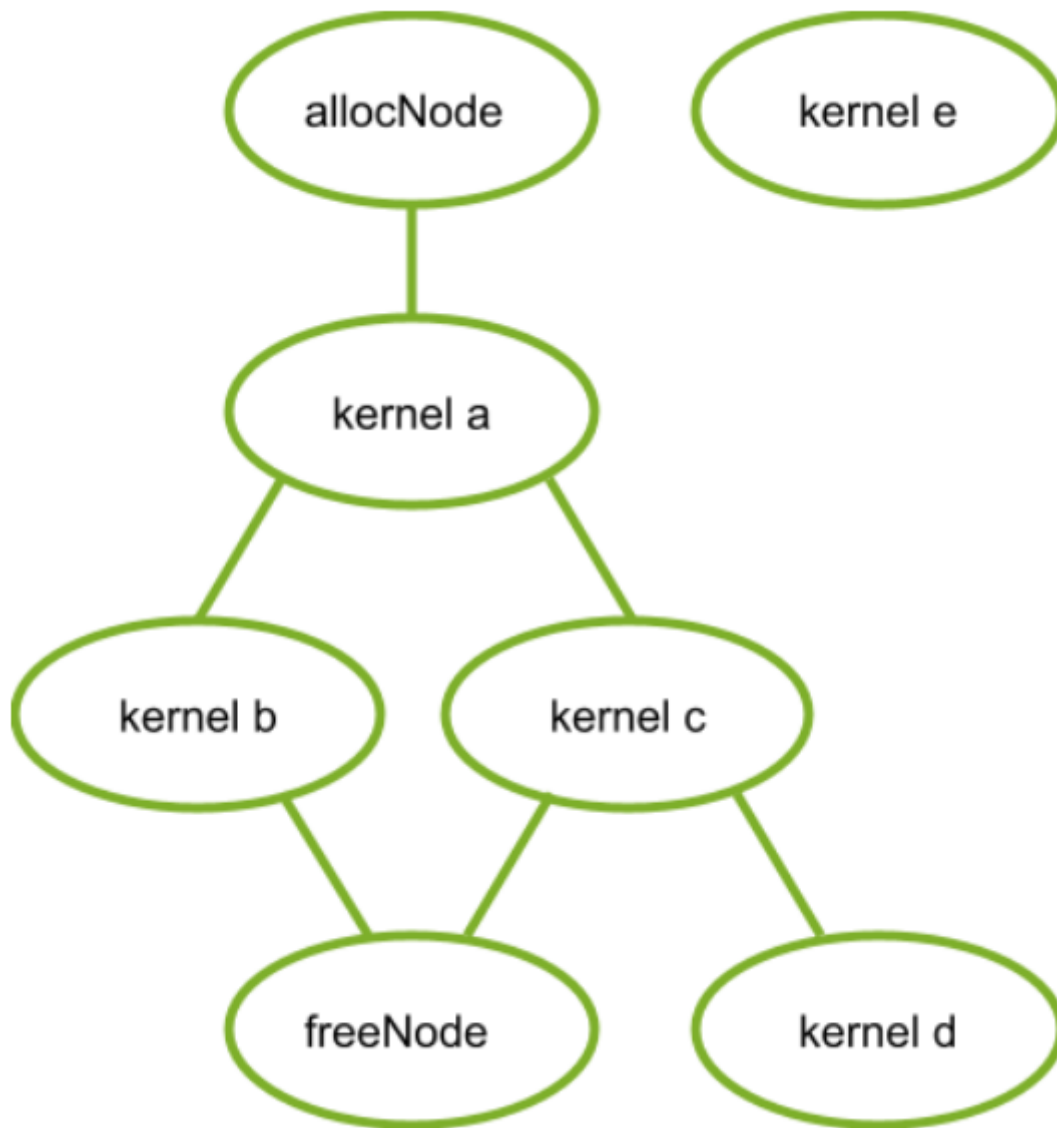


Figure 30: Kernel Nodes

(continued from previous page)

```

freeNodeParams.free.dptr = params.alloc.dptr;
cudaGraphAddNode(&freeNode, graph, dependencies, NULL, 2, freeNodeParams);
// free node does not depend on kernel node d, so it must not access the freed
→graph allocation.
cudaGraphAddNode(&d, graph, &c, NULL, 1, &nodeParams);

// node e does not depend on the allocation node, so it must not access the
→allocation. This would be true even if the freeNode depended on kernel node
→e.
cudaGraphAddNode(&e, graph, NULL, NULL, 0, &nodeParams);

```

4.2.5.2.2 Stream Capture

Graph memory nodes can be created by capturing the corresponding stream ordered allocation and free calls `cudaMallocAsync` and `cudaFreeAsync`. In this case, the virtual addresses returned by the captured allocation API can be used by other operations inside the graph. Since the stream ordered dependencies will be captured into the graph, the ordering requirements of the stream ordered allocation APIs guarantee that the graph memory nodes will be properly ordered with respect to the captured stream operations (for correctly written stream code).

Ignoring kernel nodes **d** and **e**, for clarity, the following code snippet shows how to use stream capture to create the graph from the previous figure:

```

cudaMallocAsync(&dptr, size, stream1);
kernel_A<<< ..., stream1 >>>(dptr, ...);

// Fork into stream2
cudaEventRecord(event1, stream1);
cudaStreamWaitEvent(stream2, event1);

kernel_B<<< ..., stream1 >>>(dptr, ...);
// event dependencies translated into graph dependencies, so the kernel node
→created by the capture of kernel C will depend on the allocation node
→created by capturing the cudaMallocAsync call.
kernel_C<<< ..., stream2 >>>(dptr, ...);

// Join stream2 back to origin stream (stream1)
cudaEventRecord(event2, stream2);
cudaStreamWaitEvent(stream1, event2);

// Free depends on all work accessing the memory.
cudaFreeAsync(dptr, stream1);

// End capture in the origin stream
cudaStreamEndCapture(stream1, &graph);

```

4.2.5.2.3 Accessing and Freeing Graph Memory Outside of the Allocating Graph

Graph allocations do not have to be freed by the allocating graph. When a graph does not free an allocation, that allocation persists beyond the execution of the graph and can be accessed by subsequent CUDA operations. These allocations may be accessed in another graph or directly using a stream operation as long as the accessing operation is ordered after the allocation through CUDA events and other

stream ordering mechanisms. An allocation may subsequently be freed by regular calls to `cudaFree`, `cudaFreeAsync`, or by the launch of another graph with a corresponding free node, or a subsequent launch of the allocating graph (if it was instantiated with the `cudaGraphInstantiateFlagAutoFreeOnLaunch` flag). It is illegal to access memory after it has been freed - the free operation must be ordered after all operations accessing the memory using graph dependencies, CUDA events, and other stream ordering mechanisms.

Note

Since graph allocations may share underlying physical memory, free operations must be ordered after all device operations complete. Out-of-band synchronization (such as memory-based synchronization within a compute kernel) is insufficient for ordering between memory writes and free operations. For more information, see the [Virtual Aliasing Support](#) rules relating to consistency and coherency.

The three following code snippets demonstrate accessing graph allocations outside of the allocating graph with ordering properly established by: using a single stream, using events between streams, and using events baked into the allocating and freeing graph.

First, ordering established by using a single stream:

```
// Contents of allocating graph
void *dptr;
cudaGraphNodeParams params = { cudaGraphNodeTypeMemAlloc };
params.alloc.poolProps.allocType = cudaMemAllocationTypePinned;
params.alloc.poolProps.location.type = cudaMemLocationTypeDevice;
params.alloc.bytesize = size;
cudaGraphAddNode(&allocNode, allocGraph, NULL, NULL, 0, &params);
dptr = params.alloc.dptr;

cudaGraphInstantiate(&allocGraphExec, allocGraph, NULL, NULL, 0);

cudaGraphLaunch(allocGraphExec, stream);
kernel<<< ..., stream >>>(dptr, ...);
cudaFreeAsync(dptr, stream);
```

Second, ordering established by recording and waiting on CUDA events:

```
// Contents of allocating graph
void *dptr;

// Contents of allocating graph
cudaGraphAddNode(&allocNode, allocGraph, NULL, NULL, 0, &allocNodeParams);
dptr = allocNodeParams.alloc.dptr;

// contents of consuming/freeing graph
kernelNodeParams.kernel.kernelParams[0] = allocNodeParams.alloc.dptr;
cudaGraphAddNode(&freeNode, freeGraph, NULL, NULL, 1, dptr);

cudaGraphInstantiate(&allocGraphExec, allocGraph, NULL, NULL, 0);
cudaGraphInstantiate(&freeGraphExec, freeGraph, NULL, NULL, 0);

cudaGraphLaunch(allocGraphExec, allocStream);
```

(continues on next page)

(continued from previous page)

```

// establish the dependency of stream2 on the allocation node
// note: the dependency could also have been established with a stream
↳synchronize operation
cudaEventRecord(allocEvent, allocStream);
cudaStreamWaitEvent(stream2, allocEvent);

kernel<<< ..., stream2 >>> (dpPtr, ...);

// establish the dependency between the stream 3 and the allocation use
cudaStreamRecordEvent(streamUseDoneEvent, stream2);
cudaStreamWaitEvent(stream3, streamUseDoneEvent);

// it is now safe to launch the freeing graph, which may also access the memory
cudaGraphLaunch(freeGraphExec, stream3);

```

Third, ordering established by using graph external event nodes:

```

// Contents of allocating graph
void *dpPtr;
cudaEvent_t allocEvent; // event indicating when the allocation will be ready
↳for use.
cudaEvent_t streamUseDoneEvent; // event indicating when the stream operations
↳are done with the allocation.

// Contents of allocating graph with event record node
cudaGraphAddNode(&allocNode, allocGraph, NULL, NULL, 0, &allocNodeParams);
dpPtr = allocNodeParams.alloc.dpPtr;
// note: this event record node depends on the alloc node

cudaGraphNodeParams allocEventNodeParams = { cudaGraphNodeTypeEventRecord };
allocEventNodeParams.eventRecord.event = allocEvent;
cudaGraphAddNode(&recordNode, allocGraph, &allocNode, NULL, 1,
↳allocEventNodeParams);
cudaGraphInstantiate(&allocGraphExec, allocGraph, NULL, NULL, 0);

// contents of consuming/freeing graph with event wait nodes
cudaGraphNodeParams streamWaitEventNodeParams = { cudaGraphNodeTypeEventWait }
↳;
streamWaitEventNodeParams.eventWait.event = streamUseDoneEvent;
cudaGraphAddNode(&streamUseDoneEventNode, waitAndFreeGraph, NULL, NULL, 0,
↳streamWaitEventNodeParams);

cudaGraphNodeParams allocWaitEventNodeParams = { cudaGraphNodeTypeEventWait };
allocWaitEventNodeParams.eventWait.event = allocEvent;
cudaGraphAddNode(&allocReadyEventNode, waitAndFreeGraph, NULL, NULL, 0,
↳allocWaitEventNodeParams);

kernelNodeParams->kernelParams[0] = allocNodeParams.alloc.dpPtr;

// The allocReadyEventNode provides ordering with the alloc node for use in a
↳consuming graph.

```

(continues on next page)

(continued from previous page)

```

cudaGraphAddNode(&kernelNode, waitAndFreeGraph, &allocReadyEventNode, NULL, 1,
    ↪ &kernelNodeParams);

// The free node has to be ordered after both external and internal users.
// Thus the node must depend on both the kernelNode and the
    ↪ streamUseDoneEventNode.
dependencies[0] = kernelNode;
dependencies[1] = streamUseDoneEventNode;

cudaGraphNodeParams freeNodeParams = { cudaGraphNodeTypeMemFree };
freeNodeParams.free.dptr = dptr;
cudaGraphAddNode(&freeNode, waitAndFreeGraph, &dependencies, NULL, 2,
    ↪ freeNodeParams);
cudaGraphInstantiate(&waitAndFreeGraphExec, waitAndFreeGraph, NULL, NULL, 0);

cudaGraphLaunch(allocGraphExec, allocStream);

// establish the dependency of stream2 on the event node satisfies the ordering
    ↪ requirement
cudaStreamWaitEvent(stream2, allocEvent);
kernel<<< ..., stream2 >>> (dpPtr, ...);
cudaStreamRecordEvent(streamUseDoneEvent, stream2);

// the event wait node in the waitAndFreeGraphExec establishes the dependency
    ↪ on the "readyForFreeEvent" that is needed to prevent the kernel running in
    ↪ stream two from accessing the allocation after the free node in execution
    ↪ order.
cudaGraphLaunch(waitAndFreeGraphExec, stream3);

```

4.2.5.2.4 cudaGraphInstantiateFlagAutoFreeOnLaunch

Under normal circumstances, CUDA will prevent a graph from being relaunched if it has unfreed memory allocations because multiple allocations at the same address will leak memory. Instantiating a graph with the `cudaGraphInstantiateFlagAutoFreeOnLaunch` flag allows the graph to be relaunched while it still has unfreed allocations. In this case, the launch automatically inserts an asynchronous free of the unfreed allocations.

Auto free on launch is useful for single-producer multiple-consumer algorithms. At each iteration, a producer graph creates several allocations, and, depending on runtime conditions, a varying set of consumers accesses those allocations. This type of variable execution sequence means that consumers cannot free the allocations because a subsequent consumer may require access. Auto free on launch means that the launch loop does not need to track the producer's allocations - instead, that information remains isolated to the producer's creation and destruction logic. In general, auto free on launch simplifies an algorithm which would otherwise need to free all the allocations owned by a graph before each relaunch.

Note

The `cudaGraphInstantiateFlagAutoFreeOnLaunch` flag does not change the behavior of graph destruction. The application must explicitly free the unfreed memory in order to avoid memory leaks, even for graphs instantiated with the flag. The following code shows the use of `cudaGraphInstantiateFlagAutoFreeOnLaunch` to simplify a single-producer / multiple-consumer

algorithm:

```
// Create producer graph which allocates memory and populates it with data
cudaStreamBeginCapture(cudaStreamPerThread, cudaStreamCaptureModeGlobal);
cudaMallocAsync(&data1, blocks * threads, cudaStreamPerThread);
cudaMallocAsync(&data2, blocks * threads, cudaStreamPerThread);
produce<<<blocks, threads, 0, cudaStreamPerThread>>>(data1, data2);
...
cudaStreamEndCapture(cudaStreamPerThread, &graph);
cudaGraphInstantiateWithFlags(&producer,
                             graph,
                             cudaGraphInstantiateFlagAutoFreeOnLaunch);
cudaGraphDestroy(graph);

// Create first consumer graph by capturing an asynchronous library call
cudaStreamBeginCapture(cudaStreamPerThread, cudaStreamCaptureModeGlobal);
consumerFromLibrary(data1, cudaStreamPerThread);
cudaStreamEndCapture(cudaStreamPerThread, &graph);
cudaGraphInstantiateWithFlags(&consumer1, graph, 0); //regular instantiation
cudaGraphDestroy(graph);

// Create second consumer graph
cudaStreamBeginCapture(cudaStreamPerThread, cudaStreamCaptureModeGlobal);
consume2<<<blocks, threads, 0, cudaStreamPerThread>>>(data2);
...
cudaStreamEndCapture(cudaStreamPerThread, &graph);
cudaGraphInstantiateWithFlags(&consumer2, graph, 0);
cudaGraphDestroy(graph);

// Launch in a loop
bool launchConsumer2 = false;
do {
    cudaGraphLaunch(producer, myStream);
    cudaGraphLaunch(consumer1, myStream);
    if (launchConsumer2) {
        cudaGraphLaunch(consumer2, myStream);
    }
} while (determineAction(&launchConsumer2));

cudaFreeAsync(data1, myStream);
cudaFreeAsync(data2, myStream);

cudaGraphExecDestroy(producer);
cudaGraphExecDestroy(consumer1);
cudaGraphExecDestroy(consumer2);
```

4.2.5.2.5 Memory Nodes in Child Graphs

CUDA 12.9 introduces the ability to move child graph ownership to a parent graph. Child graphs which are moved to the parent are allowed to contain memory allocation and free nodes. This allows a child graph containing allocation or free nodes to be independently constructed prior to its addition in a parent graph.

The following restrictions apply to child graphs after they have been moved:

- ▶ Cannot be independently instantiated or destroyed.
- ▶ Cannot be added as a child graph of a separate parent graph.
- ▶ Cannot be used as an argument to `cuGraphExecUpdate`.
- ▶ Cannot have additional memory allocation or free nodes added.

```
// Create the child graph
cudaGraphCreate(&child, 0);

// parameters for a basic allocation
cudaGraphNodeParams allocNodeParams = { cudaGraphNodeTypeMemAlloc };
allocNodeParams.alloc.poolProps.allocType = cudaMemAllocationTypePinned;
allocNodeParams.alloc.poolProps.location.type = cudaMemLocationTypeDevice;
// specify device 0 as the resident device
allocNodeParams.alloc.poolProps.location.id = 0;
allocNodeParams.alloc.bytesize = size;

cudaGraphAddNode(&allocNode, child, NULL, NULL, 0, &allocNodeParams);
// Additional nodes using the allocation could be added here
cudaGraphNodeParams freeNodeParams = { cudaGraphNodeTypeMemFree };
freeNodeParams.free.dptr = allocNodeParams.alloc.dptr;
cudaGraphAddNode(&freeNode, child, &allocNode, NULL, 1, freeNodeParams);

// Create the parent graph
cudaGraphCreate(&parent, 0);

// Move the child graph to the parent graph
cudaGraphNodeParams childNodeParams = { cudaGraphNodeTypeGraph };
childNodeParams.graph.graph = child;
childNodeParams.graph.ownership = cudaGraphChildGraphOwnershipMove;
cudaGraphAddNode(&parentNode, parent, NULL, NULL, 0, &childNodeParams);
```

4.2.5.3 Optimized Memory Reuse

CUDA reuses memory in two ways:

- ▶ Virtual and physical memory reuse within a graph is based on virtual address assignment, like in the stream ordered allocator.
- ▶ Physical memory reuse between graphs is done with virtual aliasing: different graphs can map the same physical memory to their unique virtual addresses.

4.2.5.3.1 Address Reuse within a Graph

CUDA may reuse memory within a graph by assigning the same virtual address ranges to different allocations whose lifetimes do not overlap. Since virtual addresses may be reused, pointers to different allocations with disjoint lifetimes are not guaranteed to be unique.

The following figure shows adding a new allocation node (2) that can reuse the address freed by a dependent node (1).



Figure 31: Adding New Alloc Node 2

The following figure shows adding a new alloc node (4). The new alloc node is not dependent on the free node (2) so cannot reuse the address from the associated alloc node (2). If the alloc node (2) used the address freed by free node (1), the new alloc node 3 would need a new address.

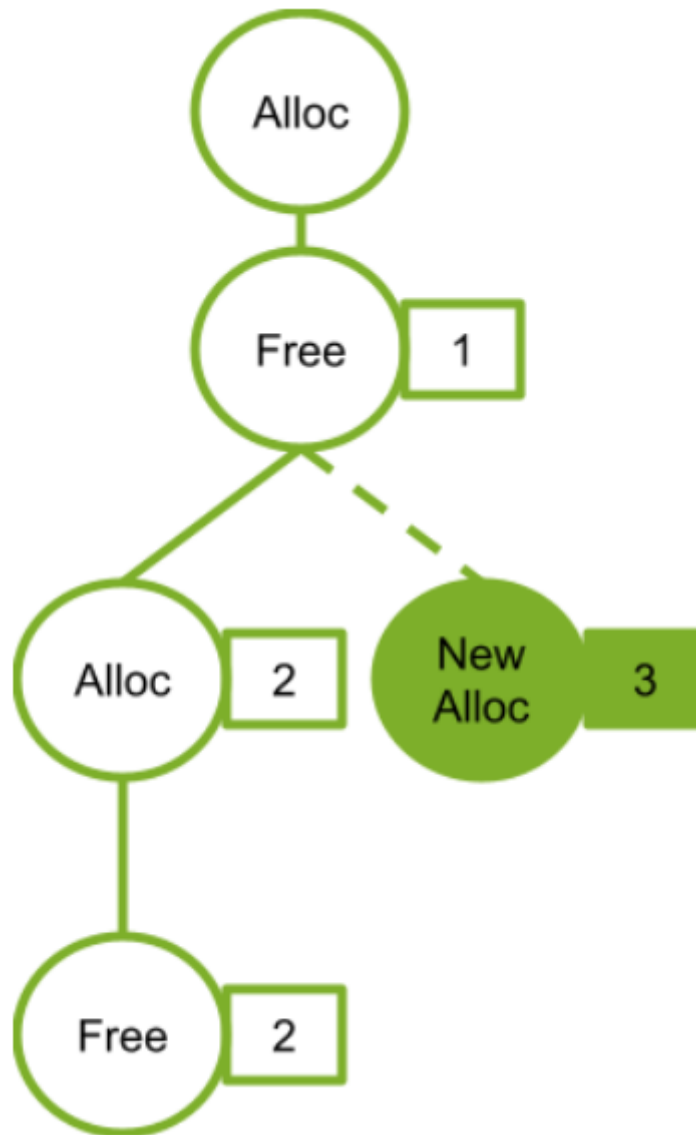


Figure 32: Adding New Alloc Node 3

4.2.5.3.2 Physical Memory Management and Sharing

CUDA is responsible for mapping physical memory to the virtual address before the allocating node is reached in GPU order. As an optimization for memory footprint and mapping overhead, multiple graphs may use the same physical memory for distinct allocations if they will not run simultaneously; however, physical pages cannot be reused if they are bound to more than one executing graph at the same time, or to a graph allocation which remains unfreed.

CUDA may update physical memory mappings at any time during graph instantiation, launch, or execution. CUDA may also introduce synchronization between future graph launches in order to prevent live graph allocations from referring to the same physical memory. As for any allocate-free-allocate pattern, if a program accesses a pointer outside of an allocation's lifetime, the erroneous access may silently read or write live data owned by another allocation (even if the virtual address of the allocation is unique). Use of compute sanitizer tools can catch this error.

The following figure shows graphs sequentially launched in the same stream. In this example, each graph frees all the memory it allocates. Since the graphs in the same stream never run concurrently, CUDA can and should use the same physical memory to satisfy all the allocations.

4.2.5.4 Performance Considerations

When multiple graphs are launched into the same stream, CUDA attempts to allocate the same physical memory to them because the execution of these graphs cannot overlap. Physical mappings for a graph are retained between launches as an optimization to avoid the cost of remapping. If, at a later time, one of the graphs is launched such that its execution may overlap with the others (for example if it is launched into a different stream) then CUDA must perform some remapping because concurrent graphs require distinct memory to avoid data corruption.

In general, remapping of graph memory in CUDA is likely caused by these operations:

- ▶ Changing the stream into which a graph is launched
- ▶ A trim operation on the graph memory pool, which explicitly frees unused memory (discussed in *Physical Memory Footprint*)
- ▶ Relaunching a graph while an unfreed allocation from another graph is mapped to the same memory will cause a remap of memory before relaunch

Remapping must happen in execution order, but after any previous execution of that graph is complete (otherwise memory that is still in use could be unmapped). Due to this ordering dependency, as well as because mapping operations are OS calls, mapping operations can be relatively expensive. Applications can avoid this cost by launching graphs containing allocation memory nodes consistently into the same stream.

4.2.5.4.1 First Launch / `cudaGraphUpload`

Physical memory cannot be allocated or mapped during graph instantiation because the stream in which the graph will execute is unknown. Mapping is done instead during graph launch. Calling `cudaGraphUpload` can separate out the cost of allocation from the launch by performing all mappings for that graph immediately and associating the graph with the upload stream. If the graph is then launched into the same stream, it will launch without any additional remapping.

Using different streams for graph upload and graph launch behaves similarly to switching streams, likely resulting in remap operations. In addition, unrelated memory pool management is permitted to pull memory from an idle stream, which could negate the impact of the uploads.

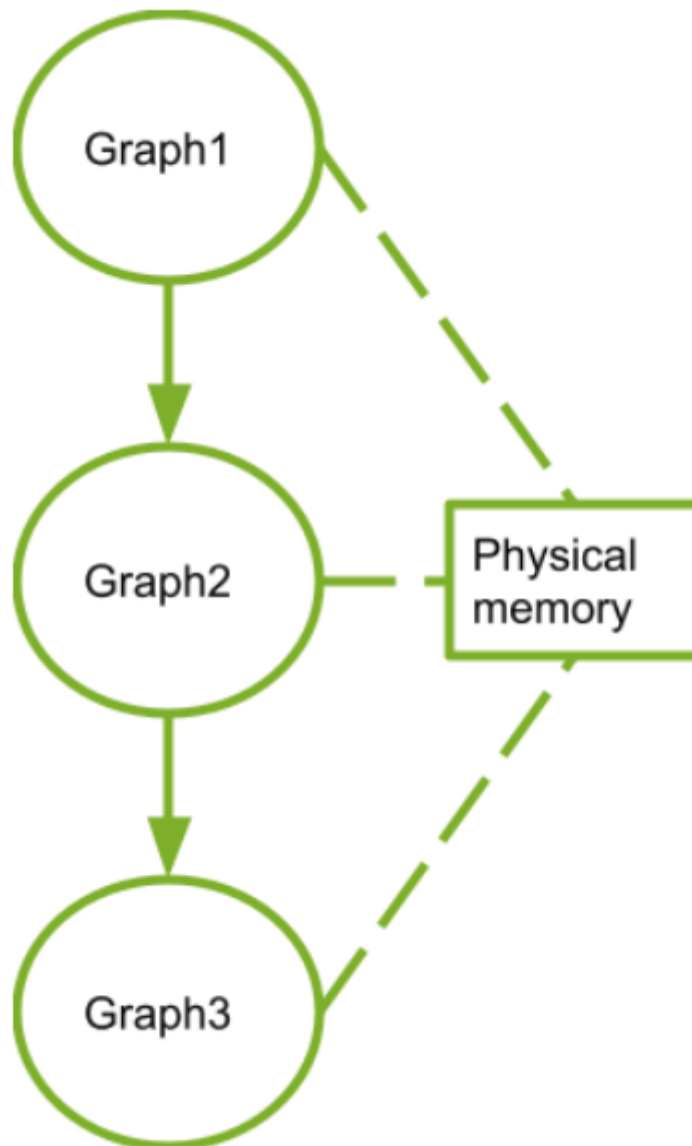


Figure 33: Sequentially Launched Graphs

4.2.5.5 Physical Memory Footprint

The pool-management behavior of asynchronous allocation means that destroying a graph which contains memory nodes (even if their allocations are free) will not immediately return physical memory to the OS for use by other processes. To explicitly release memory back to the OS, an application should use the `cudaDeviceGraphMemTrim` API.

`cudaDeviceGraphMemTrim` will unmap and release any physical memory reserved by graph memory nodes that is not actively in use. Allocations that have not been freed and graphs that are scheduled or running are considered to be actively using the physical memory and will not be impacted. Use of the trim API will make physical memory available to other allocation APIs and other applications or processes, but will cause CUDA to reallocate and remap memory when the trimmed graphs are next launched. Note that `cudaDeviceGraphMemTrim` operates on a different pool from `cudaMemPoolTrimTo()`. The graph memory pool is not exposed to the stream ordered memory allocator. CUDA allows applications to query their graph memory footprint through the `cudaDeviceGetGraphMemAttribute` API. Querying the attribute `cudaGraphMemAttrReservedMemCurrent` returns the amount of physical memory reserved by the driver for graph allocations in the current process. Querying `cudaGraphMemAttrUsedMemCurrent` returns the amount of physical memory currently mapped by at least one graph. Either of these attributes can be used to track when new physical memory is acquired by CUDA for the sake of an allocating graph. Both of these attributes are useful for examining how much memory is saved by the sharing mechanism.

4.2.5.6 Peer Access

Graph allocations can be configured for access from multiple GPUs, in which case CUDA will map the allocations onto the peer GPUs as required. CUDA allows graph allocations requiring different mappings to reuse the same virtual address. When this occurs, the address range is mapped onto all GPUs required by the different allocations. This means an allocation may sometimes allow more peer access than was requested during its creation; however, relying on these extra mappings is still an error.

4.2.5.6.1 Peer Access with Graph Node APIs

The `cudaGraphAddNode` API accepts mapping requests in the `accessDescs` array field of the alloc node parameters structures. The `poolProps.location` embedded structure specifies the resident device for the allocation. Access from the allocating GPU is assumed to be needed, thus the application does not need to specify an entry for the resident device in the `accessDescs` array.

```
cudaGraphNodeParams allocNodeParams = { cudaGraphNodeTypeMemAlloc };
allocNodeParams.alloc.poolProps.allocType = cudaMemAllocationTypePinned;
allocNodeParams.alloc.poolProps.location.type = cudaMemLocationTypeDevice;
// specify device 1 as the resident device
allocNodeParams.alloc.poolProps.location.id = 1;
allocNodeParams.alloc.bytesize = size;

// allocate an allocation resident on device 1 accessible from device 1
cudaGraphAddNode(&allocNode, graph, NULL, NULL, 0, &allocNodeParams);

accessDescs[2];
// boilerplate for the access descs (only ReadWrite and Device access supported
// by the add node api)
accessDescs[0].flags = cudaMemAccessFlagsProtReadWrite;
accessDescs[0].location.type = cudaMemLocationTypeDevice;
accessDescs[1].flags = cudaMemAccessFlagsProtReadWrite;
accessDescs[1].location.type = cudaMemLocationTypeDevice;
```

(continues on next page)

(continued from previous page)

```
// access being requested for device 0 & 2. Device 1 access requirement left
→implicit.
accessDescs[0].location.id = 0;
accessDescs[1].location.id = 2;

// access request array has 2 entries.
allocNodeParams.accessDescCount = 2;
allocNodeParams.accessDescs = accessDescs;

// allocate an allocation resident on device 1 accessible from devices 0, 1 and
→2. (0 & 2 from the descriptors, 1 from it being the resident device).
cudaGraphAddNode(&allocNode, graph, NULL, NULL, 0, &allocNodeParams);
```

4.2.5.6.2 Peer Access with Stream Capture

For stream capture, the allocation node records the peer accessibility of the allocating pool at the time of the capture. Altering the peer accessibility of the allocating pool after a `cudaMallocFromPoolAsync` call is captured does not affect the mappings that the graph will make for the allocation.

```
// boilerplate for the access descs (only ReadWrite and Device access supported
→by the add node api)
accessDesc.flags = cudaMemAccessFlagsProtReadWrite;
accessDesc.location.type = cudaMemLocationTypeDevice;
accessDesc.location.id = 1;

// let memPool be resident and accessible on device 0

cudaStreamBeginCapture(stream);
cudaMallocAsync(&dptr1, size, memPool, stream);
cudaStreamEndCapture(stream, &graph1);

cudaMemPoolSetAccess(memPool, &accessDesc, 1);

cudaStreamBeginCapture(stream);
cudaMallocAsync(&dptr2, size, memPool, stream);
cudaStreamEndCapture(stream, &graph2);

//The graph node allocating dp1r1 would only have the device 0 accessibility
→even though memPool now has device 1 accessibility.
//The graph node allocating dp1r2 will have device 0 and device 1 accessibility,
→ since that was the pool accessibility at the time of the cudaMallocAsync
→call.
```

4.2.6. Device Graph Launch

There are many workflows which need to make data-dependent decisions during runtime and execute different operations depending on those decisions. Rather than offloading this decision-making process to the host, which may require a round-trip from the device, users may prefer to perform it on the device. To that end, CUDA provides a mechanism to launch graphs from the device.

Device graph launch provides a convenient way to perform dynamic control flow from the device, be it something as simple as a loop or as complex as a device-side work scheduler.

Graphs which can be launched from the device will henceforth be referred to as device graphs, and graphs which cannot be launched from the device will be referred to as host graphs.

Device graphs can be launched from both the host and device, whereas host graphs can only be launched from the host. Unlike host launches, launching a device graph from the device while a previous launch of the graph is running will result in an error, returning `cudaErrorInvalidValue`; therefore, a device graph cannot be launched twice from the device at the same time. Launching a device graph from the host and device simultaneously will result in undefined behavior.

4.2.6.1 Device Graph Creation

In order for a graph to be launched from the device, it must be instantiated explicitly for device launch. This is achieved by passing the `cudaGraphInstantiateFlagDeviceLaunch` flag to the `cudaGraphInstantiate()` call. As is the case for host graphs, device graph structure is fixed at time of instantiation and cannot be updated without re-instantiation, and instantiation can only be performed on the host. In order for a graph to be able to be instantiated for device launch, it must adhere to various requirements.

4.2.6.1.1 Device Graph Requirements

General requirements:

- ▶ The graph's nodes must all reside on a single device.
- ▶ The graph can only contain kernel nodes, memcpy nodes, memset nodes, and child graph nodes.

Kernel nodes:

- ▶ Use of CUDA Dynamic Parallelism by kernels in the graph is not permitted.
- ▶ Cooperative launches are permitted so long as MPS is not in use.

Memcpy nodes:

- ▶ Only copies involving device memory and/or pinned device-mapped host memory are permitted.
- ▶ Copies involving CUDA arrays are not permitted.
- ▶ Both operands must be accessible from the current device at time of instantiation. Note that the copy operation will be performed from the device on which the graph resides, even if it is targeting memory on another device.

4.2.6.1.2 Device Graph Upload

In order to launch a graph on the device, it must first be uploaded to the device to populate the necessary device resources. This can be achieved in one of two ways.

Firstly, the graph can be uploaded explicitly, either via `cudaGraphUpload()` or by requesting an upload as part of instantiation via `cudaGraphInstantiateWithParams()`.

Alternatively, the graph can first be launched from the host, which will perform this upload step implicitly as part of the launch.

Examples of all three methods can be seen below:

```
// Explicit upload after instantiation
cudaGraphInstantiate(&deviceGraphExec1, deviceGraph1,
    ↪ cudaGraphInstantiateFlagDeviceLaunch);
cudaGraphUpload(deviceGraphExec1, stream);

// Explicit upload as part of instantiation
cudaGraphInstantiateParams instantiateParams = {0};
instantiateParams.flags = cudaGraphInstantiateFlagDeviceLaunch |
    ↪ cudaGraphInstantiateFlagUpload;
instantiateParams.uploadStream = stream;
cudaGraphInstantiateWithParams(&deviceGraphExec2, deviceGraph2, &
    ↪ instantiateParams);

// Implicit upload via host launch
cudaGraphInstantiate(&deviceGraphExec3, deviceGraph3,
    ↪ cudaGraphInstantiateFlagDeviceLaunch);
cudaGraphLaunch(deviceGraphExec3, stream);
```

4.2.6.1.3 Device Graph Update

Device graphs can only be updated from the host, and must be re-uploaded to the device upon executable graph update in order for the changes to take effect. This can be achieved using the same methods outlined in Section [Device Graph Upload](#). Unlike host graphs, launching a device graph from the device while an update is being applied will result in undefined behavior.

4.2.6.2 Device Launch

Device graphs can be launched from both the host and the device via `cudaGraphLaunch()`, which has the same signature on the device as on the host. Device graphs are launched via the same handle on the host and the device. Device graphs must be launched from another graph when launched from the device.

Device-side graph launch is per-thread and multiple launches may occur from different threads at the same time, so the user will need to select a single thread from which to launch a given graph.

Unlike host launch, device graphs cannot be launched into regular CUDA streams, and can only be launched into distinct named streams, which each denote a specific launch mode. The following table lists the available launch modes.

Table 9: Device-only Graph Launch Streams

Stream	Launch Mode
<code>cudaStreamGraphFireAndForget</code>	Fire and forget launch
<code>cudaStreamGraphTailLaunch</code>	Tail launch
<code>cudaStreamGraphFireAndForgetAsSibling</code>	Sibling launch

4.2.6.2.1 Fire and Forget Launch

As the name suggests, a fire and forget launch is submitted to the GPU immediately, and it runs independently of the launching graph. In a fire-and-forget scenario, the launching graph is the parent, and the launched graph is the child.

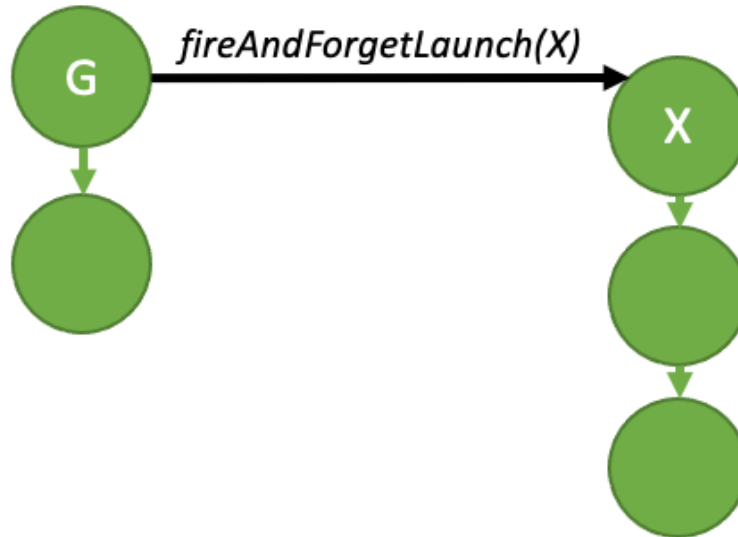


Figure 34: Fire and forget launch

The above diagram can be generated by the sample code below:

```
--global__ void launchFireAndForgetGraph(cudaGraphExec_t graph) {
    cudaGraphLaunch(graph, cudaStreamGraphFireAndForget);
}

void graphSetup() {
    cudaGraphExec_t gExec1, gExec2;
    cudaGraph_t g1, g2;

    // Create, instantiate, and upload the device graph.
    create_graph(&g2);
    cudaGraphInstantiate(&gExec2, g2, cudaGraphInstantiateFlagDeviceLaunch);
    cudaGraphUpload(gExec2, stream);

    // Create and instantiate the launching graph.
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    launchFireAndForgetGraph<<<1, 1, 0, stream>>>(gExec2);
    cudaStreamEndCapture(stream, &g1);
    cudaGraphInstantiate(&gExec1, g1);

    // Launch the host graph, which will in turn launch the device graph.
    cudaGraphLaunch(gExec1, stream);
}
```

A graph can have up to 120 total fire-and-forget graphs during the course of its execution. This total resets between launches of the same parent graph.

4.2.6.2.1.1 Graph Execution Environments

In order to fully understand the device-side synchronization model, it is first necessary to understand the concept of an execution environment.

When a graph is launched from the device, it is launched into its own execution environment. The execution environment of a given graph encapsulates all work in the graph as well as all generated fire and forget work. The graph can be considered complete when it has completed execution and when all generated child work is complete.

The below diagram shows the environment encapsulation that would be generated by the fire-and-forget sample code in the previous section.

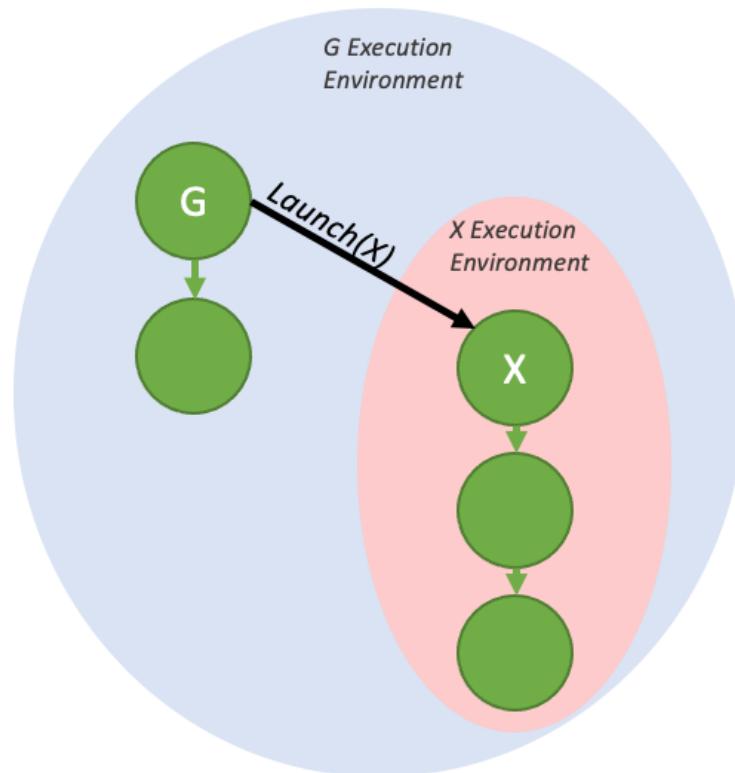


Figure 35: Fire and forget launch, with execution environments

These environments are also hierarchical, so a graph environment can include multiple levels of child-environments from fire and forget launches.

When a graph is launched from the host, there exists a stream environment that parents the execution environment of the launched graph. The stream environment encapsulates all work generated as part of the overall launch. The stream launch is complete (i.e. downstream dependent work may now run) when the overall stream environment is marked as complete.

4.2.6.2.2 Tail Launch

Unlike on the host, it is not possible to synchronize with device graphs from the GPU via traditional methods such as `cudaDeviceSynchronize()` or `cudaStreamSynchronize()`. Rather, in order to enable serial work dependencies, a different launch mode - tail launch - is offered, to provide similar functionality.

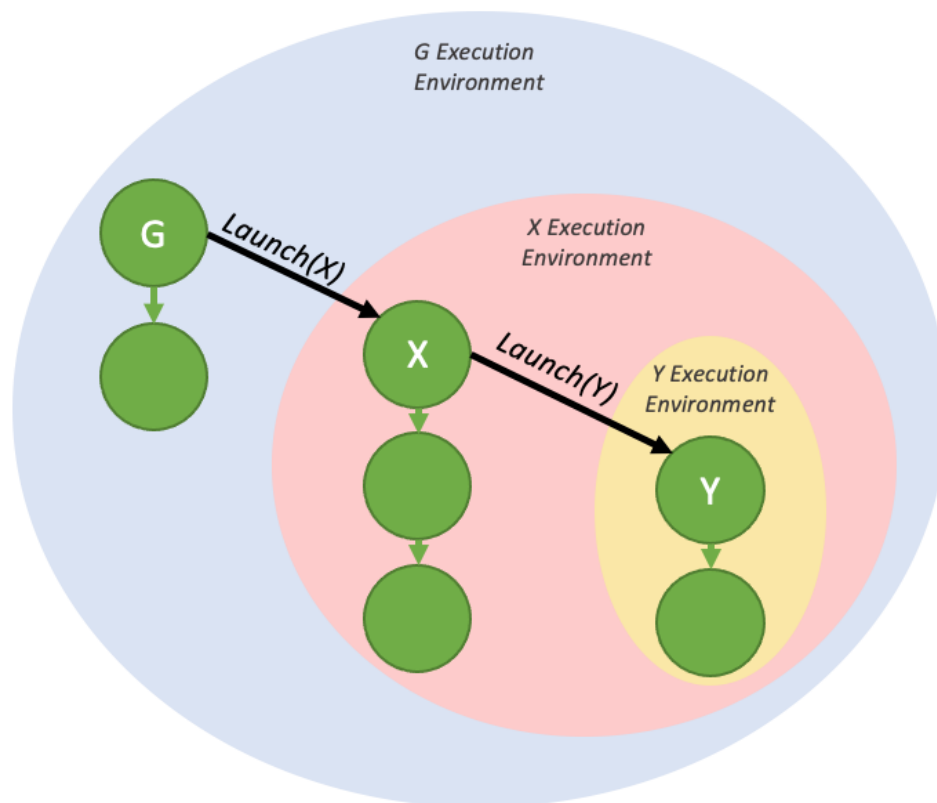


Figure 36: Nested fire and forget environments

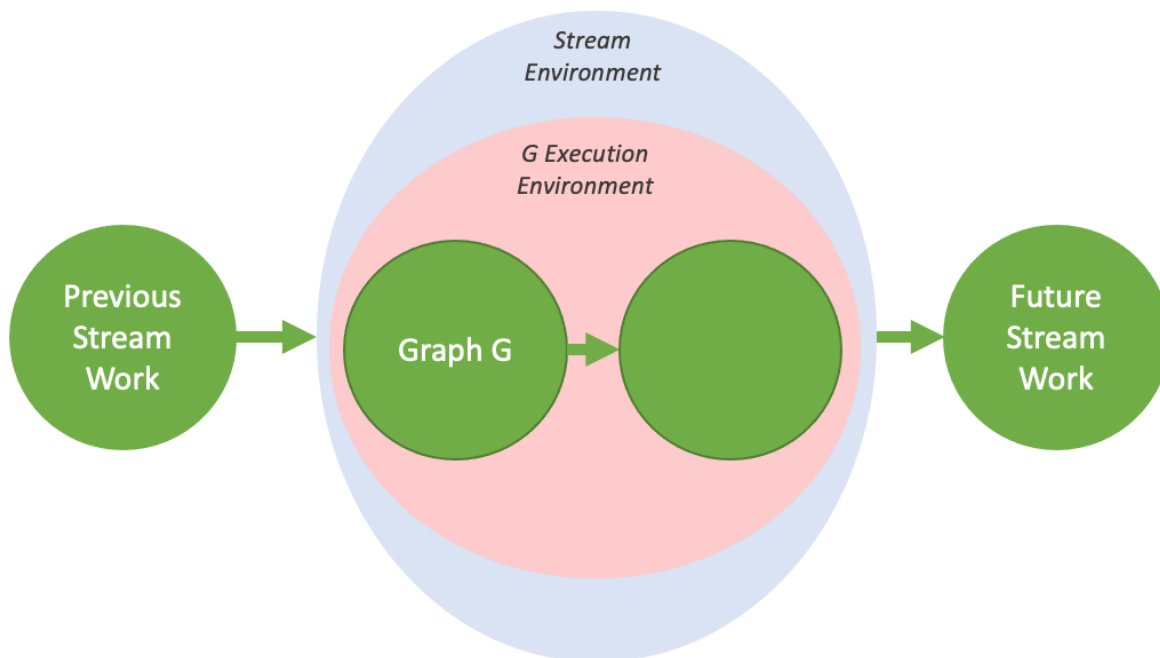


Figure 37: The stream environment, visualized

A tail launch executes when a graph's environment is considered complete - ie, when the graph and all its children are complete. When a graph completes, the environment of the next graph in the tail launch list will replace the completed environment as a child of the parent environment. Like fire-and-forget launches, a graph can have multiple graphs enqueued for tail launch.

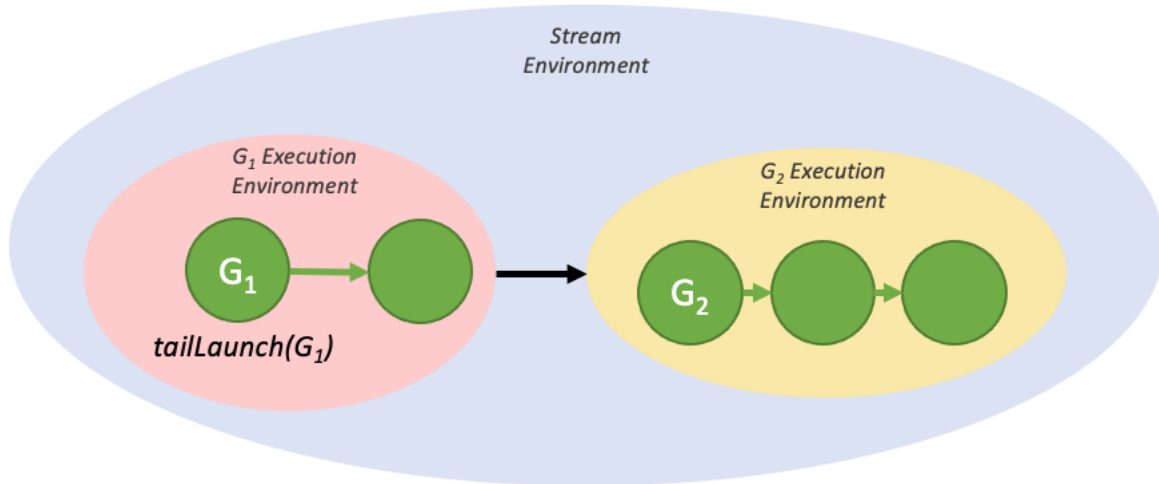


Figure 38: A simple tail launch

The above execution flow can be generated by the code below:

```
__global__ void launchTailGraph(cudaGraphExec_t graph) {
    cudaGraphLaunch(graph, cudaStreamGraphTailLaunch);
}

void graphSetup() {
    cudaGraphExec_t gExec1, gExec2;
    cudaGraph_t g1, g2;

    // Create, instantiate, and upload the device graph.
    create_graph(&g2);
    cudaGraphInstantiate(&gExec2, g2, cudaGraphInstantiateFlagDeviceLaunch);
    cudaGraphUpload(gExec2, stream);

    // Create and instantiate the launching graph.
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    launchTailGraph<<<1, 1, 0, stream>>>(gExec2);
    cudaStreamEndCapture(stream, &g1);
    cudaGraphInstantiate(&gExec1, g1);

    // Launch the host graph, which will in turn launch the device graph.
    cudaGraphLaunch(gExec1, stream);
}
```

Tail launches enqueued by a given graph will execute one at a time, in order of when they were enqueued. So the first enqueued graph will run first, and then the second, and so on.

Tail launches enqueued by a tail graph will execute before tail launches enqueued by previous graphs in the tail launch list. These new tail launches will execute in the order they are enqueued.

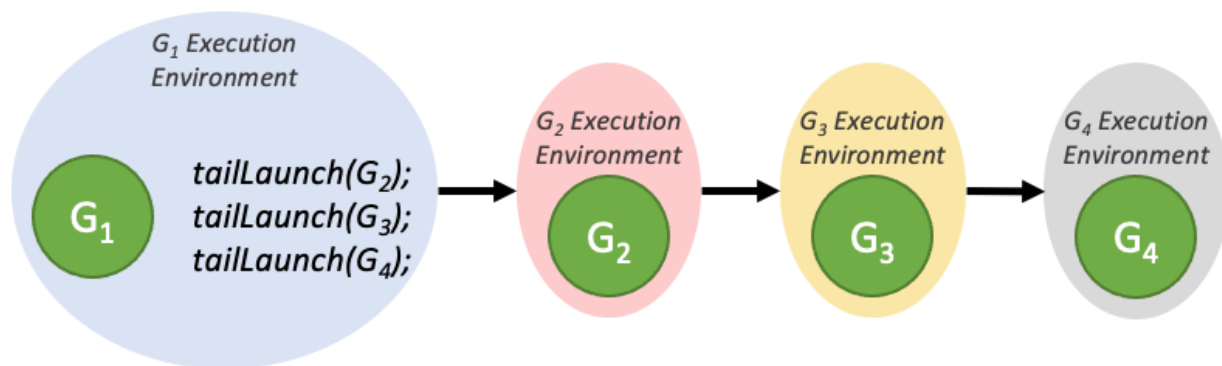


Figure 39: Tail launch ordering

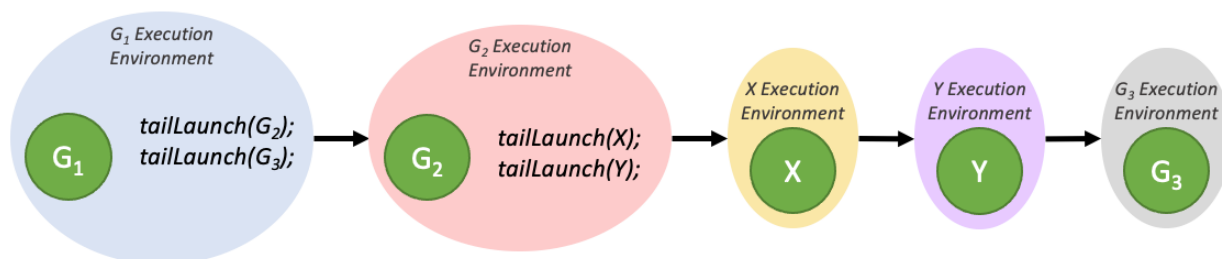


Figure 40: Tail launch ordering when enqueued from multiple graphs

A graph can have up to 255 pending tail launches.

4.2.6.2.2.1 Tail Self-launch

It is possible for a device graph to enqueue itself for a tail launch, although a given graph can only have one self-launch enqueued at a time. In order to query the currently running device graph so that it can be relaunched, a new device-side function is added:

```
cudaGraphExec_t cudaGetCurrentGraphExec();
```

This function returns the handle of the currently running graph if it is a device graph. If the currently executing kernel is not a node within a device graph, this function will return NULL.

Below is sample code showing usage of this function for a relaunch loop:

```
__device__ int relaunchCount = 0;

__global__ void relaunchSelf() {
    int relaunchMax = 100;

    if (threadIdx.x == 0) {
        if (relaunchCount < relaunchMax) {
            cudaGraphLaunch(cudaGetCurrentGraphExec(),
                ↪ cudaStreamGraphTailLaunch);
        }
    }
}
```

(continues on next page)

(continued from previous page)

```

    relaunchCount++;
}
}

```

4.2.6.2.3 Sibling Launch

Sibling launch is a variation of fire-and-forget launch in which the graph is launched not as a child of the launching graph's execution environment, but rather as a child of the launching graph's parent environment. Sibling launch is equivalent to a fire-and-forget launch from the launching graph's parent environment.

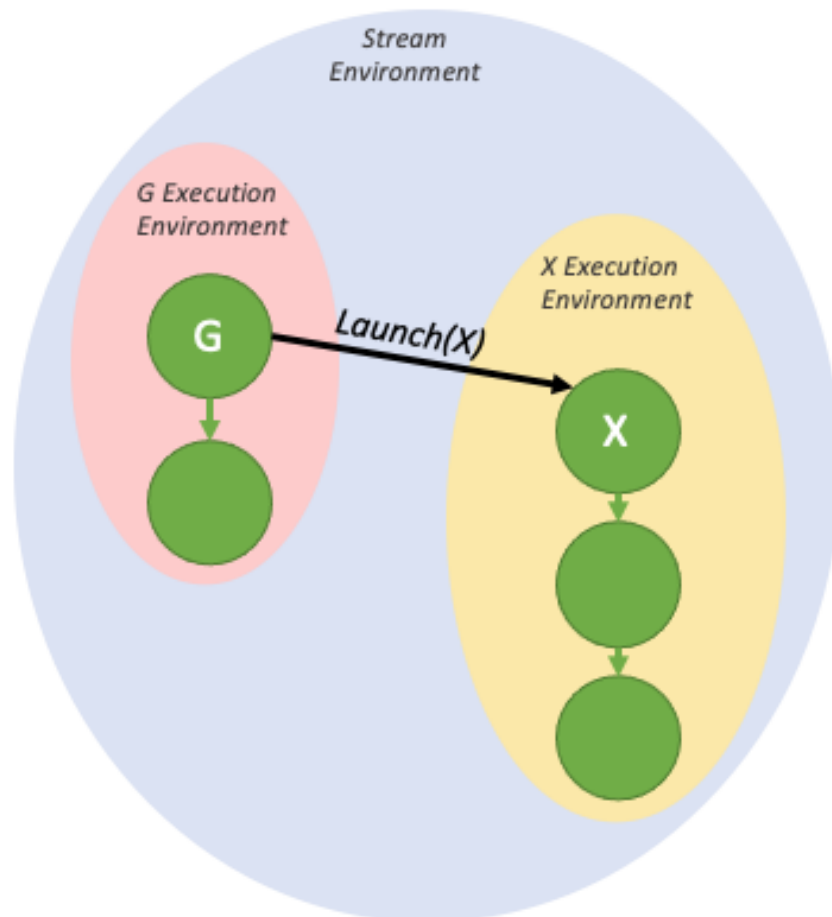


Figure 41: A simple sibling launch

The above diagram can be generated by the sample code below:

```

__global__ void launchSiblingGraph(cudaGraphExec_t graph) {
    cudaGraphLaunch(graph, cudaStreamGraphFireAndForgetAsSibling);
}

void graphSetup() {

```

(continues on next page)

(continued from previous page)

```

cudaGraphExec_t gExec1, gExec2;
cudaGraph_t g1, g2;

// Create, instantiate, and upload the device graph.
create_graph(&g2);
cudaGraphInstantiate(&gExec2, g2, cudaGraphInstantiateFlagDeviceLaunch);
cudaGraphUpload(gExec2, stream);

// Create and instantiate the launching graph.
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
launchSiblingGraph<<<1, 1, 0, stream>>>(gExec2);
cudaStreamEndCapture(stream, &g1);
cudaGraphInstantiate(&gExec1, g1);

// Launch the host graph, which will in turn launch the device graph.
cudaGraphLaunch(gExec1, stream);
}

```

Since sibling launches are not launched into the launching graph's execution environment, they will not gate tail launches enqueued by the launching graph.

4.2.7. Using Graph APIs

`cudaGraph_t` objects are not thread-safe. It is the responsibility of the user to ensure that multiple threads do not concurrently access the same `cudaGraph_t`.

A `cudaGraphExec_t` cannot run concurrently with itself. A launch of a `cudaGraphExec_t` will be ordered after previous launches of the same executable graph.

Graph execution is done in streams for ordering with other asynchronous work. However, the stream is for ordering only; it does not constrain the internal parallelism of the graph, nor does it affect where graph nodes execute.

See [Graph API](#).

4.2.8. CUDA User Objects

CUDA User Objects can be used to help manage the lifetime of resources used by asynchronous work in CUDA. In particular, this feature is useful for [CUDA Graphs](#) and [stream capture](#).

Various resource management schemes are not compatible with CUDA graphs. Consider for example an event-based pool or a synchronous-create, asynchronous-destroy scheme.

```

// Library API with pool allocation
void libraryWork(cudaStream_t stream) {
    auto &resource = pool.claimTemporaryResource();
    resource.waitOnReadyEventInStream(stream);
    launchWork(stream, resource);
    resource.recordReadyEvent(stream);
}

```

```
// Library API with asynchronous resource deletion
void libraryWork(cudaStream_t stream) {
    Resource *resource = new Resource(...);
    launchWork(stream, resource);
    cudaLaunchHostFunc(
        stream,
        [](void *resource) {
            delete static_cast<Resource *>(resource);
        },
        resource,
        0);
    // Error handling considerations not shown
}
```

These schemes are difficult with CUDA graphs because of the non-fixed pointer or handle for the resource which requires indirection or graph update, and the synchronous CPU code needed each time the work is submitted. They also do not work with stream capture if these considerations are hidden from the caller of the library, and because of use of disallowed APIs during capture. Various solutions exist such as exposing the resource to the caller. CUDA user objects present another approach.

A CUDA user object associates a user-specified destructor callback with an internal refcount, similar to C++ `shared_ptr`. References may be owned by user code on the CPU and by CUDA graphs. Note that for user-owned references, unlike C++ smart pointers, there is no object representing the reference; users must track user-owned references manually. A typical use case would be to immediately move the sole user-owned reference to a CUDA graph after the user object is created.

When a reference is associated to a CUDA graph, CUDA will manage the graph operations automatically. A cloned `cudaGraph_t` retains a copy of every reference owned by the source `cudaGraph_t`, with the same multiplicity. An instantiated `cudaGraphExec_t` retains a copy of every reference in the source `cudaGraph_t`. When a `cudaGraphExec_t` is destroyed without being synchronized, the references are retained until the execution is completed.

Here is an example use.

```
cudaGraph_t graph; // Preexisting graph

Object *object = new Object; // C++ object with possibly nontrivial destructor
cudaUserObject_t cuObject;
cudaUserObjectCreate(
    &cuObject,
    object, // Here we use a CUDA-provided template wrapper for this API,
            // which supplies a callback to delete the C++ object pointer
    1, // Initial refcount
    cudaUserObjectNoDestructorSync // Acknowledge that the callback cannot be
                                    // waited on via CUDA
);
cudaGraphRetainUserObject(
    graph,
    cuObject,
    1, // Number of references
    cudaGraphUserObjectMove // Transfer a reference owned by the caller (do
                            // not modify the total reference count)
);
// No more references owned by this thread; no need to call release API
```

(continues on next page)

(continued from previous page)

```

cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, nullptr, nullptr, 0); // Will retain
↳a                                                                    // new
↳reference
cudaGraphDestroy(graph); // graphExec still owns a reference
cudaGraphLaunch(graphExec, 0); // Async launch has access to the user objects
cudaGraphExecDestroy(graphExec); // Launch is not synchronized; the release
                                // will be deferred if needed
cudaStreamSynchronize(0); // After the launch is synchronized, the remaining
                           // reference is released and the destructor will
                           // execute. Note this happens asynchronously.
// If the destructor callback had signaled a synchronization object, it would
// be safe to wait on it at this point.

```

References owned by graphs in child graph nodes are associated to the child graphs, not the parents. If a child graph is updated or deleted, the references change accordingly. If an executable graph or child graph is updated with `cudaGraphExecUpdate` or `cudaGraphExecChildGraphNodeSetParams`, the references in the new source graph are cloned and replace the references in the target graph. In either case, if previous launches are not synchronized, any references which would be released are held until the launches have finished executing.

There is not currently a mechanism to wait on user object destructors via a CUDA API. Users may signal a synchronization object manually from the destructor code. In addition, it is not legal to call CUDA APIs from the destructor, similar to the restriction on `cudaLaunchHostFunc`. This is to avoid blocking a CUDA internal shared thread and preventing forward progress. It is legal to signal another thread to perform an API call, if the dependency is one way and the thread doing the call cannot block forward progress of CUDA work.

User objects are created with `cudaUserObjectCreate`, which is a good starting point to browse related APIs.

4.3. Stream-Ordered Memory Allocator

4.3.1. Introduction

Managing memory allocations using `cudaMalloc` and `cudaFree` causes the GPU to synchronize across all executing CUDA streams. The stream-ordered memory allocator enables applications to order memory allocation and deallocation with other work launched into a CUDA stream such as kernel launches and asynchronous copies. This improves application memory use by taking advantage of stream-ordering semantics to reuse memory allocations. The allocator also allows applications to control the allocator's memory caching behavior. When set up with an appropriate release threshold, the caching behavior allows the allocator to avoid expensive calls into the OS when the application indicates it is willing to accept a bigger memory footprint. The allocator also supports easy and secure allocation sharing between processes.

Stream-Ordered Memory Allocator:

- ▶ Reduces the need for custom memory management abstractions, and makes it easier to create high-performance custom memory management for applications that need it.
- ▶ Enables multiple libraries to share a common memory pool managed by the driver. This can reduce excess memory consumption.

- Allows, the driver to perform optimizations based on its awareness of the allocator and other stream management APIs.

Note

Nsight Compute and the Next-Gen CUDA debugger is aware of the allocator since CUDA 11.3.

4.3.2. Memory Management

`cudaMallocAsync` and `cudaFreeAsync` are the APIs which enable stream-ordered memory management. `cudaMallocAsync` returns an allocation and `cudaFreeAsync` frees an allocation. Both APIs accept stream arguments to define when the allocation will become and stop being available for use. These functions allow memory operations to be tied to specific CUDA streams, enabling them to occur without blocking the host or other streams. Application performance can be improved by avoiding potentially costly synchronization of `cudaMalloc` and `cudaFree`.

These APIs can be used for further performance optimization through memory pools, which manage and reuse large blocks of memory for more efficient allocation and deallocation. Memory pools help reduce overhead and prevent fragmentation, improving performance in scenarios with frequent memory allocation operations.

4.3.2.1 Allocating Memory

The `cudaMallocAsync` function triggers asynchronous memory allocation on the GPU, linked to a specific CUDA stream. `cudaMallocAsync` allows memory allocation to occur without hindering the host or other streams, eliminating the need for expensive synchronization.

Note

`cudaMallocAsync` ignores the current device/context when determining where the allocation will reside. Instead, `cudaMallocAsync` determines the appropriate device based on the specified memory pool or the supplied stream.

The listing below illustrates a fundamental use pattern: the memory is allocated, used, and then freed back into the same stream.

```
void *ptr;
size_t size = 512;
cudaMallocAsync(&ptr, size, cudaStreamPerThread);
// do work using the allocation
kernel<<<..., cudaStreamPerThread>>>(ptr, ...);
// An asynchronous free can be specified without synchronizing the cpu and GPU
cudaFreeAsync(ptr, cudaStreamPerThread);
```

Note

When accessing allocation from a stream other than the stream that made the allocation, the user must guarantee that the access occurs after the allocation operation, otherwise the behavior is undefined.

4.3.2.2 Freeing Memory

`cudaFreeAsync()` asynchronously frees device memory in a stream-ordered fashion, meaning the memory deallocation is assigned to a specific CUDA stream and does not block the host or other streams.

The user must guarantee that the free operation happens after the allocation operation and any uses of the allocation. Any use of the allocation after the free operation starts results in undefined behavior.

Events and/or stream synchronizing operations should be used to guarantee any access to the allocation from other streams is complete before the free operation begins, as illustrated in the following example.

```
cudaMallocAsync(&ptr, size, stream1);
cudaEventRecord(event1, stream1);
//stream2 must wait for the allocation to be ready before accessing
cudaStreamWaitEvent(stream2, event1);
kernel<<<..., stream2>>>(ptr, ...);
cudaEventRecord(event2, stream2);
// stream3 must wait for stream2 to finish accessing the allocation before
// freeing the allocation
cudaStreamWaitEvent(stream3, event2);
cudaFreeAsync(ptr, stream3);
```

Memory allocated with `cudaMalloc()` can be freed with `cudaFreeAsync()`. As above, all accesses to the memory must be complete before the free operation begins.

```
cudaMalloc(&ptr, size);
kernel<<<..., stream>>>(ptr, ...);
cudaFreeAsync(ptr, stream);
```

Likewise, memory allocated with `cudaMallocAsync` can be freed with `cudaFree()`. When freeing such allocations through the `cudaFree()` API, the driver assumes that all accesses to the allocation are complete and performs no further synchronization. The user can use `cudaStreamQuery` / `cudaStreamSynchronize` / `cudaEventQuery` / `cudaEventSynchronize` / `cudaDeviceSynchronize` to guarantee that the appropriate asynchronous work is complete and that the GPU will not try to access the allocation.

```
cudaMallocAsync(&ptr, size, stream);
kernel<<<..., stream>>>(ptr, ...);
// synchronize is needed to avoid prematurely freeing the memory
cudaStreamSynchronize(stream);
cudaFree(ptr);
```

4.3.3. Memory Pools

Memory pools encapsulate virtual address and physical memory resources that are allocated and managed according to the pools attributes and properties. The primary aspect of a memory pool is the kind and location of memory it manages.

All calls to `cudaMallocAsync` use resources from memory pool. If a memory pool is not specified, `cudaMallocAsync` uses the current memory pool of the supplied stream's device. The current memory pool for a device may be set with `cudaDeviceSetMempool` and queried with `cudaDeviceGetMempool`. Each device has a default memory pool, which is active if `cudaDeviceSetMempool` has not been called.

The API `cudaMallocFromPoolAsync` and [C++ overloads of `cudaMallocAsync`](#) allow a user to specify the pool to be used for an allocation without setting it as the current pool. The APIs `cudaDeviceGetDefaultMempool` and `cudaMemPoolCreate` return handles to memory pools. `cudaMemPoolSetAttribute` and `cudaMemPoolGetAttribute` control the attributes of memory pools.

Note

The mempool current to a device will be local to that device. So allocating without specifying a memory pool will always yield an allocation local to the stream's device.

4.3.3.1 Default/Implicit Pools

The default memory pool of a device can be retrieved by calling `cudaDeviceGetDefaultMempool`. Allocations from the default memory pool of a device are non-migratable device allocation located on that device. These allocations will always be accessible from that device. The accessibility of the default memory pool can be modified with `cudaMemPoolSetAccess` and queried with `cudaMemPoolGetAccess`. Since the default pools do not need to be explicitly created, they are sometimes referred to as implicit pools. The default memory pool of a device does not support IPC.

4.3.3.2 Explicit Pools

`cudaMemPoolCreate` creates an explicit pool. This allows applications to request properties for their allocation beyond what is provided by the default/implicit pools. These include properties such as IPC capability, maximum pool size, allocations resident on a specific CPU NUMA node on supported platforms etc.

```
// create a pool similar to the implicit pool on device 0
int device = 0;
cudaMemPoolProps poolProps = { };
poolProps.allocType = cudaMemAllocationTypePinned;
poolProps.location.id = device;
poolProps.location.type = cudaMemLocationTypeDevice;

cudaMemPoolCreate(&memPool, &poolProps));
```

The following code snippet illustrates an example of creating an IPC capable memory pool on a valid CPU NUMA node.

```
// create a pool resident on a CPU NUMA node that is capable of IPC sharing
↳(via a file descriptor).
int cpu_numa_id = 0;
cudaMemPoolProps poolProps = { };
poolProps.allocType = cudaMemAllocationTypePinned;
poolProps.location.id = cpu_numa_id;
poolProps.location.type = cudaMemLocationTypeHostNuma;
poolProps.handleType = cudaMemHandleTypePosixFileDescriptor;

cudaMemPoolCreate(&ipcMemPool, &poolProps));
```

4.3.3.3 Device Accessibility for Multi-GPU Support

Like allocation accessibility controlled through the virtual memory management APIs, memory pool allocation accessibility does not follow `cudaDeviceEnablePeerAccess` or `cuCtxEnablePeerAccess`. For memory pools, the API `cudaMemPoolSetAccess` modifies what devices can access allocations from a pool. By default, allocations are accessible only from the device where the allocations are located. This access cannot be revoked. To enable access from other devices, the accessing device must be peer capable with the memory pool's device. This can be verified with `cudaDeviceCanAccessPeer`. If the peer capability is not checked, the set access may fail with `cudaErrorInvalidDevice`. However, if no allocations had been made from the pool, the `cudaMemPoolSetAccess` call may succeed even when the devices are not peer capable. In this case, the next allocation from the pool will fail.

It is worth noting that `cudaMemPoolSetAccess` affects all allocations from the memory pool, not just future ones. Likewise, the accessibility reported by `cudaMemPoolGetAccess` applies to all allocations from the pool, not just future ones. Changing the accessibility settings of a pool for a given GPU frequently is not recommended. That is, once a pool is made accessible from a given GPU, it should remain accessible from that GPU for the lifetime of the pool.

```
// snippet showing usage of cudaMemPoolSetAccess:
cudaError_t setAccessOnDevice(cudaMemPool_t memPool, int residentDevice,
                             int accessingDevice) {
    cudaMemAccessDesc accessDesc = {};
    accessDesc.location.type = cudaMemLocationTypeDevice;
    accessDesc.location.id = accessingDevice;
    accessDesc.flags = cudaMemAccessFlagsProtReadWrite;

    int canAccess = 0;
    cudaError_t error = cudaDeviceCanAccessPeer(&canAccess, accessingDevice,
                                                residentDevice);
    if (error != cudaSuccess) {
        return error;
    } else if (canAccess == 0) {
        return cudaErrorPeerAccessUnsupported;
    }

    // Make the address accessible
    return cudaMemPoolSetAccess(memPool, &accessDesc, 1);
}
```

4.3.3.4 Enabling Memory Pools for IPC

Memory pools can be enabled for interprocess communication (IPC) to allow easy, efficient and secure sharing of GPU memory between processes. CUDA's IPC memory pools provide the same security benefits as CUDA's [virtual memory management APIs](#).

There are two steps to sharing memory between processes with memory pools: the processes first needs to share access to the pool, then share specific allocations from that pool. The first step establishes and enforces security. The second step coordinates what virtual addresses are used in each process and when mappings need to be valid in the importing process.

4.3.3.4.1 Creating and Sharing IPC Memory Pools

Sharing access to a pool involves retrieving an OS-native handle to the pool with `cudaMemPoolExportToShareableHandle()`, transferring the handle to the importing process using OS-native IPC mechanisms, and then creating an imported memory pool with the `cudaMemPoolImportFromShareableHandle()` API. For `cudaMemPoolExportToShareableHandle()` to succeed, the memory pool must have been created with the requested handle type specified in the pool properties structure.

Please reference [samples](#) for the appropriate IPC mechanisms to transfer the OS-native handle between processes. The rest of the procedure can be found in the following code snippets.

```
// in exporting process
// create an exportable IPC capable pool on device 0
cudaMemPoolProps poolProps = { };
poolProps.allocType = cudaMemAllocationTypePinned;
poolProps.location.id = 0;
poolProps.location.type = cudaMemLocationTypeDevice;

// Setting handleTypes to a non zero value will make the pool exportable (IPC
// capable)
poolProps.handleTypes = CU_MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR;

cudaMemPoolCreate(&memPool, &poolProps));

// FD based handles are integer types
int fdHandle = 0;

// Retrieve an OS native handle to the pool.
// Note that a pointer to the handle memory is passed in here.
cudaMemPoolExportToShareableHandle(&fdHandle,
                                   memPool,
                                   CU_MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR,
                                   0);

// The handle must be sent to the importing process with the appropriate
// OS-specific APIs.
```

```
// in importing process
int fdHandle;
// The handle needs to be retrieved from the exporting process with the
// appropriate OS-specific APIs.
// Create an imported pool from the shareable handle.
// Note that the handle is passed by value here.
cudaMemPoolImportFromShareableHandle(&importedMemPool,
                                     (void*)fdHandle,
                                     CU_MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR,
                                     0);
```

4.3.3.4.2 Set Access in the Importing Process

Imported memory pools are initially only accessible from their resident device. The imported memory pool does not inherit any accessibility set by the exporting process. The importing process needs to enable access with `cudaMemPoolSetAccess` from any GPU it plans to access the memory from.

If the imported memory pool belongs to a device that is not visible to importing process, the user must use the `cudaMemPoolSetAccess` API to enable access from the GPUs the allocations will be used on. (See [Device Accessibility for Multi-GPU Support](#))

4.3.3.4.3 Creating and Sharing Allocations from an Exported Pool

Once the pool has been shared, allocations made with `cudaMallocAsync()` from the pool in the exporting process can be shared with processes that have imported the pool. Since the pool's security policy is established and verified at the pool level, the OS does not need extra bookkeeping to provide security for specific pool allocations. In other words, the opaque `cudaMemPoolPtrExportData` required to import a pool allocation may be sent to the importing process using any mechanism.

While allocations may be exported and imported without synchronizing with the allocating stream in any way, the importing process must follow the same rules as the exporting process when accessing the allocation. Specifically, access to the allocation must happen after the allocation operation in the allocating stream executes. The two following code snippets show `cudaMemPoolExportPointer()` and `cudaMemPoolImportPointer()` sharing the allocation with an IPC event used to guarantee that the allocation isn't accessed in the importing process before the allocation is ready.

```
// preparing an allocation in the exporting process
cudaMemPoolPtrExportData exportData;
cudaEvent_t readyIpcEvent;
cudaIpcEventHandle_t readyIpcEventHandle;

// ipc event for coordinating between processes
// cudaEventInterprocess flag makes the event an ipc event
// cudaEventDisableTiming is set for performance reasons

cudaEventCreate(&readyIpcEvent, cudaEventDisableTiming |
    ↪ cudaEventInterprocess)

// allocate from the exporting mem pool
cudaMallocAsync(&ptr, size, exportMemPool, stream);

// event for sharing when the allocation is ready.
cudaEventRecord(readyIpcEvent, stream);
cudaMemPoolExportPointer(&exportData, ptr);
cudaIpcGetEventHandle(&readyIpcEventHandle, readyIpcEvent);

// Share IPC event and pointer export data with the importing process using
// any mechanism. Here we copy the data into shared memory
shmem->ptrData = exportData;
shmem->readyIpcEventHandle = readyIpcEventHandle;
// signal consumers data is ready
```

```
// Importing an allocation
cudaMemPoolPtrExportData *importData = &shmem->ptrData;
cudaEvent_t readyIpcEvent;
```

(continues on next page)

(continued from previous page)

```

cudaIpcEventHandle_t *readyIpcEventHandle = &shmem->readyIpcEventHandle;

// Need to retrieve the ipc event handle and the export data from the
// exporting process using any mechanism. Here we are using shmem and just
// need synchronization to make sure the shared memory is filled in.

cudaIpcOpenEventHandle(&readyIpcEvent, readyIpcEventHandle);

// import the allocation. The operation does not block on the allocation being
// →ready.
cudaMemPoolImportPointer(&ptr, importedMemPool, importData);

// Wait for the prior stream operations in the allocating stream to complete
// →before
// using the allocation in the importing process.
cudaStreamWaitEvent(stream, readyIpcEvent);
kernel<<<..., stream>>>(ptr, ...);

```

When freeing the allocation, the allocation must be freed in the importing process before it is freed in the exporting process. The following code snippet demonstrates the use of CUDA IPC events to provide the required synchronization between the `cudaFreeAsync` operations in both processes. Access to the allocation from the importing process is obviously restricted by the free operation in the importing process side. It is worth noting that `cudaFree` can be used to free the allocation in both processes and that other stream synchronization APIs may be used instead of CUDA IPC events.

```

// The free must happen in importing process before the exporting process
kernel<<<..., stream>>>(ptr, ...);

```

```

// Last access in importing process
cudaFreeAsync(ptr, stream);

```

```

// Access not allowed in the importing process after the free
cudaIpcEventRecord(finishedIpcEvent, stream);

```

```

// Exporting process
// The exporting process needs to coordinate its free with the stream order
// of the importing process's free.
cudaStreamWaitEvent(stream, finishedIpcEvent);
kernel<<<..., stream>>>(ptrInExportingProcess, ...);

// The free in the importing process doesn't stop the exporting process
// from using the allocation.
cudaFreeAsync(ptrInExportingProcess, stream);

```

4.3.3.4.4 IPC Export Pool Limitations

IPC pools currently do not support releasing physical blocks back to the OS. As a result the `cudaMemPoolTrimTo` API has no effect and the `cudaMemPoolAttrReleaseThreshold` is effectively ignored. This behavior is controlled by the driver, not the runtime and may change in a future driver update.

4.3.3.4.5 IPC Import Pool Limitations

Allocating from an import pool is not allowed; specifically, import pools cannot be set current and cannot be used in the `cudaMallocFromPoolAsync` API. As such, the allocation reuse policy attributes do not have meaning for these pools.

IPC Import pools, like IPC export pools, currently do not support releasing physical blocks back to the OS.

The resource usage stat attribute queries only reflect the allocations imported into the process and the associated physical memory.

4.3.4. Best Practices and Tuning

4.3.4.1 Query for Support

An application can determine whether or not a device supports the stream-ordered memory allocator by calling `cudaDeviceGetAttribute()` (see [developer blog](#)) with the device attribute `cudaDevAttrMemoryPoolsSupported`.

IPC memory pool support can be queried with the `cudaDevAttrMemoryPoolSupportedHandleTypes` device attribute. This attribute was added in CUDA 11.3, and older drivers will return `cudaErrorInvalidValue` when this attribute is queried.

```
int driverVersion = 0;
int deviceSupportsMemoryPools = 0;
int poolSupportedHandleTypes = 0;
cudaDriverGetVersion(&driverVersion);
if (driverVersion >= 11020) {
    cudaDeviceGetAttribute(&deviceSupportsMemoryPools,
                          cudaDevAttrMemoryPoolsSupported, device);
}
if (deviceSupportsMemoryPools != 0) {
    // `device` supports the Stream-Ordered Memory Allocator
}

if (driverVersion >= 11030) {
    cudaDeviceGetAttribute(&poolSupportedHandleTypes,
                          cudaDevAttrMemoryPoolSupportedHandleTypes, device);
}
if (poolSupportedHandleTypes & cudaMemHandleTypePosixFileDescriptor) {
    // Pools on the specified device can be created with posix file descriptor-
    // based IPC
}
```

Performing the driver version check before the query avoids hitting a `cudaErrorInvalidValue` error on drivers where the attribute was not yet defined. One can use `cudaGetLastError` to clear the error instead of avoiding it.

4.3.4.2 Physical Page Caching Behavior

By default, the allocator tries to minimize the physical memory owned by a pool. To minimize the OS calls to allocate and free physical memory, applications must configure a memory footprint for each pool. Applications can do this with the release threshold attribute (`cudaMemPoolAttrReleaseThreshold`).

The release threshold is the amount of memory in bytes a pool should hold onto before trying to release memory back to the OS. When more than the release threshold bytes of memory are held by the memory pool, the allocator will try to release memory back to the OS on the next call to stream, event or device synchronize. Setting the release threshold to `UINT64_MAX` will prevent the driver from attempting to shrink the pool after every synchronization.

```
Cuuint64_t setVal = UINT64_MAX;
cudaMemPoolSetAttribute(memPool, cudaMemPoolAttrReleaseThreshold, &setVal);
```

Applications that set `cudaMemPoolAttrReleaseThreshold` high enough to effectively disable memory pool shrinking may wish to explicitly shrink a memory pool's memory footprint. `cudaMemPoolTrimTo` allows applications to do so. When trimming a memory pool's footprint, the `minBytesToKeep` parameter allows an application to hold onto a specified amount of memory, for example the amount it expects to need in a subsequent phase of execution.

```
Cuuint64_t setVal = UINT64_MAX;
cudaMemPoolSetAttribute(memPool, cudaMemPoolAttrReleaseThreshold, &setVal);

// application phase needing a lot of memory from the stream-ordered allocator
for (i=0; i<10; i++) {
    for (j=0; j<10; j++) {
        cudaMallocAsync(&ptrs[j], size[j], stream);
    }
    kernel<<<..., stream>>>(ptrs, ...);
    for (j=0; j<10; j++) {
        cudaFreeAsync(ptrs[j], stream);
    }
}

// Process does not need as much memory for the next phase.
// Synchronize so that the trim operation will know that the allocations are no
// longer in use.
cudaStreamSynchronize(stream);
cudaMemPoolTrimTo(mempool, 0);

// Some other process/allocation mechanism can now use the physical memory
// released by the trimming operation.
```

4.3.4.3 Resource Usage Statistics

Querying the `cudaMemPoolAttrReservedMemCurrent` attribute of a pool reports the current total physical GPU memory consumed by the pool. Querying the `cudaMemPoolAttrUsedMemCurrent` of a pool returns the total size of all of the memory allocated from the pool and not available for reuse.

The `cudaMemPoolAttr*MemHigh` attributes are watermarks recording the max value achieved by the respective `cudaMemPoolAttr*MemCurrent` attribute since last reset. They can be reset to the current value by using the `cudaMemPoolSetAttribute` API.

```
// sample helper functions for getting the usage statistics in bulk
struct usageStatistics {
    cuuint64_t reserved;
    cuuint64_t reservedHigh;
    cuuint64_t used;
    cuuint64_t usedHigh;
```

(continues on next page)

(continued from previous page)

```
};

void getUsageStatistics(cudaMemoryPool_t memPool, struct usageStatistics
    ↪ *statistics)
{
    cudaMemPoolGetAttribute(memPool, cudaMemPoolAttrReservedMemCurrent,
    ↪ statistics->reserved);
    cudaMemPoolGetAttribute(memPool, cudaMemPoolAttrReservedMemHigh,
    ↪ statistics->reservedHigh);
    cudaMemPoolGetAttribute(memPool, cudaMemPoolAttrUsedMemCurrent,
    ↪ statistics->used);
    cudaMemPoolGetAttribute(memPool, cudaMemPoolAttrUsedMemHigh, statistics->
    ↪ usedHigh);
}

// resetting the watermarks will make them take on the current value.
void resetStatistics(cudaMemoryPool_t memPool)
{
    cuuint64_t value = 0;
    cudaMemPoolSetAttribute(memPool, cudaMemPoolAttrReservedMemHigh, &value);
    cudaMemPoolSetAttribute(memPool, cudaMemPoolAttrUsedMemHigh, &value);
}
```

4.3.4.4 Memory Reuse Policies

In order to service an allocation request, the driver attempts to reuse memory that was previously freed via `cudaFreeAsync()` before attempting to allocate more memory from the OS. For example, memory freed in a stream can be reused immediately in a subsequent allocation request on the same stream. When a stream is synchronized with the CPU, the memory that was previously freed in that stream becomes available for reuse for an allocation in any stream. Reuse policies can be applied to both default and explicit memory pools.

The stream-ordered allocator has a few controllable allocation policies. The pool attributes `cudaMemPoolReuseFollowEventDependencies`, `cudaMemPoolReuseAllowOpportunistic`, and `cudaMemPoolReuseAllowInternalDependencies` control these policies and are detailed below. These policies can be enabled or disabled through a call to `cudaMemPoolSetAttribute`. Upgrading to a newer CUDA driver may change, enhance, augment and/or reorder the enumeration of the reuse policies.

4.3.4.4.1 `cudaMemPoolReuseFollowEventDependencies`

Before allocating more physical GPU memory, the allocator examines dependency information established by CUDA events and tries to allocate from memory freed in another stream.

```
cudaMallocAsync(&ptr, size, originalStream);
kernel<<<..., originalStream>>>(ptr, ...);
cudaFreeAsync(ptr, originalStream);
cudaEventRecord(event, originalStream);

// waiting on the event that captures the free in another stream
// allows the allocator to reuse the memory to satisfy
```

(continues on next page)

(continued from previous page)

```
// a new allocation request in the other stream when
// cudaMemPoolReuseFollowEventDependencies is enabled.
cudaStreamWaitEvent(otherStream, event);
cudaMallocAsync(&ptr2, size, otherStream);
```

4.3.4.4.2 cudaMemPoolReuseAllowOpportunistic

When the `cudaMemPoolReuseAllowOpportunistic` policy is enabled, the allocator examines freed allocations to see if the free operations stream order semantic has been met, for example the stream has passed the point of execution indicated by the free operation. When this policy is disabled, the allocator will still reuse memory made available when a stream is synchronized with the CPU. Disabling this policy does not stop the `cudaMemPoolReuseFollowEventDependencies` from applying.

```
cudaMallocAsync(&ptr, size, originalStream);
kernel<<<..., originalStream>>>(ptr, ...);
cudaFreeAsync(ptr, originalStream);

// after some time, the kernel finishes running
wait(10);

// When cudaMemPoolReuseAllowOpportunistic is enabled this allocation request
// can be fulfilled with the prior allocation based on the progress of
// originalStream.
cudaMallocAsync(&ptr2, size, otherStream);
```

4.3.4.4.3 cudaMemPoolReuseAllowInternalDependencies

Failing to allocate and map more physical memory from the OS, the driver will look for memory whose availability depends on another stream's pending progress. If such memory is found, the driver will insert the required dependency into the allocating stream and reuse the memory.

```
cudaMallocAsync(&ptr, size, originalStream);
kernel<<<..., originalStream>>>(ptr, ...);
cudaFreeAsync(ptr, originalStream);

// When cudaMemPoolReuseAllowInternalDependencies is enabled
// and the driver fails to allocate more physical memory, the driver may
// effectively perform a cudaStreamWaitEvent in the allocating stream
// to make sure that future work in 'otherStream' happens after the work
// in the original stream that would be allowed to access the original
// allocation.
cudaMallocAsync(&ptr2, size, otherStream);
```

4.3.4.4.4 Disabling Reuse Policies

While the controllable reuse policies improve memory reuse, users may want to disable them. Allowing opportunistic reuse (such as `cudaMemPoolReuseAllowOpportunistic`) introduces run to run variance in allocation patterns based on the interleaving of CPU and GPU execution. Internal dependency insertion (such as `cudaMemPoolReuseAllowInternalDependencies`) can serialize work in

unexpected and potentially non-deterministic ways when the user would rather explicitly synchronize an event or stream on allocation failure.

4.3.4.5 Synchronization API Actions

One of the optimizations that comes with the allocator being part of the CUDA driver is integration with the synchronize APIs. When the user requests that the CUDA driver synchronize, the driver waits for asynchronous work to complete. Before returning, the driver will determine what frees the synchronization guaranteed to be completed. These allocations are made available for allocation regardless of specified stream or disabled allocation policies. The driver also checks `cudaMemPoolAttrReleaseThreshold` here and releases any excess physical memory that it can.

4.3.5. Addendums

4.3.5.1 `cudaMemcpyAsync` Current Context/Device Sensitivity

In the current CUDA driver, any async `memcpy` involving memory from `cudaMallocAsync` should be done using the specified stream's context as the calling thread's current context. This is not necessary for `cudaMemcpyPeerAsync`, as the device primary contexts specified in the API are referenced instead of the current context.

4.3.5.2 `cudaPointerGetAttributes` Query

Invoking `cudaPointerGetAttributes` on an allocation after invoking `cudaFreeAsync` on it results in undefined behavior. Specifically, it does not matter if an allocation is still accessible from a given stream: the behavior is still undefined.

4.3.5.3 `cudaGraphAddMemsetNode`

`cudaGraphAddMemsetNode` does not work with memory allocated via the stream ordered allocator. However, memsets of the allocations can be stream captured.

4.3.5.4 Pointer Attributes

The `cudaPointerGetAttributes` query works on stream-ordered allocations. Since stream-ordered allocations are not context associated, querying `CU_POINTER_ATTRIBUTE_CONTEXT` will succeed but return NULL in `*data`. The attribute `CU_POINTER_ATTRIBUTE_DEVICE_ORDINAL` can be used to determine the location of the allocation: this can be useful when selecting a context for making p2h2p copies using `cudaMemcpyPeerAsync`. The attribute `CU_POINTER_ATTRIBUTE_MEMPOOL_HANDLE` was added in CUDA 11.3 and can be useful for debugging and for confirming which pool an allocation comes from before doing IPC.

4.3.5.5 CPU Virtual Memory

When using CUDA stream-ordered memory allocator APIs, avoid setting VRAM limitations with “`ulimit -v`” as this is not supported.

4.4. Cooperative Groups

4.4.1. Introduction

Cooperative Groups are an extension to the CUDA programming model for organizing groups of collaborating threads. Cooperative Groups allow developers to control the granularity at which threads are collaborating, helping them to express richer, more efficient parallel decompositions. Cooperative Groups also provide implementations of common parallel primitives like scan and parallel reduce.

Historically, the CUDA programming model has provided a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a thread block, as implemented with the `__sync_threads()` intrinsic function. In an effort to express broader patterns of parallel interaction, many performance-oriented programmers have resorted to writing their own ad hoc and unsafe primitives for synchronizing threads within a single warp, or across sets of thread blocks running on a single GPU. Whilst the performance improvements achieved have often been valuable, this has resulted in an ever-growing collection of brittle code that is expensive to write, tune, and maintain over time and across GPU generations. Cooperative Groups provides a safe and future-proof mechanism for writing performant code.

The full Cooperative Groups API is available in the [Cooperative Groups API](#).

4.4.2. Cooperative Group Handle & Member Functions

Cooperative Groups are managed via a Cooperative Group Handle. The Cooperative Group handle allows participating threads to learn their position in the group, the group size, and other group information. Select group member functions are shown in the following table.

Table 10: Select Member Functions

Accessor	Returns
<code>thread_rank()</code>	The rank of the calling thread.
<code>num_threads()</code>	The total number of threads in the group .
<code>thread_index()</code>	A 3-Dimensional index of the thread within the launched block.
<code>dim_threads()</code>	The 3D dimensions of the launched block in units of threads.

A complete list of member functions is available in the [Cooperative Groups API](#).

4.4.3. Default Behavior / Groupless Execution

Groups representing the grid and thread blocks are implicitly created based on the kernel launch configuration. These “implicit” groups provide a starting point that developers can explicitly decompose into finer grained groups. Implicit groups can be accessed using the following methods:

Table 11: Cooperative Groups Implicitly Created by CUDA Runtime

Accessor	Group Scope
<code>this_thread_block()</code>	Returns the handle to a group containing all threads in current thread block.
<code>this_grid()</code>	Returns the handle to a group containing all threads in the grid.
<code>coalesced_threads()</code> ¹	Returns the handle to a group of currently active threads in a warp.
<code>this_cluster()</code> ²	Returns the handle to a group of threads in the current cluster.

More information is available in the [Cooperative Groups API](#).

4.4.3.1 Create Implicit Group Handles As Early As Possible

For best performance it is recommended that you create a handle for the implicit group upfront (as early as possible, before any branching has occurred) and use that handle throughout the kernel.

4.4.3.2 Only Pass Group Handles by Reference

It is recommended that you pass group handles by reference to functions when passing a group handle into a function. Group handles must be initialized at declaration time, as there is no default constructor. Copy-constructing group handles is discouraged.

4.4.4. Creating Cooperative Groups

Groups are created by partitioning a parent group into subgroups. When a group is partitioned, a group handle is created to manage the resulting subgroup. The following partitioning operations are available to developers:

Table 12: Cooperative Group Partitioning Operations

Partition Type	Description
<code>tiled_partition</code>	Divides parent group into a series of fixed-size subgroups arranged in a one-dimensional, row-major format.
<code>labeled_partition</code>	Divides parent group into one-dimensional subgroups based on a conditional label, which can be any integral type.
<code>binary_partition</code>	Specialized form of labeled partitioning where label can only be “0” or “1”.

The following example shows how a tiled partition is created:

```
namespace cg = cooperative_groups;
// Obtain the current thread's cooperative group
```

(continues on next page)

¹ The `coalesced_threads()` operator returns the set of active threads at that point in time, and makes no guarantee about which threads are returned (as long as they are active) or that they will stay coalesced throughout execution.

² The `this_cluster()` assumes a 1x1x1 cluster when a non-cluster grid is launched. Requires Compute Capability 9.0 or greater.

(continued from previous page)

```
cg::thread_block my_group = cg::this_thread_block();

// Partition the cooperative group into tiles of size 8
cg::thread_block_tile<8> my_subgroup = cg::tiled_partition<8>(my_group);

// do work as my_subgroup
```

The best partitioning strategy to use depends on the context. More information is available in the [Cooperative Groups API](#).

4.4.4.1 Avoiding Group Creation Hazards

Partitioning a group is a collective operation and all threads in the group must participate. If the group was created in a conditional branch that not all threads reach, this can lead to deadlocks or data corruption.

4.4.5. Synchronization

Prior to the introduction of Cooperative Groups, the CUDA programming model only allowed synchronization between thread blocks at a kernel completion boundary. Cooperative groups allows developers to synchronize groups of cooperating threads at different granularities.

4.4.5.1 Sync

You can synchronize a group by calling the collective `sync()` function. Like `__syncthreads()`, the `sync()` function makes the following guarantees:

- ▶ All memory accesses (e.g., reads and writes) made by threads in the group before the synchronization point are visible to all threads in the group after the synchronization point.
- ▶ All threads in the group reach the synchronization point before any thread is allowed to proceed beyond it.

The following example shows a `cooperative_groups::sync()` that is equivalent to `__syncthreads()`.

```
namespace cg = cooperative_groups;

cg::thread_block my_group = cg::this_thread_block();

// Synchronize threads in the block
cg::sync(my_group);
```

Cooperative groups can be used to synchronize the entire grid. As of CUDA 13, cooperative groups can no longer be used for multi-device synchronization. For details see the [Large Scale Groups](#) section.

More information about synchronization is available in the [Cooperative Groups API](#).

4.4.5.2 Barriers

Cooperative Groups provides a barrier API similar to `cuda::barrier` that can be used for more advanced synchronization. Cooperative Groups barrier API differs from `cuda::barrier` in a few key ways:

- Cooperative Groups barriers are automatically initialized
- All threads in the group must arrive and wait at the barrier once per phase.
- `barrier_arrive` returns an `arrival_token` object that must be passed into the corresponding `barrier_wait`, where it is consumed and cannot be used again.

Programmers must take care to avoid hazards when using Cooperative Groups barriers:

- No collective operations can be used by a group after calling `barrier_arrive` and before calling `barrier_wait`.
- `barrier_wait` only guarantees that all threads in the group have called `barrier_arrive`. `barrier_wait` does NOT guarantee that all threads have called `barrier_wait`.

```
namespace cg = cooperative_groups;

cg::thread_block my_group = this_block();
cg::cluster_group cluster = this_cluster();

auto token = cluster.barrier_arrive();

// Optional: Do some local processing to hide the synchronization latency
    local_processing(my_group);

// Make sure all other blocks in the cluster are running and initialized shared
// data before accessing dsmem
cluster.barrier_wait(std::move(token));
```

4.4.6. Collective Operations

Cooperative Groups includes a set of collective operations that can be performed by a group of threads. These operations require participation of all threads in the specified group in order to complete the operation.

All threads in the group must pass the same values for corresponding arguments to each collective call, unless different values are explicitly allowed in the *Cooperative Groups API*. Otherwise the behavior of the call is undefined.

4.4.6.1 Reduce

The `reduce` function is used to perform a parallel reduction on the data provided by each thread in the specified group. The type of reduction must be specified by providing one of the operators shown in the following table.

Table 13: Cooperative Groups Reduction Operators

Operator	Returns
plus	Sum of all values in group
less	Minimum value
greater	Maximum value

continues on next page

Table 13 – continued from previous page

Operator	Returns
bit_and	Bitwise AND reduction
bit_or	Bitwise OR reduction
bit_xor	Bitwise XOR reduction

Hardware acceleration is used for reductions when available (requires Compute Capability 8.0 or greater). A software fallback is available for older hardware where hardware acceleration is not available. Only 4B types are accelerated by hardware.

More information about reductions is available in the [Cooperative Groups API](#).

The following example shows how to use `cooperative_groups::reduce()` to perform a block-wide sum reduction.

```
namespace cg = cooperative_groups;

cg::thread_block my_group = cg::this_thread_block();

int val = data[threadIdx.x];

int sum = cg::reduce(my_group, val, cg::plus<int>());

// Store the result from the reduction
if (my_group.thread_rank() == 0) {
    result[blockIdx.x] = sum;
}
```

4.4.6.2 Scans

Cooperative Groups includes implementations of `inclusive_scan` and `exclusive_scan` that can be used on arbitrary group sizes. The functions perform a scan operation on the data provided by each thread named in the specified group.

Programmers can optionally specify a reduction operator, as listed in [Reduction Operators Table](#) above.

```
namespace cg = cooperative_groups;

cg::thread_block my_group = cg::this_thread_block();

int val = data[my_group.thread_rank()];

int exclusive_sum = cg::exclusive_scan(my_group, val, cg::plus<int>());

result[my_group.thread_rank()] = exclusive_sum;
```

More information about scans is available in the [Cooperative Groups Scan API](#).

4.4.6.3 Invoke One

Cooperative Groups provides an `invoke_one` function for use when a single thread must perform a serial portion of work on behalf of a group.

- ▶ `invoke_one` selects a single arbitrary thread from the calling group and uses that thread to call the supplied invocable function using the supplied arguments.
- ▶ `invoke_one_broadcast` is the same as `invoke_one` except the result of the call is also broadcast to all threads in the group.

The thread selection mechanism is not guaranteed to be deterministic.

The following example shows basic `invoke_one` utilization.

```
namespace cg = cooperative_groups;
cg::thread_block my_group = cg::this_thread_block();

// Ensure only one thread in the thread block prints the message
cg::invoke_one(my_group, []() {
    printf("Hello from one thread in the block!");
});

// Synchronize to make sure all threads wait until the message is printed
cg::sync(my_group);
```

Communication or synchronization within the calling group is not allowed inside the invocable function. Communication with threads outside of the calling group is allowed.

4.4.7. Asynchronous Data Movement

Cooperative Groups `memcpy_async` functionality in CUDA provides a way to perform asynchronous memory copies between global memory and shared memory. `memcpy_async` is particularly useful for optimizing memory transfers and overlapping computation with data transfer to improve performance.

The `memcpy_async` function is used to start an asynchronous load from global memory to shared memory. `memcpy_async` is intended to be used like a “prefetch” where data is loaded before it is needed.

The `wait` function forces all threads in a group to wait until the asynchronous memory transfer is completed. `wait` must be called by all threads in the group before the data can be accessed in shared memory.

The following example shows how to use `memcpy_async` and `wait` to prefetch data.

```
namespace cg = cooperative_groups;

cg::thread_group my_group = cg::this_thread_block();

__shared__ int shared_data[];

// Perform an asynchronous copy from global memory to shared memory
cg::memcpy_async(my_group, shared_data + my_group.rank(), input + my_group.
    ↪rank(), sizeof(int));
```

(continues on next page)

(continued from previous page)

```
// Hide latency by doing work here. Cannot use shared_data

// Wait for the asynchronous copy to complete
cg::wait(my_group);

// Prefetched data is now available
```

See the [Cooperative Groups API](#) for more information.

4.4.7.1 Memcpy Async Alignment Requirements

`memcpy_async` is only asynchronous if the source is global memory and the destination is shared memory and both are at least 4-byte aligned. For achieving best performance: an alignment of 16 bytes for both shared memory and global memory is recommended.

4.4.8. Large Scale Groups

Cooperative Groups allows for large groups that span the entire grid. All Cooperative Group functionality described previously is available to these large groups, with one notable exception: synchronizing the entire grid requires using the `cudaLaunchCooperativeKernel` runtime launch API.

Multi-device launch APIs and related references for Cooperative Groups have been removed as of CUDA 13.

4.4.8.1 When to use `cudaLaunchCooperativeKernel`

`cudaLaunchCooperativeKernel` is a CUDA runtime API function used to launch a single-device kernel that employs cooperative groups, specifically designed for executing kernels that require inter-block synchronization. This function ensures that all threads in the kernel can synchronize and cooperate across the entire grid, which is not possible with traditional CUDA kernels that only allow synchronization within individual thread blocks. `cudaLaunchCooperativeKernel` ensures that the kernel launch is atomic, i.e. if the API call succeeds, then the provided number of thread blocks will launch on the specified device.

It is good practice to first ensure the device supports cooperative launches by querying the device attribute `cudaDevAttrCooperativeLaunch`:

```
int dev = 0;
int supportsCoopLaunch = 0;
cudaDeviceGetAttribute(&supportsCoopLaunch, cudaDevAttrCooperativeLaunch,
    ↪ dev);
```

which will set `supportsCoopLaunch` to 1 if the property is supported on device 0. Only devices with compute capability of 6.0 and higher are supported. In addition, you need to be running on either of these:

- ▶ The Linux platform without MPS
- ▶ The Linux platform with MPS and on a device with compute capability 7.0 or higher
- ▶ The latest Windows platform

4.5. Programmatic Dependent Launch and Synchronization

The *Programmatic Dependent Launch* mechanism allows for a dependent *secondary* kernel to launch before the *primary* kernel it depends on in the same CUDA stream has finished executing. Available starting with devices of compute capability 9.0, this technique can provide performance benefits when the *secondary* kernel can complete significant work that does not depend on the results of the *primary* kernel.

4.5.1. Background

A CUDA application utilizes the GPU by launching and executing multiple kernels on it. A typical GPU activity timeline is shown in [Figure 42](#).

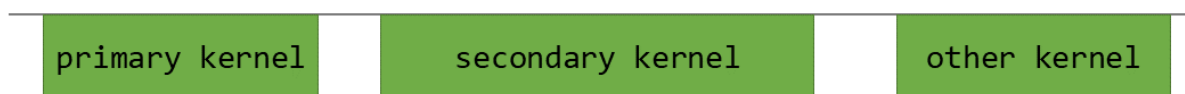


Figure 42: GPU activity timeline

Here, `secondary_kernel` is launched after `primary_kernel` finishes its execution. Serialized execution is usually necessary because `secondary_kernel` depends on result data produced by `primary_kernel`. If `secondary_kernel` has no dependency on `primary_kernel`, both of them can be launched concurrently by using [CUDA Streams](#). Even if `secondary_kernel` is dependent on `primary_kernel`, there is some potential for concurrent execution. For example, almost all the kernels have some sort of *preamble* section during which tasks such as zeroing buffers or loading constant values are performed.

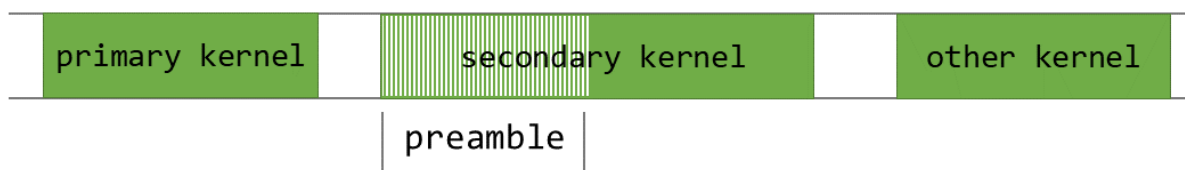
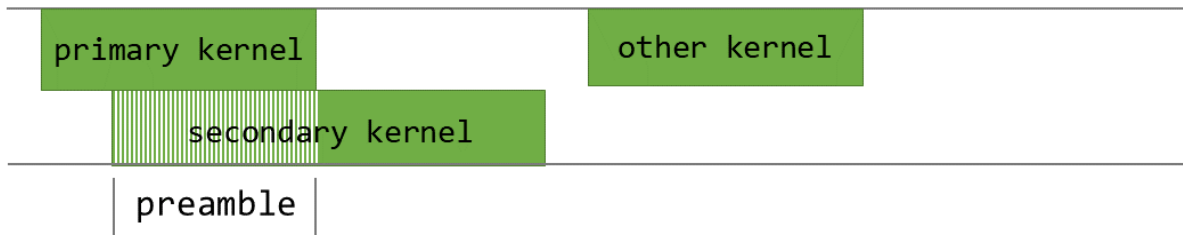


Figure 43: Preamble section of `secondary_kernel`

[Figure 43](#) demonstrates the portion of `secondary_kernel` that could be executed concurrently without impacting the application. Note that concurrent launch also allows us to hide the launch latency of `secondary_kernel` behind the execution of `primary_kernel`.

The concurrent launch and execution of `secondary_kernel` shown in [Figure 44](#) is achievable using *Programmatic Dependent Launch*.

Programmatic Dependent Launch introduces changes to the CUDA kernel launch APIs as explained in following section. These APIs require at least compute capability 9.0 to provide overlapping execution.

Figure 44: Concurrent execution of `primary_kernel` and `secondary_kernel`

4.5.2. API Description

In Programmatic Dependent Launch, a primary and a secondary kernel are launched in the same CUDA stream. The primary kernel should execute `cudaTriggerProgrammaticLaunchCompletion` with all thread blocks when it's ready for the secondary kernel to launch. The secondary kernel must be launched using the extensible launch API as shown.

```
__global__ void primary_kernel() {
    // Initial work that should finish before starting secondary kernel

    // Trigger the secondary kernel
    cudaTriggerProgrammaticLaunchCompletion();

    // Work that can coincide with the secondary kernel
}

__global__ void secondary_kernel()
{
    // Independent work

    // Will block until all primary kernels the secondary kernel is dependent
    // on have completed and flushed results to global memory
    cudaGridDependencySynchronize();

    // Dependent work
}

cudaLaunchAttribute attribute[1];
attribute[0].id = cudaLaunchAttributeProgrammaticStreamSerialization;
attribute[0].val.programmaticStreamSerializationAllowed = 1;
configSecondary.attrs = attribute;
configSecondary.numAttrs = 1;

primary_kernel<<<grid_dim, block_dim, 0, stream>>>();
cudaLaunchKernelEx(&configSecondary, secondary_kernel);
```

When the secondary kernel is launched using the `cudaLaunchAttributeProgrammaticStreamSerialization` attribute, the CUDA driver is safe to launch the secondary kernel early and not wait on the completion and memory flush of the primary before launching the secondary.

The CUDA driver can launch the secondary kernel when all primary thread blocks have launched and executed `cudaTriggerProgrammaticLaunchCompletion`. If the primary kernel doesn't execute the trigger, it implicitly occurs after all thread blocks in the primary kernel exit.

In either case, the secondary thread blocks might launch before data written by the primary kernel is visible. As such, when the secondary kernel is configured with *Programmatic Dependent Launch*, it must always use `cudaGridDependencySynchronize` or other means to verify that the result data from the primary is available.

Please note that these methods provide the opportunity for the primary and secondary kernels to execute concurrently, however this behavior is opportunistic and not guaranteed to lead to concurrent kernel execution. Reliance on concurrent execution in this manner is unsafe and can lead to deadlock.

4.5.3. Use in CUDA Graphs

Programmatic Dependent Launch can be used in *CUDA Graphs* via *stream capture* or directly via *edge data*. To program this feature in a CUDA Graph with edge data, use a `cudaGraphDependencyType` value of `cudaGraphDependencyTypeProgrammatic` on an edge connecting two kernel nodes. This edge type makes the upstream kernel visible to a `cudaGridDependencySynchronize()` in the downstream kernel. This type must be used with an outgoing port of either `cudaGraphKernelNodePortLaunchCompletion` or `cudaGraphKernelNodePortProgrammatic`.

The resulting graph equivalents for stream capture are as follows:

Stream code (abbreviated)	Resulting graph edge
<pre> cudaLaunchAttribute attribute; attribute.id = ↪ cudaLaunchAttributeProgrammaticStreamCapture; ↪ attribute.val. ↪ programmaticStreamSerializationAllowance = 1; </pre>	<pre> cudaGraphEdgeData edgeData; edgeData.type = ↪ cudaGraphDependencyTypeProgrammatic; ↪ edgeData.from_port = ↪ cudaGraphKernelNodePortProgrammatic; ↪ </pre>
<pre> cudaLaunchAttribute attribute; attribute.id = ↪ cudaLaunchAttributeProgrammaticEventTriggerAtBlockStart; ↪ attribute.val.programmaticEvent. ↪ triggerAtBlockStart = 0; </pre>	<pre> cudaGraphEdgeData edgeData; edgeData.type = ↪ cudaGraphDependencyTypeProgrammatic; ↪ edgeData.from_port = ↪ cudaGraphKernelNodePortProgrammatic; ↪ </pre>
<pre> cudaLaunchAttribute attribute; attribute.id = ↪ cudaLaunchAttributeProgrammaticEventTriggerAtBlockStart; ↪ attribute.val.programmaticEvent. ↪ triggerAtBlockStart = 1; </pre>	<pre> cudaGraphEdgeData edgeData; edgeData.type = ↪ cudaGraphDependencyTypeProgrammatic; ↪ edgeData.from_port = ↪ cudaGraphKernelNodePortLaunchCompletion; ↪ </pre>

4.6. Green Contexts

A green context (GC) is a lightweight context associated, from its creation, with a set of specific GPU resources. Users can partition GPU resources, currently streaming multiprocessors (SMs) and work queues (WQs), during green context creation, so that GPU work targeting a green context can only use its provisioned SMs and work queues. Doing so can be beneficial in reducing, or better controlling, interference due to use of common resources. An application can have multiple green contexts.

Using green contexts does not require any GPU code (kernel) changes, just small host-side changes (e.g., green context creation and stream creation for this green context). The green context functionality can be useful in various scenarios. For example, it can help ensure some SMs are always available for a latency-sensitive kernel to start executing, assuming no other constraints, or provide a quick way to test the effect of using fewer SMs without any kernel modifications.

Green context support first became available via the [CUDA Driver API](#). Starting from CUDA 13.1, contexts are exposed in the CUDA runtime via the execution context (EC) abstraction. Currently, an execution context can correspond to either the primary context (the context runtime API users have always implicitly interacted with) or a green context. This section will use the terms *execution context* and *green context* interchangeably when referring to a green context.

With the runtime exposure of green contexts, using the CUDA runtime API directly is strongly recommended. This section will also solely use the CUDA runtime API.

The remaining of this section is organized as follows: [Section 4.6.1](#) provides a motivating example, [Section 4.6.2](#) highlights ease of use, and [Section 4.6.3](#) presents the device resource and resource descriptor structs. [Section 4.6.4](#) explains how to create a green context, [Section 4.6.5](#) how to launch work that targets it, and [Section 4.6.6](#) highlights some additional green context APIs. Finally, [Section 4.6.7](#) wraps up with an example.

4.6.1. Motivation / When to Use

When launching a CUDA kernel, the user has no direct control over the number of SMs that kernel will execute on. One can only indirectly influence this by changing the kernel's launch geometry or anything that can affect the kernel's maximum number of active thread blocks per SM. Additionally, when multiple kernels execute in parallel on the GPU (kernels running on different CUDA streams or as part of a CUDA graph), they may also contend for the same SM resources.

There are, however, use cases where the user needs to ensure there are always GPU resources available for latency-sensitive work to start, and thus complete, as soon as possible. Green contexts provide a way towards that by partitioning SM resources, so a given green context can only use specific SMs (the ones provisioned during its creation).

[Figure 45](#) illustrates such an example. Assume an application where two independent kernels A and B run on two different non-blocking CUDA streams. Kernel A is launched first and starts executing occupying all available SM resources. When, later in time, latency-sensitive kernel B is launched, no SM resources are available. As a result, kernel B can only start executing once kernel A ramps down, i.e., once thread blocks from kernel A finish executing. The first graph illustrates this scenario where critical work B gets delayed. The y-axis shows the percentage of SMs occupied and x-axis depicts time.

Using green contexts, one could partition the GPU's SMs, so that green context A, targeted by kernel A, has access to some SMs of the GPU, while green context B, targeted by kernel B, has access to the remaining SMs. In this setting, kernel A can only use the SMs provisioned for green context A, irrespective of its launch configuration. As a result, when critical kernel B gets launched, it is guaranteed that there will be available SMs for it to start executing immediately, barring any other resource constraints. As the second graph in [Figure 45](#) illustrates, even though the duration of kernel A may

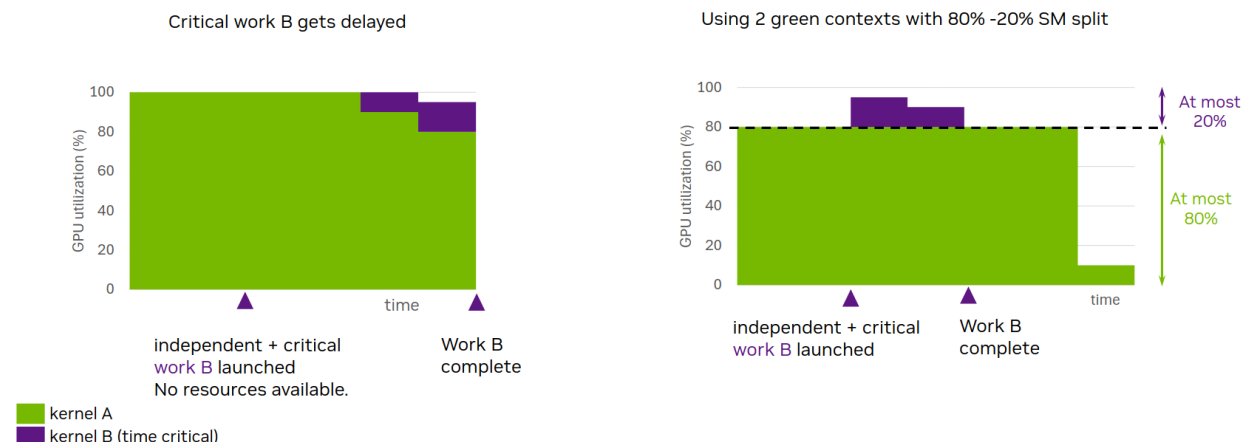


Figure 45: Motivation: GCs' static resource partitioning enables latency-sensitive work B to start and complete sooner

increase, latency-sensitive work B will no longer be delayed due to unavailable SMs. The figure shows that green context A is provisioned with an SM count equivalent to 80% SMs of the GPU for illustration purposes.

This behavior can be achieved without any code modifications to kernels A and B. One simply needs to ensure they are launched on CUDA streams belonging to the appropriate green contexts. The number of SMs each green context will have access to should be decided by the user during green context creation on a per case basis.

Work Queues:

Streaming multiprocessors are one resource type that can be provisioned for a green context. Another resource type is work queues. Think of a workqueue as a black-box resource abstraction, which can also influence GPU work execution concurrency, along with other factors. If independent GPU work tasks (e.g., kernels submitted on different CUDA streams) map to the same workqueue, a false dependence between these tasks may be introduced, which can lead to their serialized execution. The user can influence the upper limit of work queues on the GPU via the `CUDA_DEVICE_MAX_CONNECTIONS` environment variable (see [Section 5.2](#), [Section 3.1](#)).

Building on top of the previous example, assume work B maps to the same workqueue as work A. In that case, even if SM resources are available (green contexts case), work B may still need to wait for work A to complete in its entirety. Similar to SMs, the user has no direct control over the specific work queues that may be used under the hood. But green contexts allow the user to express the maximum concurrency they would expect in terms of expected number of concurrent stream-ordered workloads. The driver can then use this value as a hint to try to prevent work from different execution contexts from using the same workqueue(s), thus preventing unwanted interference across execution contexts.

Attention

Even when different SM resources and work queues are provisioned per green context, concurrent execution of independent GPU work is not guaranteed. It is best to think of all the techniques described under the [Green Contexts](#) section as removing factors which can prevent concurrent execution (i.e., reducing potential interference).

Green Contexts versus MIG or MPS

For completeness, this section briefly compares green contexts with two other resource partitioning mechanisms: [MIG \(Multi-Instance GPU\)](#) and [MPS \(Multi-Process Service\)](#).

MIG statically partitions a MIG-supported GPU into multiple MIG instances (“smaller GPUs”). This partitioning has to happen before the launch of an application, and different applications can use different MIG instances. Using MIG can be beneficial for users whose applications consistently underutilize the available GPU resources; an issue more pronounced as GPUs get bigger. With MIG, users can run these different applications on different MIG instances, thus improving GPU utilization. MIG can be attractive for cloud service providers (CSPs) not only for the increased GPU utilization for such applications, but also for the quality of service (QoS) and isolation it can provide across clients running on different MIG instances. Please refer to the MIG documentation linked above for more details.

But using MIG cannot address the problematic scenario described earlier, where critical work B is delayed because all SM resources are occupied by other GPU work from the same application. This issue can still exist for an application running on a single MIG instance. To address it, one can use green contexts alongside MIG. In that case, the SM resources available for partitioning would be the resources of the given MIG instance.

MPS primarily targets different processes (e.g., MPI programs), allowing them to run on the GPU at the same time without time-slicing. It requires an MPS daemon to be running before the application is launched. By default, MPS clients will contend for all available SM resources of the GPU or the MIG instance they are running on. In this multiple client processes setting, MPS can support dynamic partitioning of SM resources, using the *active thread percentage* option, which places an upper limit on the percentage of SMs an MPS client process can use. Unlike green contexts, the active thread percentage partitioning happens with MPS at the process level, and the percentage is typically specified by an environment variable before the application is launched. The MPS active thread percentage signifies that a given client application cannot use more than $x\%$ of a GPU’s SMs, let that be N SMs. However, these SMs can be *any* N SMs of the GPU, which can also vary over time. On the other hand, a green context provisioned with N SMs during its creation can only use these specific N SMs.

Starting with CUDA 13.1, MPS also supports static partitioning, if it is explicitly enabled when starting the MPS control daemon. With static partitioning, the user has to specify the static partition an MPS client process can use, when the application is launched. Dynamic sharing with active thread percentage is no longer applicable in that case. A key difference between MPS in static partitioning mode and green contexts is that MPS targets different processes, while green contexts is applicable within a single process too. Also, contrary to green contexts, MPS with static partitioning does not allow oversubscription of SM resources.

With MPS, programmatic partitioning of SM resources is also possible for a CUDA context created via the `cuCtxCreate` driver API, with execution affinity. This programmatic partitioning allows different client CUDA contexts from one or more processes to each use up to a specified number of SMs. As with the active thread percentage partitioning, these SMs can be *any* SMs of the GPU and can vary over time, unlike the green contexts case. This option is possible even under the presence of static MPS partitioning. Please note that creating a green context is much more lightweight in comparison to an MPS context, as many underlying structures are owned by the primary context and thus shared.

4.6.2. Green Contexts: Ease of use

To highlight how easy it is to use green contexts, assume you have the following code snippet that creates two CUDA streams and then calls a function that launches kernels via `<<<>>>` on these CUDA streams. As discussed earlier, other than changing the kernels’ launch geometries, one cannot influence how many SMs these kernels can use.

```
int gpu_device_index = 0; // GPU ordinal
CUDA_CHECK(cudaSetDevice(gpu_device_index));
```

(continues on next page)

(continued from previous page)

```

cudaStream_t strm1, strm2;
CUDA_CHECK(cudaStreamCreateWithFlags(&strm1, cudaStreamNonBlocking));
CUDA_CHECK(cudaStreamCreateWithFlags(&strm2, cudaStreamNonBlocking));

// No control over how many SMs kernel(s) running on each stream can use
code_that_launches_kernels_on_streams(strm1, strm2); // what is abstracted in
→this function + the kernels is the vast majority of your code

// cleanup code not shown

```

Starting with CUDA 13.1, one can control the number of SMs a given kernel can have access to, using green contexts. The code snippet below shows how easy it is to do that. With a few extra lines and without any kernel modifications, you can control the SMs resources kernel(s) launched on these different streams can use.

```

int gpu_device_index = 0; // GPU ordinal
CUDA_CHECK(cudaSetDevice(gpu_device_index));

/* ----- Code required to create green contexts -----
→----- */

// Get all available GPU SM resources
cudaDevResource initial_GPU_SM_resources {};
CUDA_CHECK(cudaDeviceGetDevResource(gpu_device_index, &initial_GPU_SM_
→resources, cudaDevResourceTypeSm));

// Split SM resources. This example creates one group with 16 SMs and one with
→8. Assuming your GPU has >= 24 SMs
cudaDevSmResource result[2] {{}, {}};
cudaDevSmResourceGroupParams group_params[2] = {
    {.smCount=16, .coscheduledSmCount=0, .preferredCoscheduledSmCount=0, .
→flags=0},
    {.smCount=8, .coscheduledSmCount=0, .preferredCoscheduledSmCount=0, .
→flags=0}};
CUDA_CHECK(cudaDevSmResourceSplit(&result[0], 2, &initial_GPU_SM_resources,
→nullptr, 0, &group_params[0]));

// Generate resource descriptors for each resource
cudaDevResourceDesc_t resource_desc1 {};
cudaDevResourceDesc_t resource_desc2 {};
CUDA_CHECK(cudaDevResourceGenerateDesc(&resource_desc1, &result[0], 1));
CUDA_CHECK(cudaDevResourceGenerateDesc(&resource_desc2, &result[1], 1));

// Create green contexts
cudaExecutionContext_t my_green_ctx1 {};
cudaExecutionContext_t my_green_ctx2 {};
CUDA_CHECK(cudaGreenCtxCreate(&my_green_ctx1, resource_desc1, gpu_device_
→index, 0));
CUDA_CHECK(cudaGreenCtxCreate(&my_green_ctx2, resource_desc2, gpu_device_
→index, 0));

```

(continues on next page)

(continued from previous page)

```

/* ----- Modified code ----- */

// You just need to use a different CUDA API to create the streams
cudaStream_t strm1, strm2;
CUDA_CHECK(cudaExecutionCtxStreamCreate(&strm1, my_green_ctx1,
    ↪ cudaStreamDefault, 0));
CUDA_CHECK(cudaExecutionCtxStreamCreate(&strm2, my_green_ctx2,
    ↪ cudaStreamDefault, 0));

/* ----- Unchanged code ----- */

// No need to modify any code in this function or in your kernel(s).
// Reminder: what is abstracted in this function + kernels is the vast majority
    ↪ of your code
// Now kernel(s) running on stream strm1 will use at most 16 SMs and kernel(s)
    ↪ on strm2 at most 8 SMs.
code_that_launches_kernels_on_streams(strm1, strm2);

// cleanup code not shown

```

Various execution context APIs, some of which were shown in the previous example, take an explicit `cudaExecutionContext_t` handle and thus ignore the context that is current to the calling thread. Until now, CUDA runtime users who did not use the driver API would by default only interact with the primary context that is implicitly set as current to a thread via `cudaSetDevice()`. This shift to explicit context-based programming provides easier to understand semantics and can have additional benefits compared to the previous implicit context-based programming that relied on thread-local state (TLS).

The following sections will explain all the steps shown in the previous code snippet in detail.

4.6.3. Green Contexts: Device Resource and Resource Descriptor

At the heart of a green context is a device resource (`cudaDevResource`) tied to a specific GPU device. Resources can be combined and encapsulated into a descriptor (`cudaDevResourceDesc_t`). A green context only has access to the resources encapsulated into the descriptor used for its creation.

Currently the `cudaDevResource` data structure is defined as:

```

struct {
    enum cudaDevResourceType type;
    union {
        struct cudaDevSmResource sm;
        struct cudaDevWorkqueueConfigResource wqConfig;
        struct cudaDevWorkqueueResource wq;
    };
};

```

The supported valid resource types are `cudaDevResourceTypeSm`, `cudaDevResourceTypeWorkqueueConfig` and `cudaDevResourceTypeWorkqueue`, while `cudaDevResourceTypeInvalid` identifies an invalid resource type.

A valid device resource can be associated with:

- ▶ a specific set of streaming multiprocessors (SMs) (resource type `cudaDevResourceTypeSm`),
- ▶ a specific workqueue configuration (resource type `cudaDevResourceTypeWorkqueueConfig`) or
- ▶ a pre-existing workqueue resource (resource type `cudaDevResourceTypeWorkqueue`).

One can query if a given execution context or CUDA stream is associated with a `cudaDevResource` resource of a given type, using the `cudaExecutionContextGetDevResource` and `cudaStreamGetDevResource` APIs respectively. Being associated with different types of device resources (e.g., SMs and work queues) is also possible for an execution context, while a stream can only be associated with an SM-type resource.

A given GPU device has, by default, all three device resource types: an SM-type resource encompassing all the SMs of the GPU, a workqueue configuration resource encompassing all available work queues and its corresponding workqueue resource. These resources can be retrieved via the `cudaDeviceGetDevResource` API.

Overview of relevant device resource structs

The different resource type structs have fields that are set either explicitly by the user or by a relevant CUDA API call. It is recommended to zero-initialize all device resource structs.

- ▶ An SM-type device resource (`cudaDevSmResource`) has the following relevant fields:
 - ▶ `unsigned int smCount`: number of SMs available in this resource
 - ▶ `unsigned int minSmPartitionSize`: minimum SM count required to partition this resource
 - ▶ `unsigned int smCoscheduledAlignment`: number of SMs in the resource guaranteed to be co-scheduled on the same GPU processing cluster, which is relevant for thread block clusters. `smCount` is a multiple of this value when `flags` is zero.
 - ▶ `unsigned int flags`: supported flags are 0 (default) and `cudaDevSmResourceGroupBackfill` (see `cudaDevSmResourceGroup` flags).

The above fields will be set via either the appropriate split API (`cudaDevSmResourceSplitByCount` or `cudaDevSmResourceSplit`) used to create this SM-type resource or will be populated by the `cudaDeviceGetDevResource` API which retrieves the SM resources of a given GPU device. These fields should never be set directly by the user. See next section for more details.

- ▶ A workqueue configuration device resource (`cudaDevWorkqueueConfigResource`) has the following relevant fields:
 - ▶ `int device`: the device on which the workqueue resources are available
 - ▶ `unsigned int wqConcurrencyLimit`: the number of stream-ordered workloads expected to avoid false dependencies
 - ▶ `enum cudaDevWorkqueueConfigScope sharingScope`: the sharing scope for the workqueue resources. Supported values are: `cudaDevWorkqueueConfigScopeDeviceCtx` (default) and `cudaDevWorkqueueConfigScopeGreenCtxBalanced`. With the default option, all workqueue resources are shared across all contexts, while with the balanced option the driver tries to use non-overlapping workqueue resources across green contexts whenever possible, using the user-specified `wqConcurrencyLimit` as a hint.

These fields need to be set by the user. There is no CUDA API similar to the split APIs that generates a workqueue configuration resource, with the exception of the workqueue configu-